

# 基于库存价值与滚动决策的微网购电调控

## 摘要

微网储能既能利用峰谷价差转移用电，也能在供需偏差出现后调整放电时机。针对计划购电、实时补购及日内调单之间的耦合关系，本文以总购电费用最小为目标，将四个问题统一为信息逐步更新下的库存决策问题。

针对问题一，建立**多期线性规划模型**，联立电量平衡、储电状态和功率容量约束，利用凸分段线性的**连续状态价值函数**描述库存的跨期作用。最优日购电量为 59,482.70 kWh，费用 35,126.95 元，较无储能降低 26.90%，对应单日运行灵活性价值 12,925.10 元。

针对问题二，建立**岭回归预测与分位数风险校准模型**，以 70%历史误差分位余量确定日前需求，再通过**历史路径价值控制**将库存分配至后续高价缺口。2—12 月 334 天总费用为 13,991,392.60 元，较相同计划生成规则下的即时放电策略节省 141,219.56 元，期末储电量相同。

针对问题三，建立**非对称结算下的滚动优化模型**，结合**负载收缩回归与月度模型选择**更新供需判断，并将下一次调单机会计入库存价值。最终总费用为 13,479,283.32 元；固定其余条件时，引入 6、12、18 点预报使基础策略节省 1,469,150.15 元，说明日内信息具有明确的运行价值。

针对问题四，建立**同期、岭回归及残差 GRU 电价预测模型**，将预测价格接入日前与滚动决策。GRU 价格 MAE 为 0.04366 元/kWh，较岭回归降低 8.09%；按验证期费用确定价格预测器后，日前与滚动总费用分别为 14,765,492.68 元和 14,210,881.01 元。共同名义供需条件下，GRU 计划距真实价格最优解的费用差占 1.002%。

进一步将库存边际价值解释为储能择时权的机会成本，推导放电条件与容量配置方法。确定性规划、状态递推及逐时账单相互校核，形成从购电计划、实时运行到灵活性评价的完整决策链条。

关键词：微网购电；库存价值；线性规划；分位数校准；滚动优化

# 一 问题重述

## 1.1 研究背景

非孤岛式微网通过外网购电、光伏发电和储能放电共同满足小区负载。光伏富余或电价较低时，储能可吸收电量；负载上升或电价较高时，再将其释放。由于充放电存在损耗，且受功率、容量约束，储能调度需要权衡当前购电支出与后续供电需求。

这一权衡还受到信息和合同的限制。运营者在每日零点申报购电计划时，尚不知道全天真实负载与光伏出力。计划不足可能引发五倍价格的紧急购电，计划过量则产生无法收回的支出。日内预报提供了修订机会，但新增和取消合同各有费用。因此，购电决策既要判断未来需要多少电，也要决定何时使用有限库存。

## 1.2 问题的提出

问题一给定重复发生的电价与负载以及单日光伏预测，要求在零点制定全天购电与储能方案，保证供电不低于负载，且日初、日末储电量相等。

问题二引入逐日变化的负载与光伏，要求仅依靠零点可获得的信息申报当天计划。普通购电按计划量结算，实时缺口按当时电价的五倍补购。

问题三每天在 0、6、12、18 点提供未来 24 小时整点光伏预报，允许调整未执行计划。取消部分按半价付费，新增部分按 1.5 倍价格付费，需要据此制定滚动策略并判断日内更新是否必要。

问题四进一步考虑实时波动电价，重新计算问题二和问题三，比较价格预测、供需预测与储能调度的共同作用。

第二至四问的评价区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。各问均给出指定日期和时段的购电、储能及紧急购电结果，并按题目要求保存五份完整工作簿。

## 二 问题分析

### 2.1 问题一的分析

全天供需和价格已知，购电与储能状态由线性约束连接，适合建立多期线性规划。先求得循环运行的最低费用，再把后续费用写成当前储电量的函数。这样既能解释峰谷套利，也为后续不确定环境中的放电决策提供依据。

### 2.2 问题二的分析

供需不确定性使预测误差具有不对称成本。以日、周周期信息预测负载与光伏，并用历史净负载误差设置购电余量，可在计划层控制欠购风险。计划锁定后，利用历史场景估计后续库存价值，避免在低价缺口中提前耗尽储能。

### 2.3 问题三的分析

新预报使未来合同可以修订，储能的后续价值随之变化。将取消和新增购电分别列式，在每次发布时重解剩余时域；同时根据已观测负载偏差更新预测。实时放电不仅取决于后续缺口，还取决于这些缺口能否在下一次发布后通过普通调单解决。

### 2.4 问题四的分析

价格波动改变了当前放电和保留库存的收益。将未来价格预测纳入原有购电与价值递推模型，在执行时以已揭示的当前价格决策。预测精度与实际费用分别评价，从而辨别价格预测改善是否足以改变受约束的储能动作。

四问的递进关系如图 1。共同的物理状态与费用目标保持一致，变化的是外生信息及合同调整权限。

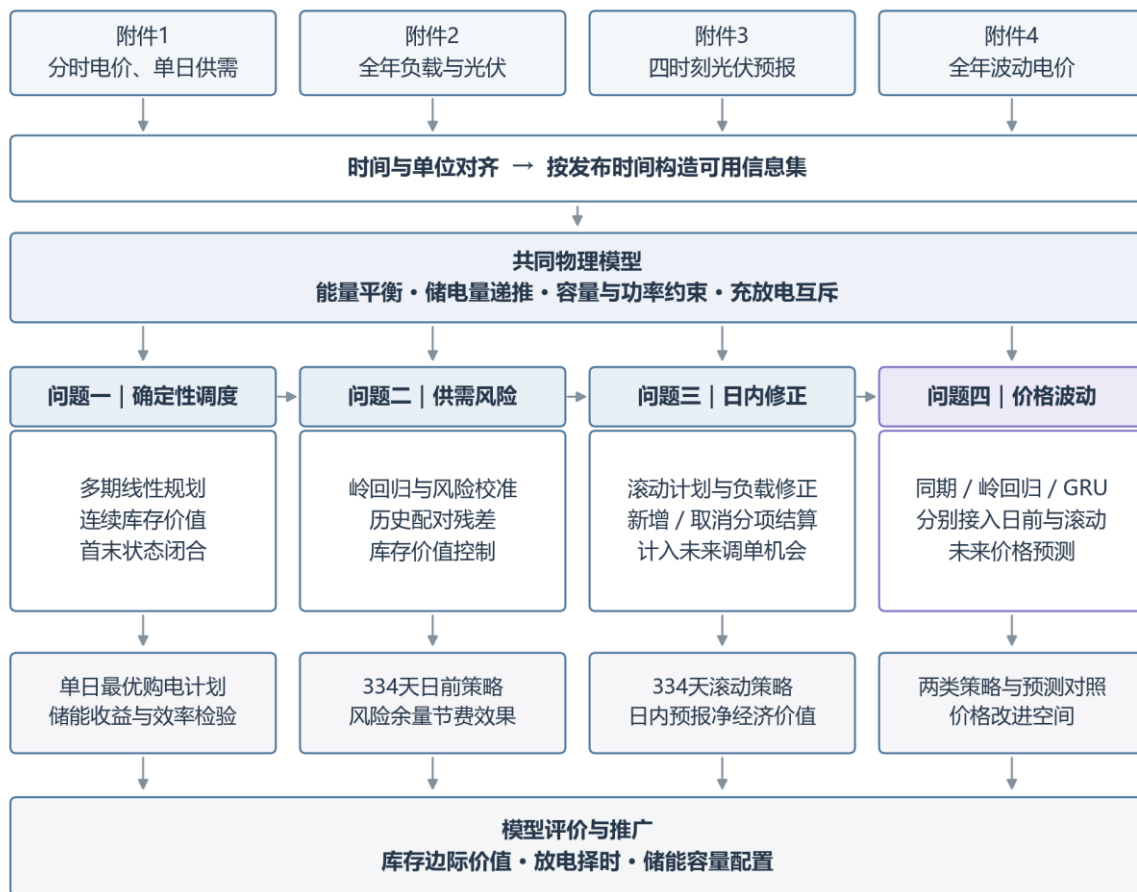


图 1 四问递进与共同决策结构

### 三 模型假设与数据处理

#### 3.1 参数与运行假设

储能参数见表 1。充电、放电效率分别取 0.9；充放电功率定义在微网母线侧。储电量记在时段边界，购电、充放电及负载统一用 kWh 表示。

表 1 储能参数及离散时段

| 参数        | 数值               | 说明        |
|-----------|------------------|-----------|
| 额定容量      | 12,000 kWh       | 题设        |
| 允许储电量     | 1,200—10,800 kWh | 题设        |
| 最大充放电功率   | 5,000 kW         | 母线侧       |
| 初始储电量     | 6,000 kWh        | 1 月 1 日零点 |
| 充电 / 放电效率 | 0.9 / 0.9        | 分别计入      |
| 时段长度      | 1/6 h            | 十分钟       |

全天划分为 144 个十分钟区间。将附件中 00:10 至 24:00 的记录视为相应前十分钟区间的平均功率或结算电价，所有问题采用同一时间口径；区间内按常值计算，功率乘 1/6

小时得到电量。整点光伏预报则视为预测时刻的功率点，以线性插值后的区间积分转换为电量。

微网只购电、不向外网售电；无法被负载或电池吸收的电量允许弃用。实际供需及交易电价在当前区间执行时可见，未来实际值不进入计划。暂不计电池退化、爬坡及开关成本。

第一问采用日周期边界，日初和日末均为 6,000 kWh。第二至四问从 1 月 1 日题设初始状态开始热身，实际储电量逐日传递；名义计划设置 6,000 kWh 的日末下限，以维持次日备用，实际日末库存由运行过程决定。

### 3.2 时间对齐与数据预处理

附件 1 含 144 组单日记录；附件 2 的负载与光伏各含 52,560 个观测；附件 3 包含 1,460 次预报，每次 24 个整点；附件 4 包含 52,560 个电价。按日期、发布时间和目标区间建立索引，保留 0:00+1 所表示的次日零点，防止跨日数据落入错误日期。

原始数值无缺失、负数或非有限值，日期覆盖完整。附件 3 每个日期占四行，其中后三行的空日期仅作标签补齐；光伏零值与极低电价均保留。预处理只统一时间格式和单位，不对真实观测插补、缩尾或平滑。训练标准化和预测非负约束作用于模型输入输出，不改变结算所用的实测数据。

$$L_{d,t} = P_{d,t}^{\text{load}} \Delta t, \quad V_{d,t} = P_{d,t}^{\text{PV}} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

每次历史训练只使用已结束日期。1 月 22—31 日用于参数和执行规则的开发验证，2—12 月按时间顺序运行；后续已观测数据可以进入下一次训练。各方案在评价期使用共同初始库存 9,902.811288 kWh，以统一运行起点。

## 四 符号说明

主要决策变量与状态变量见表 2，各问沿用同一物理量定义。

表 2 主要符号

| 符号                     | 含义            | 单位    |
|------------------------|---------------|-------|
| $L_t, V_t$             | 负载电量、光伏电量     | kWh   |
| $g_t, e_t$             | 普通购电量、紧急购电量   | kWh   |
| $c_t, d_t, w_t$        | 充电、放电、弃电      | kWh   |
| $S_t$                  | 时段开始的内部储电量    | kWh   |
| $p_t$                  | 外网交易电价        | 元/kWh |
| $M$                    | 单时段母线侧充放电上限   | kWh   |
| $x_t, y_t$             | 午夜合同、最终有效合同   | kWh   |
| $u_t, v_t$             | 取消量、新增量       | kWh   |
| $J_t(s), \bar{J}_t(s)$ | 后续费用、场景平均后续费用 | 元     |
| $\mu_t(s)$             | 每单位内部库存的边际价值  | 元/kWh |
| $\mathcal{I}_\tau$     | 发布时刻可获得的信息    | —     |

下文省略不引起歧义的时间下标。 $t$  表示十分钟时段， $\tau$  表示计划或预报的发布时间；实际电量与预测电量分别用原符号及其上方的“帽”区分。

## 五 第一问：确定性购电与库存价值

### 5.1 多期购电模型

单日负载、光伏和价格见图 2。光伏集中于日间，而负载持续存在，单纯按净负载购电无法充分利用峰谷价差。储能把不同时段联系起来：一次充电既增加当前购电量，也改变后续可用于供电的库存。

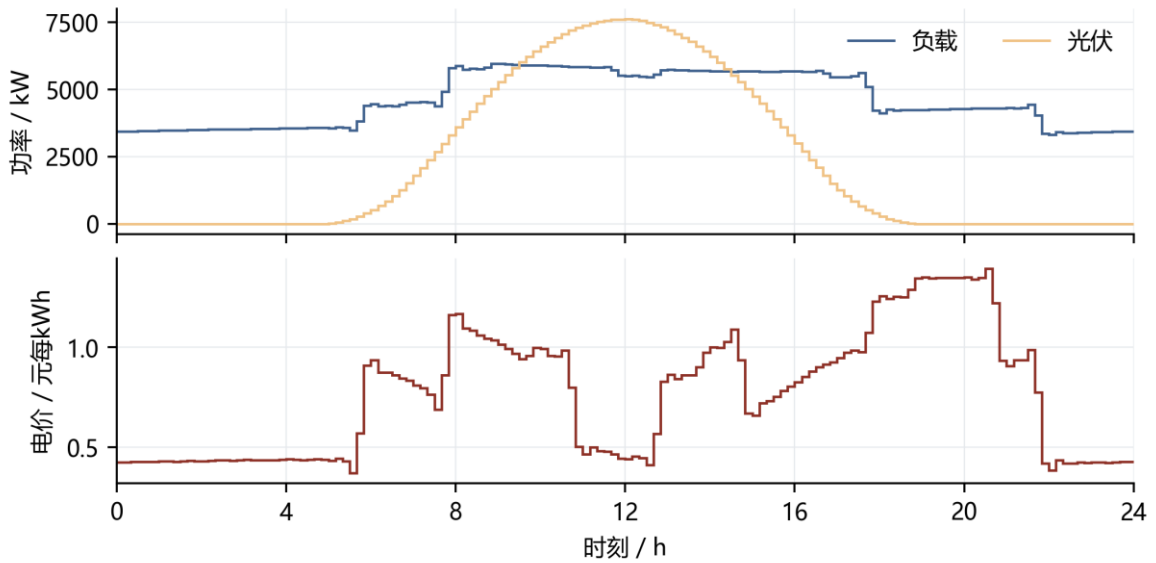


图 2 第一问供需与电价

以全天普通购电费最小为目标，建立以下模型：

$$\begin{aligned}
\min \quad & C_1 = \sum_{t=1}^{144} p_t g_t, \\
\text{s.t.} \quad & g_t + V_t + d_t = L_t + c_t + w_t, \\
& S_{t+1} = S_t + \eta_c c_t - d_t / \eta_d, \\
& S_{\min} \leq S_t \leq S_{\max}, \quad 0 \leq c_t, d_t \leq M, \\
& g_t, w_t \geq 0, \quad c_t d_t = 0, \quad S_1 = S_{145} = 6000.
\end{aligned} \tag{2}$$

其中  $w$  为富余电量，允许供给超过负载和充电需求。 $M=5,000/6=833.33$  kWh 是母线侧单时段充放电上限。状态方程计入双向效率损失，因此同样的外部充电量不能全部回收为可用放电量。

求解时先放松充放电互斥，再消除无效循环。令  $\rho = \eta_c \eta_d$ ，取  $a = \min(c, d/\rho)$ ，将充电、放电、弃电分别改为  $c-a$ 、 $d-\rho a$ 、 $w+(1-\rho)a$ 。该变换保持库存变化、电量平衡和购电费，且至少一种动作归零。因此可由线性规划直接获得同费用的可执行调度。模型使用 HiGHS 求解[1-2]。

## 5.2 最优调度结果

最优日购电量为 59,482.70 kWh，总费用 35,126.95 元，日初、日末库存均为 6,000 kWh。图 3 显示低价时集中购电充电、高价时释放库存的运行结构；指定时段与四小时汇总见表 3、表 4。

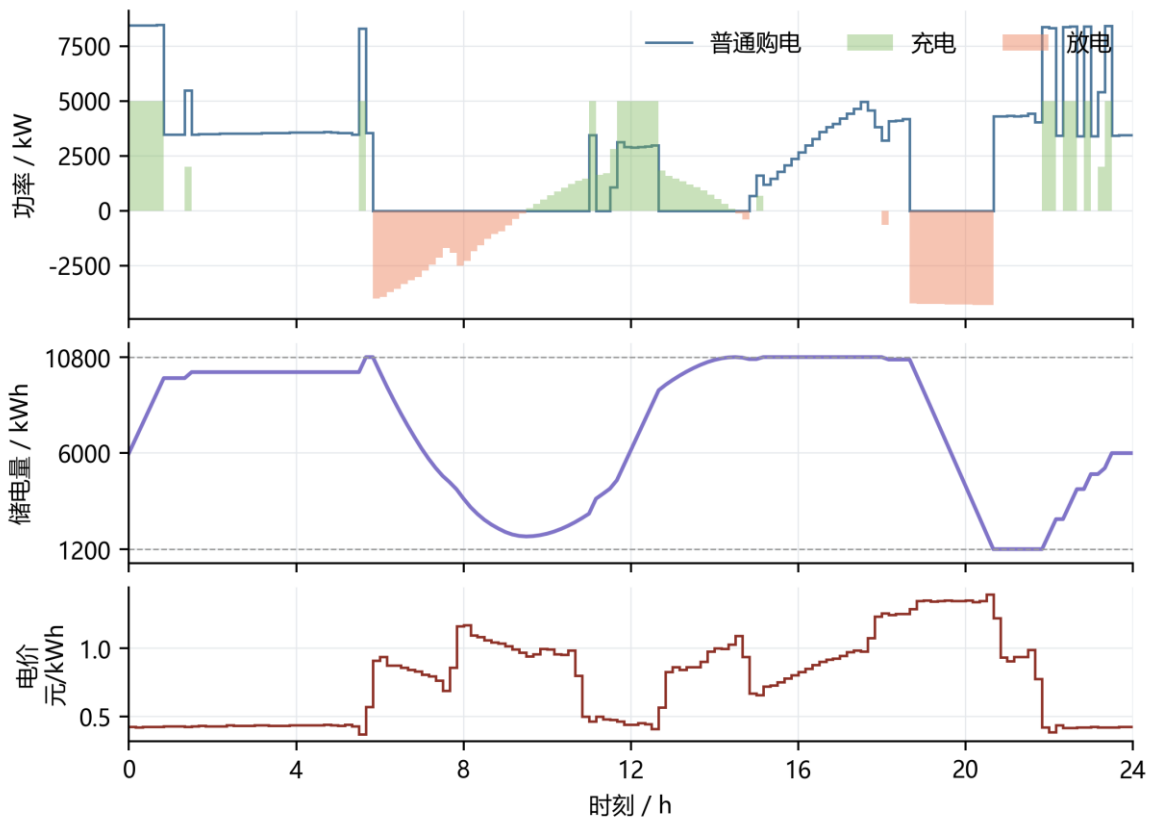


图 3 第一问购电与储能运行

表 3 第一问指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段               | 购电量    | 时间段         | 购电量  |
|---------------------|--------|-------------------|--------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10       | 480.41 | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 445.43 | 18:00-18:10       | 531.89 | 20:00-20:10 | 0.00 |
| 全天购电量 59,482.70 kWh |        | 全天购电费 35,126.95 元 |        |             |      |

表 4 第一问四小时充放电及首末库存

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,500.00 | 0.00     | 04:00-08:00 | 833.33   | 6,365.84 |
| 08:00-12:00 | 4,787.96 | 1,703.00 | 12:00-16:00 | 5,286.04 | 91.10    |
| 16:00-20:00 | 0.00     | 5,780.13 | 20:00-24:00 | 5,333.33 | 2,859.87 |
| 00:00 储电量   | 6,000.00 |          | 24:00 储电量   | 6,000.00 |          |

不使用储能时，每个区间直接购买正净负载，费用为 48,052.05 元。储能使日费用下降 12,925.10 元，即 26.90%。四小时汇总块中可以同时出现充、放电量，这是块内不同时段的动作累积；十分钟动作仍保持互斥。

### 5.3 价值函数及其经济解释

为把这一调度推广到变化的库存状态，令  $J_t(s)$  表示从时段  $t$  开始、当前库存为  $s$  时的最低后续购电费。以  $v=s-s'$  表示电池内部库存的释放量， $n=L-V$  表示净负载，则母线获得的净电量及单时段购电费为：

$$\phi(v) = \begin{cases} \eta_d v, & v \geq 0, \\ v/\eta_c, & v < 0, \end{cases} \quad h_t(v) = p_t \max\{n_t - \phi(v), 0\}. \quad (3)$$

由最优性原理，当前动作与后续费用满足：

$$J_t(s) = \min_{s' \in \mathcal{S}(s)} \{h_t(s-s') + J_{t+1}(s')\}, \quad J_{145}(s) = \begin{cases} 0, & s = 6000, \\ +\infty, & s \neq 6000. \end{cases} \quad (4)$$

集合  $\mathcal{S}(s)$  由库存及充放电上限确定。非负电价和  $\eta_c \eta_d \leq 1$  使  $h$  为凸分段线性函数；递推中的下确界卷积与可行区间限制保持凸性[7]。因此  $J$  可用有限折点表示，逐段归并斜率即可完成连续状态递推，无需离散库存网格。该日初始价值函数有 13 个折点，在 6,000 kWh 处给出上述最优费用。

定义库存边际价值  $\mu_t(s) = -\partial J_t(s)/\partial s$ ，其单位为元/内部 kWh。增加库存能够少付的后续购电费就是该库存的机会成本。由于  $J$  具有凸性，边际价值随库存增加而不增；在折点处取相应的左右边际值。图 4(a) 展示这一关系，日初 6,000 kWh 附近的边际价值约为 0.4754 元/kWh。



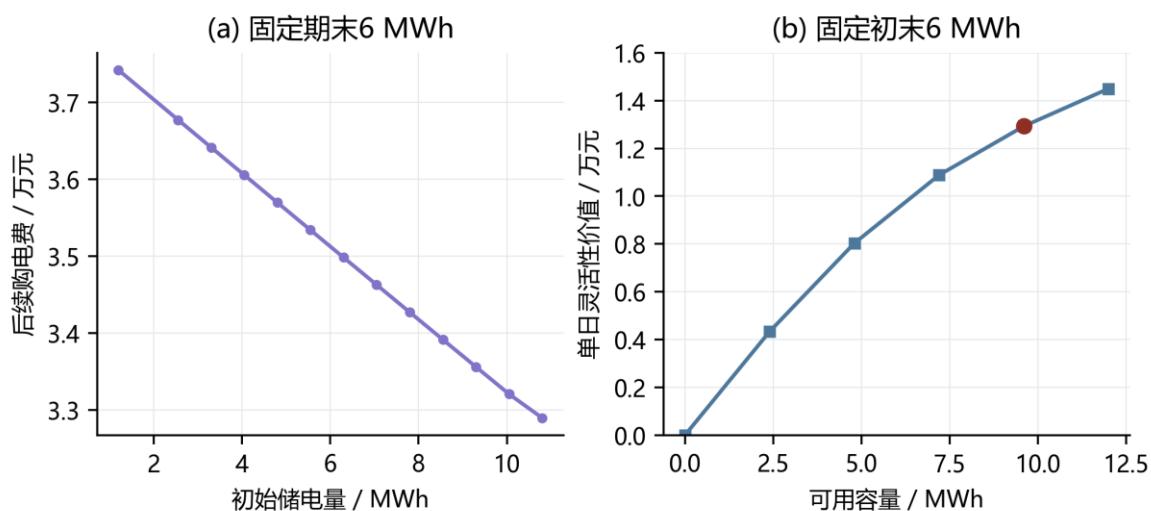


图 4 库存费用函数与容量的运行价值

从经济意义看，储能赋予运营者选择用电时机的权利，这与实物期权所刻画的运行灵活性相联系[8]。本题不涉及售电，择时收益体现为避免未来购电支出。下一问将沿用这一机会成本，将确定性后续费用扩展为历史供需场景下的价值估计。

## 六 第二问：预测、风险校准与实时控制

### 6.1 供需预测

附件负载具有明显周周期：同槽七日相关系数为 0.9855，日总负载七日相关为 0.9880。光伏则同时受昼夜形状和逐日变化影响。图 5 展示全年供需结构，这些规律用于构造预测特征，而不直接代替实际费用评价。

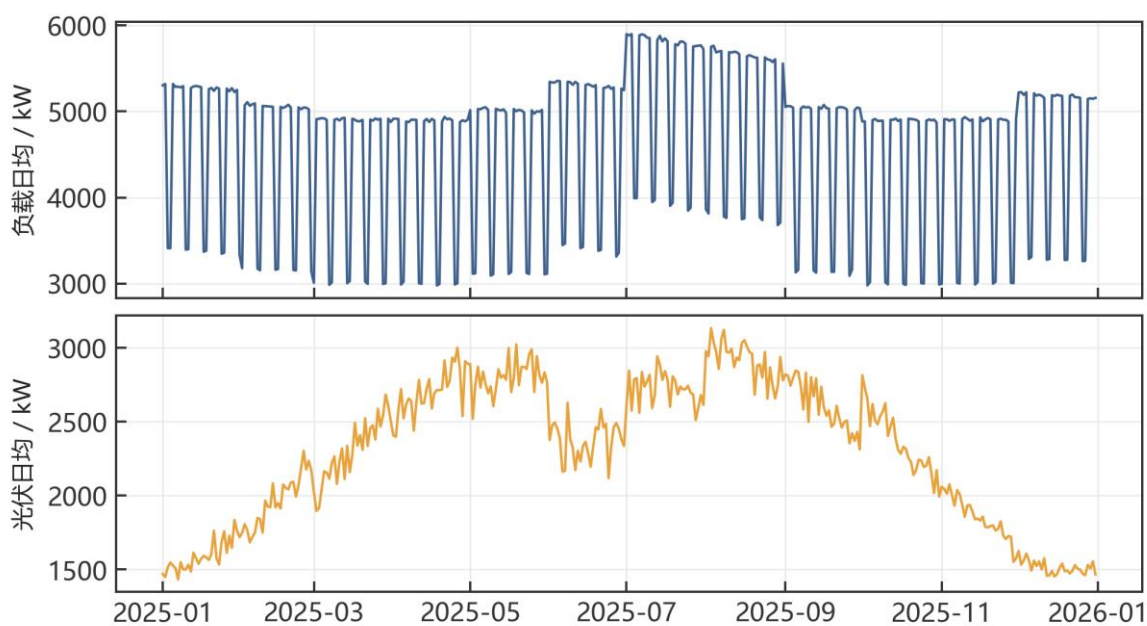


图 5 全年负载与光伏分布

对负载与光伏分别建立岭回归[3]。输入包括昨日、前日、上周同日同槽值，近三日及七日同槽均值、七日同槽标准差、昨日及近三日全天均值，以及日内前三阶正余弦和星期指示，共 21 个特征。采用带截距的目标函数：

$$\min_{\beta, b} \sum_{i \in \mathcal{T}_d} (y_i - b - \tilde{x}_i^T \beta)^2 + \lambda \|\beta\|_2^2. \quad (5)$$

标准化参数只从当次训练集估计，截距不惩罚。模型从 1 月 15 日开始每七天更新，使用最近至多 60 个完整日；惩罚系数在 1、10、100 中按一月预测误差选为 1。预测截断为非负，过去七日同槽光伏全为零时预测置零。历史不足时用近三日均值和日周同期均值完成热身。

同期预测取昨日与上周同日同槽值的平均，用作可解释的预测参照。全年岭回归的负载 MAE、RMSE 分别为 179.86、234.59 kW，光伏分别为 155.28、306.74 kW。图 6 给出指定日期的预测与实测，误差将通过风险校准和实时控制进入后续决策。

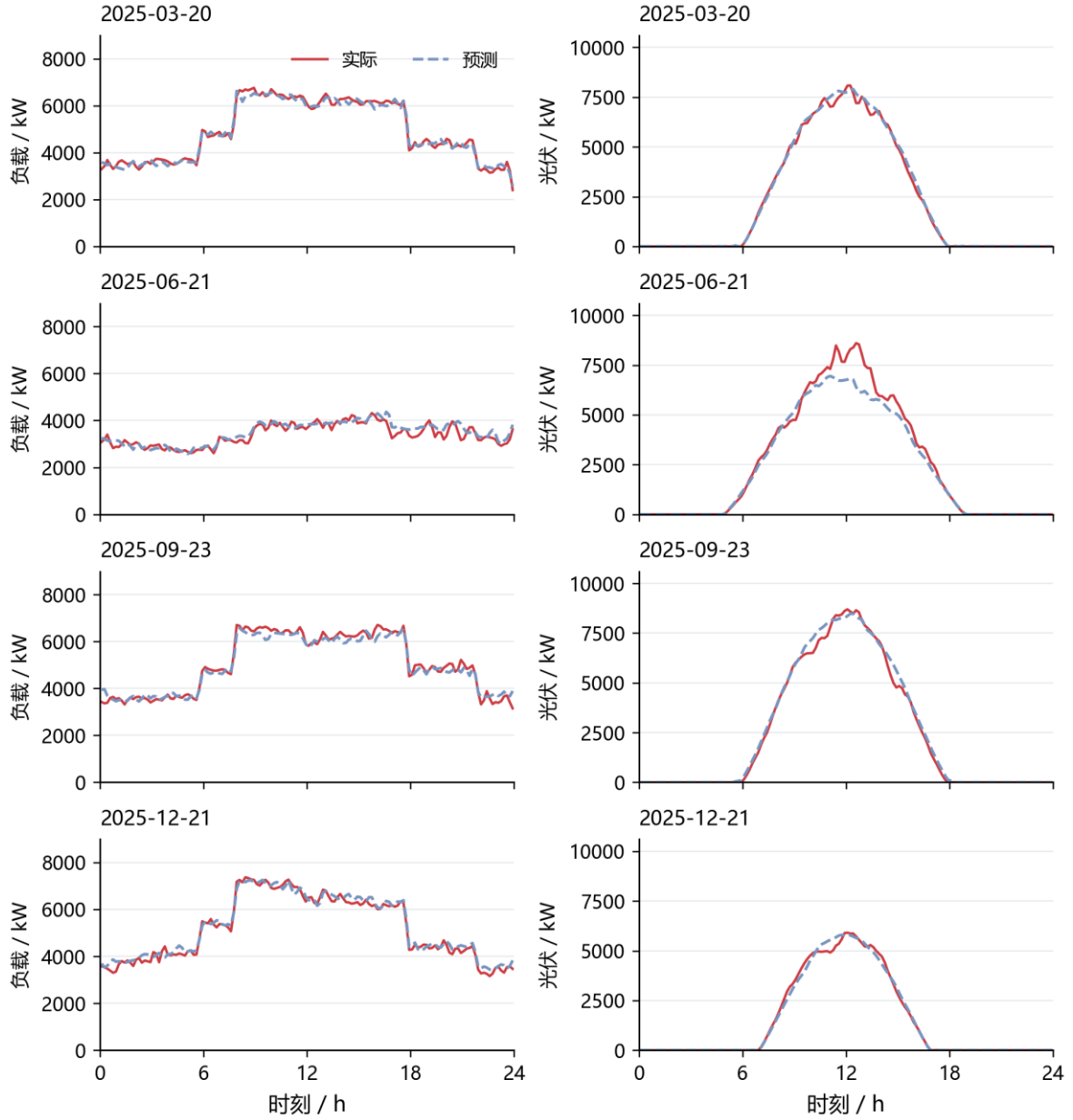


图 6 指定日期的负载与光伏预测

## 6.2 非对称风险与日前购电

低估净负载会触发五倍补购，高估则形成普通计划支出，二者的费用并不对称。定义预测净负载误差：

$$\varepsilon_{j,s} = (L_{j,s} - V_{j,s}) - (\hat{L}_{j,s} - \hat{V}_{j,s}), \quad r_{d,t}(q) = \max\{0, Q_q(\varepsilon_{d,t})\}. \quad (6)$$

误差池 E 使用此前至多 28 日、目标时段前后三槽的已发生误差，不跨日环绕；排除首日冷启动误差。q 为历史误差的分位水平，余量 r 加在预测负载上，形成名义需求  $\hat{L} + r$ 。每日零点将其与光伏预测代入第五章的规划，初始状态取实际库存，末端条件改为  $S_{145} \geq 6,000 \text{ kWh}$ 。

在无储能的单时段模型中，最小化  $p \cdot g + 5p \cdot E[(D-g)^+]$  会得到 0.8 临界分位。跨时段储能和日末备用改变了这一关系，因此本文按完整运行费用选择  $q$ 。一月验证选择 70% 余量：即时执行器下验证费 509,757.47 元；提高至 90% 虽消除该窗口紧急量，却使总费升至 633,664.05 元。风险校准由总费用约束，避免为压低缺口而过量申报。

### 6.3 历史路径下的库存价值控制

计划确定后，普通购电费按合同支付，实时阶段需要决定电池如何应对净缺口。每次发布取此前至多 28 个完整日，将负载与光伏的配对残差叠加到当前预测，分别截断为非负后得到  $K$  条场景。配对保留了同一历史日供需误差的关联及日内形状。

对第  $k$  条场景递推其后续紧急购电费用  $J_t^k(s)$ 。场景内计划量固定，富余优先充电，紧急购电仅补负载；日末附加价值取零。以路径函数均值近似当前信息下的后续费用：

$$\bar{J}_t(s) = \frac{1}{K} \sum_{k=1}^K J_t^k(s). \quad (7)$$

在当前实际净缺口  $n_t = L_t - V_t - g_t > 0$  时，下一库存按下式确定：

$$S_{t+1} \in \arg \min_{z \in [\max\{S_{\min}, S_t - \min(n_t, M)/\eta_d\}, S_t]} \{5p_t[n_t - \eta_d(S_t - z)] + \bar{J}_{t+1}(z)\}. \quad (8)$$

该目标仍为凸分段线性，只需比较可达区间端点及路径价值折点。放电一单位内部库存节省  $5p_t \eta_d$  元；当它高于库存后续边际价值  $\mu$  时，放电有利，反之应保留库存。这一条件将第五章的机会成本直接转化为实时动作。

当  $n_t \leq 0$  时，充电量取  $\min(-n_t, M, (S_{\max} - S_t)/\eta_c)$ ，剩余电量弃用。无论采用何种动作，负载缺口均由紧急购电补足。下一时段与次日从实际末状态继续，全年实际费用为：

$$C_2 = \sum_{d,t} p_t (g_{d,t} + 5e_{d,t}). \quad (9)$$

## 七 第二问：运行结果与机制分析

### 7.1 策略选择与全年结果

一月验证中，路径价值控制费用为 509,140.53 元，低于即时放电执行器的 509,757.47 元。零终端价值与低谷价折算终端价值同分，选取较简单的零值规则。随后按时间顺序运行 334 天，结果见表 5。

表 5 第二问执行策略的费用与库存

| 指标       | 即时放电          | 库存价值控制        |
|----------|---------------|---------------|
| 普通计划费/元  | 12,935,046.95 | 12,934,997.78 |
| 紧急购电费/元  | 1,197,565.21  | 1,056,394.83  |
| 总费用/元    | 14,132,612.16 | 13,991,392.60 |
| 紧急电量/kWh | 197,465.10    | 198,137.46    |
| 期末库存/kWh | 5,903.65      | 5,903.65      |

路径价值控制总费用 13,991,392.60 元，较相同预测、余量及计划生成规则下的即时执行器节省 141,219.56 元，降幅 0.999%。两方案期末库存均为 5,903.65 kWh。普通计划费仅变化 49.17 元，主要收益来自紧急费用下降 141,170.39 元。

## 7.2 放电择时的作用

即时执行器遇缺口便尽量放电，容易把库存用于较便宜的时段。价值控制在当前补购成本较低时保留部分库存，以减少后续昂贵缺口。由此，紧急购电量从 197,465.10 增至 198,137.46 kWh，费用却下降。数量与支出方向不同，说明调度优化需要同时考虑价格和时点。

由图 7 可见，累计节费与紧急量变化并不同步，库存价值控制通过调整放电时段降低账单。四个指定日期的详细结果见附录 A。

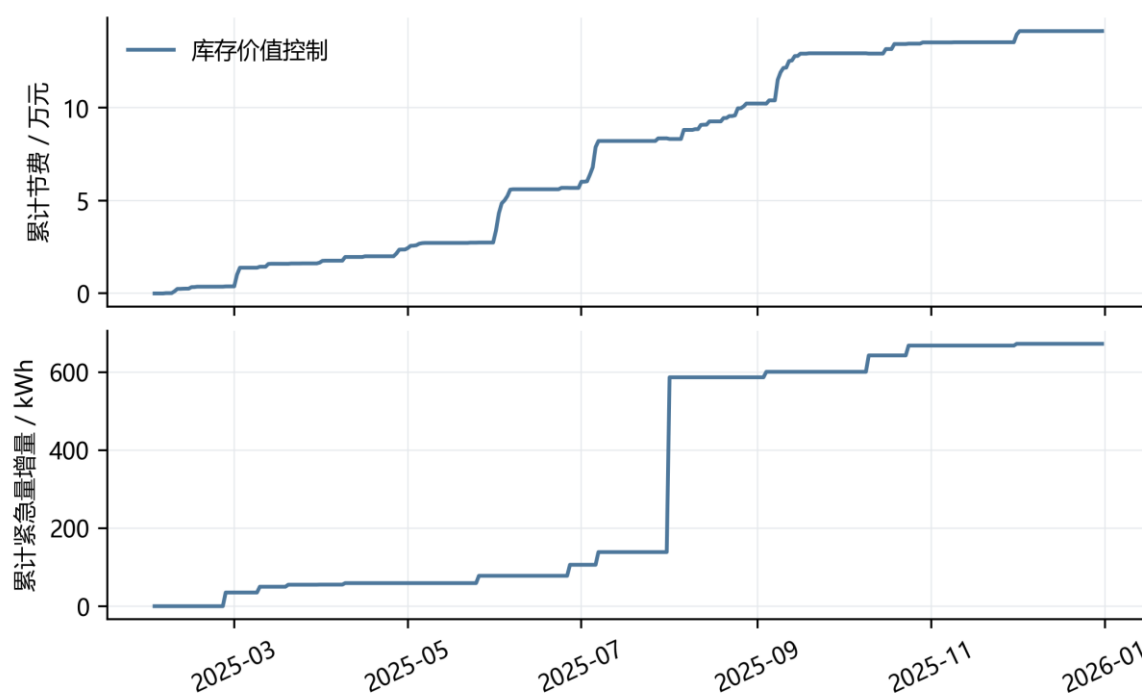


图 7 日前库存价值控制的累计费用与补购量变化

### 7.3 参数稳定性

在保持原即时执行器、风险余量和其他设置不变时，28、56、60、84 日训练窗的全年费用分别为 14,211,345.15、14,152,541.05、14,132,612.16、14,033,154.46 元。该结果说明更长历史在本年有潜力，但窗口取整周并不保证费用更低。一月训练历史尚短，无法区分这些上限，主策略继续采用既定 60 日窗。

这里的窗口对照用于识别季节稳定性；实际部署中，窗口长度需由当时可获得的验证数据确定。第三问将进一步利用日内已观测负载，而不仅依赖零点生成的全天预测。

## 八 第三问：信息更新与合同调整

### 8.1 光伏预报与可用信息

每次发布提供未来 1 至 24 小时整点功率。对相邻整点作线性插值并计算各十分钟区间积分；发布时刻的左端点由最近已结束区间的实测光伏近似。只取当前日期内尚未执行的部分，不将下一日真实数据用于边界补齐。图 8 给出同一日期各版预报与实际光伏。

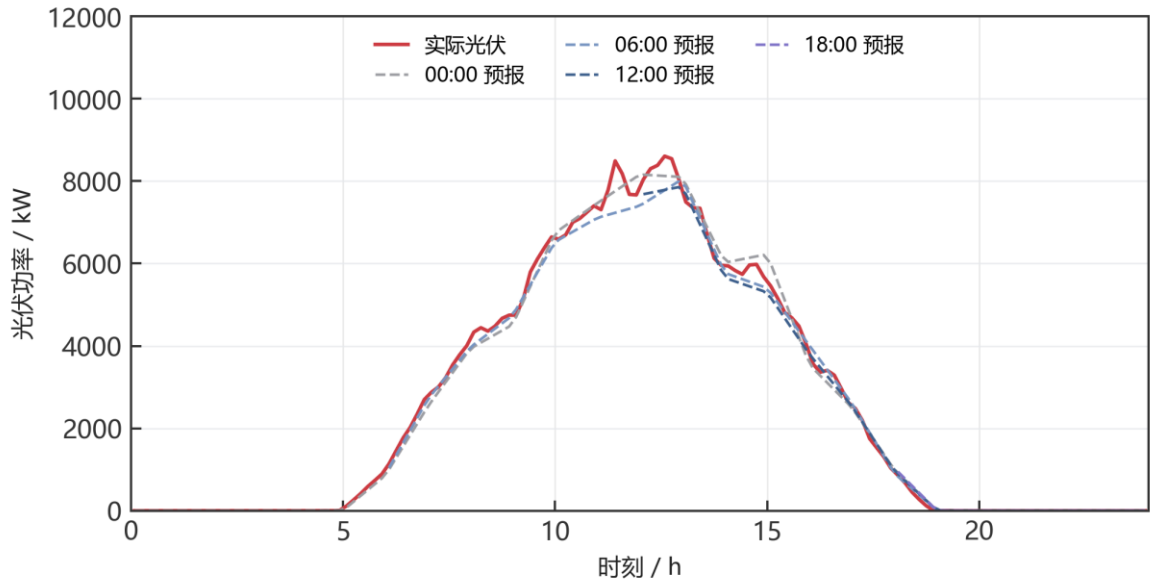


图 8 各发布时刻的光伏预报

滚动规划在 0、6、12、18 点运行，每次以当时实际库存为初值，固定过去动作，重算当前至 24 点的计划。该处理与模型预测控制的滚动时域思想一致[4]。

### 8.2 合同费用与滚动规划

记  $x_t$  为午夜计划、 $y_t$  为执行前最后一次有效计划。取消量  $u_t = (x_t - y_t)^+$ ，新增量  $v_t = (y_t - x_t)^+$ 。普通合同及紧急购电费用统一写为：

$$C_3 = \sum_{d,t} p_t \{ \min(x_t, y_t) + 0.5u_t + 1.5v_t + 5e_t \} = \sum_{d,t} p_t \{ x_t - 0.5u_t + 1.5v_t + 5e_t \}. \quad (10)$$

保留合同全额结算，取消部分仍付半价，新增部分按 1.5 倍付费。每次规划相对于同一午夜合同列式，以有效计划作为供电输入；已执行时段的合同和费用不再改变。

在发布时刻  $\tau$ ，剔除与当前决策无关的午夜合同常数后，剩余时域的目标为：

$$\min \sum_{t \in \mathcal{T}_\tau} p_t (1.5v_t - 0.5u_t), \quad y_t - x_t = v_t - u_t, \quad u_t, v_t \geq 0. \quad (11)$$

约束沿用第五章物理模型，负载与光伏替换为当次预测及其风险余量，日末名义库存下限仍为 6,000 kWh。按一月验证费用选择 50% 余量，并在三个后续发布时刻全部更新。

### 8.3 负载偏差修正

光伏预报更新时，日内已观测的负载也提供新信息。以发布前 18 个区间的平均负载预测误差作为当前偏差  $x$ ，以历史目标小时的平均误差为响应  $y$ ，对不同发布时刻和提前小时分别拟合收缩回归：

$$\hat{\beta}_{d,v,h} = \text{clip}_{[0,1.5]} \left( \frac{\sum_{i \in H_d} x_{i,v} y_{i,v,h}}{1.2 \sum_{i \in H_d} x_{i,v}^2} \right), \quad \hat{L}_{d,v,t}^A = \max\{0, \hat{L}_{d,t}^0 + \hat{\beta}_{d,v,h(t)} x_{d,v}\}. \quad (12)$$

$H$  包含此前至多 28 个已结束日期；历史不足七个完整日不启用修正，分母为零时系数取零。系数非负与上限约束限制短样本放大，1.2 倍分母相当于 20% 的平方尺度收缩。每个预测族使用自身历史误差重新计算风险余量。

月初分别回放此前 28 天的基础预测与修正预测，从共同已知库存出发，按实际费用选择本月预测族。该选择信号由即时执行器的历史影子运行生成；真实系统随后按价值控制器运行，两者的库存记录分别保持连续。2 月沿用一月选择，此后只使用过去月份的信息。

6、12、18 点修正后的负载 MAE 分别为 27.96、28.85、27.68 kWh/十分钟，较未修正降低 7.61%、6.85%、12.34%。实际在 3、6、7、8、9、10 月启用修正，其余月份采用基础预测。

### 8.4 可调单条件下的库存价值

合同可调整后，后续缺口不必全部依靠五倍紧急购电。为将这一机会纳入第六章价值递推，当前至下次发布前的场景缺口仍按五倍价估计，下一次发布后的缺口则以 1.5 倍预测价近似其可新增合同的边际成本。每次发布重建后续函数，只执行随后的 36 个时段。

这个近似把未来调单机会反映在当前放电条件中：能够在下一次发布后以较低费用补购的缺口，对保留库存的要求较弱。实际运行账单仍严格按第 8.2 节计算，所有真实紧急购电均付五倍价。

### 8.5 更新信息的经济价值

固定 50% 余量、负载预测与即时执行器，比较仅使用零点预报与使用全部四次预报，费用分别为 15,045,126.93 元和 13,575,976.78 元，节省 1,469,150.15 元，降幅 9.76%。日内预报使一部分供电缺口提前转化为普通计划调整，紧急费下降足以覆盖调整费用，因此有必要使用其他时刻预报。

在此基础上，月度负载选择将费用降至 13,489,127.94 元；再加入可调单库存价值控制，最终费用为 13,479,283.32 元，紧急购电量 79,340.34 kWh。图 9 按实际加入顺序展示增益。各阶段的比较对象不同，分解用于解释改进过程，不假设各模块收益彼此独立。

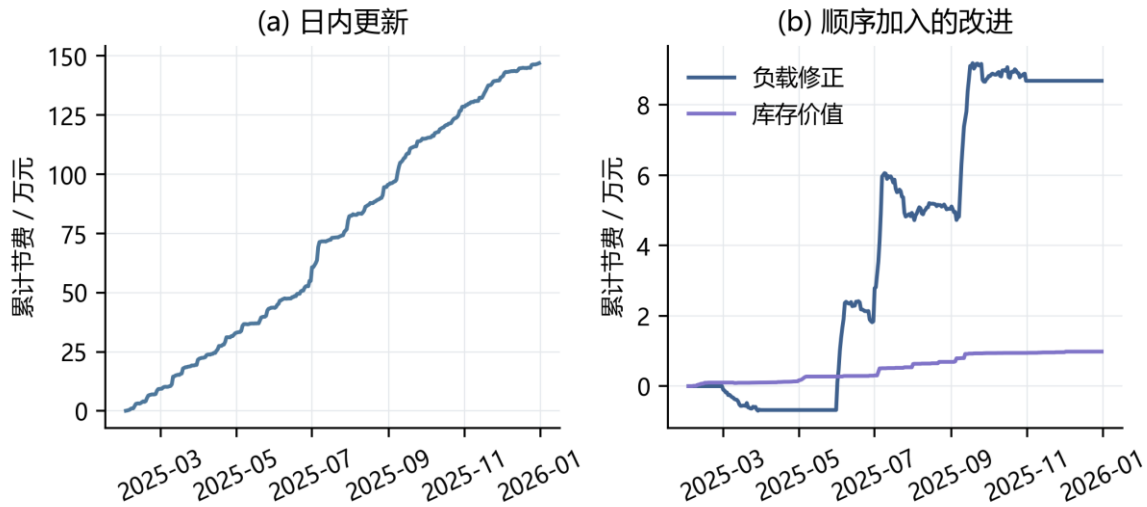


图 9 日内信息更新及后续改进的累计收益

表 6 第三问最终方案的费用构成

| 费用项目     | 金额/元          |
|----------|---------------|
| 午夜原合同    | 12,524,675.97 |
| 新增合同费用   | 739,101.35    |
| 取消合同半价抵扣 | -221,795.48   |
| 紧急购电     | 437,301.48    |
| 总费用      | 13,479,283.32 |

最终方案的费用分解见表 6，指定日计划、储能与紧急购电见附录 A。普通合同的每次修改均保留发布时间及有效区间，保证滚动计划能够对应到实际执行。

## 九 第四问：波动电价下的联合决策

### 9.1 价格预测

附件 4 电价范围为 0.0076—1.7936 元/kWh。价格的日内结构为择时提供依据，其随机偏离则使库存机会成本发生变化。三类预测器均使用发布前历史和已知日历信息，输出未来 144 个十分钟目标，只将当日剩余部分送入调度。

同期预测取昨日与上周同一目标时段价格的均值；岭回归在滞后价格、统计量和周期特征上建立 12 维线性预测；残差 GRU 在同期预测基础上学习非线性修正。GRU 输入包括昨日价、上周价、日内正余弦及星期正余弦，采用单层 32 维隐藏状态[5 - 6]：

$$\hat{p}_{\tau,j} = \max\{0.001, \hat{p}_{\tau,j}^{\text{seasonal}} + s_y(w^T h_j + b)\}, \quad h_j = \text{GRU}(x_j, h_{j-1}). \quad (13)$$



sy 为当次训练价格标准差，输出头从零初始化。使用 SmoothL1 损失与 Adam 优化器，学习率 0.003、批量 32、梯度范数上限 1，最多 50 轮，内部验证连续六轮无改进即停止。每次使用此前至多 60 日，最近三个完整日用于内部验证；未全部发生的目标窗口不进入训练。

在 1 月 15 日、22 日及此后月初拟合，三个基种子 17、42、2026 各与拟合日编号相加，最终等权集成。图 10 比较相同发布时刻和目标范围的价格误差。

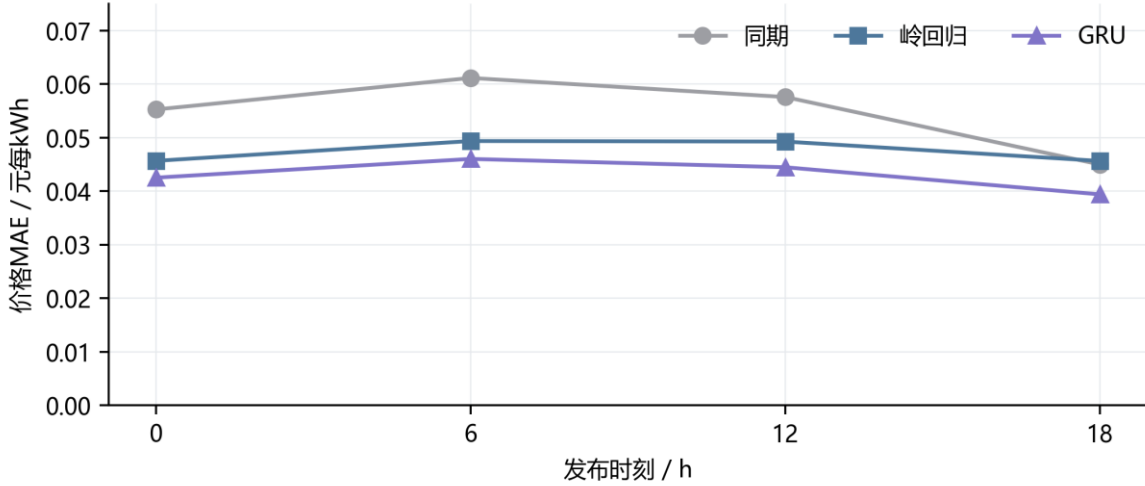


图 10 各发布时刻的电价预测误差

GRU 在四组比较中均取得最低 MAE，按全部 120,240 个发布—目标配对加权，MAE 为 0.04366 元/kWh，较同期降低 22.70%，较岭回归降低 8.09%。同一目标可在不同发布时刻被预测，所有模型按相同配对计算误差。

## 9.2 价格与库存控制的衔接

在日前分支，午夜价格预测进入合同优化及固定合同后续价值；在滚动分支，每次新价格预测与供需信息共同进入剩余时域规划，后续价值同时计入调单机会。当前区间动作使用已揭示的实际电价，未来函数使用当次预测价格，结算则使用实际交易价。

价格预测器按一月实际费用选择：日前采用岭回归，滚动采用同期预测。保持基础调度规则时，GRU 虽然价格误差更低，但其滚动年度费用仅比同期低 2,094.78 元。容量、功率和效率限制了动作空间；当几种预测给出相同峰谷价序时，误差降低未必改变购电和放电决策。

接入库存价值控制后，日前费用 14,765,492.68 元，较即时执行器节省 115,042.37 元；滚动费用 14,210,881.01 元，较仅采用月度负载选择的方案再省 5,444.97 元。完整结果见表 7，指定日期见附录 A。

表 7 四问最终费用与运行区间

| 问题  | 总费用/元         | 运行区间  |
|-----|---------------|-------|
| 一   | 35,126.95     | 单日    |
| 2   | 13,991,392.60 | 334 天 |
| 3   | 13,479,283.32 | 334 天 |
| 4-2 | 14,765,492.68 | 334 天 |
| 4-3 | 14,210,881.01 | 334 天 |

### 9.3 价格信息的名义改善空间

为衡量价格预测尚能改变多少计划费用，固定供需预测、风险余量、日初库存和日末条件，构造共同名义可行域  $\Omega_d$ 。以真实价格在该可行域求最小费用  $J_d^*$ ，并用同一真实价格评价预测器  $m$  产生的名义计划，记为  $J_{m,d}$ ，则：

$$J_d^* = \min_{g \in \Omega_d} p_d^T g, \quad J_{m,d} = p_d^T g_{m,d}, \quad 0 \leq \Delta_m \leq \sum_d (J_{m,d} - J_d^*). \quad (14)$$

基础名义计划实验中，GRU 费用为 13,656,357.29 元，真实价格最优费用为 13,519,514.66 元，由未舍入值计算差额 136,842.64 元，占 1.002%。这表示在该组供需与储能条件下，仅改善价格预测的名义节费空间有限。实际运行还包含供需误差、紧急购电和状态反馈，故与上述名义费用分别核算。

由第二至四问可见，供需校准决定缺口大小，合同调整决定补购途径，库存价值决定有限电量的使用时机。价格信息通过同一价值函数参与决策，而不是形成一套与前文分离的调度模型。

## 十 模型的评价与推广

### 10.1 模型的优点

(1) 物理模型与决策时序统一。四问共用电量平衡和库存递推，仅随问题条件增加预测与合同权限；日前计划、日内修订和实时执行均可对应到同一状态轨迹。

(2) 风险与成本直接关联。历史分位余量控制欠购风险，价值函数进一步区分缺口发生的价格和时点。第二问在相同初末库存下节省 141,219.56 元，表明有限储能的配置方式可以在总补购量变化很小时降低支出。

(3) 求解结构清晰。循环消除使线性规划获得可执行动作，连续凸分段线性递推保留库存边际信息。名义优化最优值、物理约束及逐时账单经交叉核算一致，模型能够从计划层追溯到运行结果。

(4) 信息价值可分解。固定其余条件比较预报更新，再按顺序加入负载修正和库存控制，区分信息、预测与执行的作用，为选择数据服务和控制策略提供费用依据。

## 10.2 模型的局限

(1) 十分钟均值忽略区间内快速波动，线性效率未包含退化、爬坡及开关损耗。时间记录的区间均值解释与当前电价可见性是运行假设，实际部署需与计量及交易机制匹配。

(2) 历史路径均值近似后续费用；滚动分支用 1.5 倍价格概括未来新增合同机会，未联合枚举未来取消、调单和供需的多阶段场景。确定性名义模型可证明最优，随机环境下的全年策略属于可实施近似。

(3) 数据仅覆盖一年，模型经过同年开发复核。配对区块重采样反映该年固定轨迹的波动，不能覆盖模型选择偏差和跨年分布变化；日末备用及训练窗口仍需独立数据检验。

## 10.3 模型推广

**储能灵活性的价值评估。** 令  $\Pi$  为受功率、容量和合同约束的可行策略集合，固定规则  $\pi_0$  的预计费用为  $C^{\pi_0}$ 。允许根据已揭示信息择时运行时，运营灵活性价值可定义为：

$$\mathcal{F}_t(s) = C_t^{\pi_0}(s) - \inf_{\pi \in \Pi} \mathbb{E}[C_t^\pi | \mathcal{I}_t, S_t = s]. \quad (15)$$

只要  $\pi_0$  属于  $\Pi$ ， $\mathcal{F}$  即非负。该差额对应运营者获得调节时机选择权后可避免的支出，可视为储能运行实物期权的价值[8]。本文按真实费用评价这一灵活性，不额外假定交易市场完备或构造金融衍生品的无套利价格。

**容量配置。** 将允许库存区间以 6,000 kWh 为中心对称扩展，保持初末状态和 5,000 kW 功率不变，重新求解所得结果见图 4(b)。可用容量为 2,400、4,800、7,200、9,600、12,000 kWh 时，单日节费分别为 4,337.35、8,031.64、10,882.46、12,925.10、14,495.30 元。扩容增加可行策略，节费随容量上升，但边际收益递减。以各季代表日加权得到年运行收益后，可与增容及退化费用比较，确定经济容量；本题未给出投资数据，因此不计算投资回收期。

**风险管理与其他灵活负载。** 对极端缺口敏感的运营者，可在预期费用中加入尾部风险项，将目标扩展为  $\mathbb{E}[C] + \lambda \cdot \text{CVaR}_\alpha(C)$ ，再由承受能力确定风险权重。对于电动汽车群或可调节冷热负荷，只需替换状态方程和可行动作，保留库存边际价值与信息更新的递推结构，即可研究不同设备的协同择时。新增风险与设备参数需重新验证，不能直接沿用本题数值。

## 参考文献

- [1] SciPy Developers. scipy.optimize.milp: Mixed-integer linear programming[EB/OL]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>.
- [2] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10: 119–142. DOI:10.1007/s12532-017-0130-5.
- [3] Hoerl A E, Kennard R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55–67. DOI:10.1080/00401706.1970.10488634.
- [4] Mayne D Q, Rawlings J B, Rao C V, Scokaert P O M. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789–814. DOI:10.1016/S0005-1098(99)00214-9.
- [5] PyTorch Contributors. GRU[EB/OL]. <https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html>.
- [6] Cho K, van Merriënboer B, Gulcehre C, et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation[C]. EMNLP, 2014: 1724–1734. DOI:10.3115/v1/D14-1179.
- [7] Boyd S, Vandenberghe L. Convex Optimization[M]. Cambridge University Press, 2004.
- [8] Carmona R, Ludkovski M. Valuation of energy storage: An optimal switching approach[J]. Quantitative Finance, 2010, 10(4): 359–374. DOI:10.1080/14697680902946514.

附录 A 四问指定日期完整结果

电量单位为 kWh，费用单位为元。普通购电与紧急购电分列。滚动问题的指定时段和全天购电量指最终有效普通计划；午夜计划及每次更新完整保存在 Excel 中。所有表由实际轨迹自动生成。

A.1 问题 1

对应结果见表 8。

表 8 问题 1（单日）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 480.41            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 445.43 | 18:00-18:10 | 531.89            | 20:00-20:10 | 0.00 |
| 全天购电量 59,482.70 kWh |        |             | 全天购电费 35,126.95 元 |             |      |

对应结果见表 9。

表 9 问题 1（单日）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,500.00 | 0.00     | 04:00-08:00 | 833.33   | 6,365.84 |
| 08:00-12:00 | 4,787.96 | 1,703.00 | 12:00-16:00 | 5,286.04 | 91.10    |
| 16:00-20:00 | 0.00     | 5,780.13 | 20:00-24:00 | 5,333.33 | 2,859.87 |
| 00:00 储电    | 6,000.00 |          | 24:00 储电    | 6,000.00 |          |

A.2 问题 2

对应结果见表 10。

表 10 问题 2（2025-03-20）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 574.14            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 488.93 | 18:00-18:10 | 720.76            | 20:00-20:10 | 0.00 |
| 全天购电量 68,036.47 kWh |        |             | 全天购电费 41,302.44 元 |             |      |

对应结果见表 11。

表 11 问题 2（2025-03-20）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,071.86 | 209.79   | 04:00-08:00 | 1,038.99 | 5,515.86 |
| 08:00-12:00 | 6,186.28 | 2,777.38 | 12:00-16:00 | 4,162.63 | 444.57   |
| 16:00-20:00 | 266.98   | 5,654.71 | 20:00-24:00 | 5,687.88 | 2,954.47 |
| 00:00 储电    | 6,617.05 |          | 24:00 储电    | 6,382.68 |          |

对应结果见表 12。

表 12 问题 2（2025-03-20） 紧急购电

| 时段          | 电量   |
|-------------|------|
| 17:20-17:30 | 5.19 |

对应结果见表 13。

表 13 问题 2（2025-06-21） 指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 0.00              | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 210.92 | 18:00-18:10 | 0.00              | 20:00-20:10 | 0.00 |
| 全天购电量 36,315.40 kWh |        |             | 全天购电费 21,120.39 元 |             |      |

对应结果见表 14。

表 14 问题 2（2025-06-21） 储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 526.76   | 524.94   | 04:00-08:00 | 413.77   | 4,402.74 |
| 08:00-12:00 | 9,426.35 | 0.00     | 12:00-16:00 | 0.00     | 0.00     |
| 16:00-20:00 | 151.96   | 5,493.83 | 20:00-24:00 | 6,132.83 | 2,497.81 |
| 00:00 储电    | 6,945.01 |          | 24:00 储电    | 7,576.71 |          |

对应结果见表 15。

表 15 问题 2（2025-06-21） 紧急购电

| 时段 | 电量   |
|----|------|
| 无  | 0.00 |

对应结果见表 16。

表 16 问题 2（2025-09-23） 指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 491.40            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 549.29 | 18:00-18:10 | 788.73            | 20:00-20:10 | 0.00 |
| 全天购电量 71,561.09 kWh |        |             | 全天购电费 44,415.98 元 |             |      |

对应结果见表 17。

表 17 问题 2（2025-09-23） 储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 5,252.67 | 0.00     | 04:00-08:00 | 218.14   | 5,830.96 |
| 08:00-12:00 | 5,558.85 | 1,896.62 | 12:00-16:00 | 4,033.40 | 575.17   |
| 16:00-20:00 | 231.66   | 5,642.21 | 20:00-24:00 | 6,050.86 | 2,841.96 |
| 00:00 储电    | 6,072.60 |          | 24:00 储电    | 6,631.50 |          |

对应结果见表 18。

表 18 问题 2（2025-09-23）紧急购电

| 时段          | 电量   |
|-------------|------|
| 20:20-20:30 | 0.33 |

对应结果见表 19。

表 19 问题 2（2025-12-21）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段               | 购电量    | 时间段         | 购电量  |
|---------------------|--------|-------------------|--------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10       | 967.61 | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 812.86 | 18:00-18:10       | 739.98 | 20:00-20:10 | 0.00 |
| 全天购电量 97,145.51 kWh |        | 全天购电费 62,434.14 元 |        |             |      |

对应结果见表 20。

表 20 问题 2（2025-12-21）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,629.25 | 143.74   | 04:00-08:00 | 1,398.62 | 1,538.91 |
| 08:00-12:00 | 5,704.82 | 7,168.67 | 12:00-16:00 | 6,940.07 | 2,243.11 |
| 16:00-20:00 | 24.16    | 5,708.86 | 20:00-24:00 | 5,695.76 | 2,842.53 |
| 00:00 储电    | 6,318.60 |          | 24:00 储电    | 6,443.34 |          |

对应结果见表 21。

表 21 问题 2（2025-12-21）紧急购电

| 时段          | 电量    |
|-------------|-------|
| 07:50-08:00 | 23.78 |

### A.3 问题 3

对应结果见表 22。

表 22 问题 3（2025-03-20）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段               | 购电量    | 时间段         | 购电量  |
|---------------------|--------|-------------------|--------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10       | 520.67 | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 638.18 | 18:00-18:10       | 714.94 | 20:00-20:10 | 0.00 |
| 全天购电量 66,401.94 kWh |        | 全天购电费 42,938.13 元 |        |             |      |

对应结果见表 23。

表 23 问题 3（2025-03-20）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,249.82 | 348.06   | 04:00-08:00 | 1,010.91 | 5,813.27 |
| 08:00-12:00 | 3,254.96 | 2,698.33 | 12:00-16:00 | 6,483.14 | 254.72   |
| 16:00-20:00 | 646.16   | 5,206.99 | 20:00-24:00 | 5,530.12 | 3,018.64 |
| 00:00 储电    | 6,309.41 |          | 24:00 储电    | 6,100.33 |          |

对应结果见表 24。

表 24 问题 3（2025-03-20） 紧急购电

| 时段          | 电量    |
|-------------|-------|
| 09:40-10:00 | 79.05 |
| 21:40-21:50 | 4.66  |

对应结果见表 25。

表 25 问题 3（2025-06-21） 指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 0.00              | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 120.43 | 18:00-18:10 | 0.00              | 20:00-20:10 | 0.00 |
| 全天购电量 34,874.69 kWh |        |             | 全天购电费 19,723.09 元 |             |      |

对应结果见表 26。

表 26 问题 3（2025-06-21） 储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 275.76   | 271.67   | 04:00-08:00 | 449.87   | 3,308.89 |
| 08:00-12:00 | 9,888.78 | 0.00     | 12:00-16:00 | 0.00     | 0.00     |
| 16:00-20:00 | 41.37    | 6,074.66 | 20:00-24:00 | 5,603.72 | 2,654.17 |
| 00:00 储电    | 5,225.44 |          | 24:00 储电    | 6,181.87 |          |

对应结果见表 27。

表 27 问题 3（2025-06-21） 紧急购电

| 时段 | 电量   |
|----|------|
| 无  | 0.00 |

对应结果见表 28。

表 28 问题 3（2025-09-23） 指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 347.56            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 522.47 | 18:00-18:10 | 784.63            | 20:00-20:10 | 0.00 |
| 全天购电量 67,918.94 kWh |        |             | 全天购电费 44,580.19 元 |             |      |

对应结果见表 29。

表 29 问题 3（2025-09-23） 储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,885.98 | 16.12    | 04:00-08:00 | 353.76   | 6,967.82 |
| 08:00-12:00 | 4,885.84 | 1,675.86 | 12:00-16:00 | 6,129.08 | 508.84   |
| 16:00-20:00 | 118.44   | 5,841.36 | 20:00-24:00 | 5,814.52 | 2,695.27 |
| 00:00 储电    | 6,102.47 |          | 24:00 储电    | 6,398.80 |          |



对应结果见表 30。

表 30 问题 3（2025-09-23） 紧急购电

| 时段          | 电量     |
|-------------|--------|
| 09:00-09:50 | 220.76 |
| 20:20-20:30 | 6.37   |

对应结果见表 31。

表 31 问题 3（2025-12-21） 指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 924.13            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 821.56 | 18:00-18:10 | 739.98            | 20:00-20:10 | 0.00 |
| 全天购电量 95,721.86 kWh |        |             | 全天购电费 62,219.35 元 |             |      |

对应结果见表 32。

表 32 问题 3（2025-12-21） 储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,657.30 | 179.36   | 04:00-08:00 | 1,565.97 | 1,572.10 |
| 08:00-12:00 | 5,310.40 | 7,513.79 | 12:00-16:00 | 7,773.62 | 2,250.94 |
| 16:00-20:00 | 57.98    | 5,766.67 | 20:00-24:00 | 5,676.93 | 2,842.53 |
| 00:00 储电    | 6,219.20 |          | 24:00 储电    | 6,395.64 |          |

对应结果见表 33。

表 33 问题 3（2025-12-21） 紧急购电

| 时段          | 电量    |
|-------------|-------|
| 07:50-08:00 | 33.89 |

A.4-2 问题 4-2

对应结果见表 34。

表 34 问题 4-2（2025-03-20） 指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量    |
|---------------------|--------|-------------|-------------------|-------------|--------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 574.14            | 14:00-14:10 | 0.00   |
| 16:00-16:10         | 488.93 | 18:00-18:10 | 310.56            | 20:00-20:10 | 756.24 |
| 全天购电量 67,791.08 kWh |        |             | 全天购电费 42,355.14 元 |             |        |

对应结果见表 35。

表 35 问题 4-2（2025-03-20）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 3,655.77 | 152.63   | 04:00-08:00 | 1,195.22 | 5,572.84 |
| 08:00-12:00 | 6,186.28 | 2,777.38 | 12:00-16:00 | 4,162.63 | 444.57   |
| 16:00-20:00 | 214.58   | 7,108.02 | 20:00-24:00 | 5,717.23 | 1,458.76 |
| 00:00 储电    | 6,850.72 |          | 24:00 储电    | 6,409.05 |          |

对应结果见表 36。

表 36 问题 4-2（2025-03-20）紧急购电

| 时段 | 电量   |
|----|------|
| 无  | 0.00 |

对应结果见表 37。

表 37 问题 4-2（2025-06-21）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 0.00              | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 210.92 | 18:00-18:10 | 0.00              | 20:00-20:10 | 0.00 |
| 全天购电量 36,422.89 kWh |        |             | 全天购电费 18,518.97 元 |             |      |

对应结果见表 38。

表 38 问题 4-2（2025-06-21）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 438.52   | 3,694.59 | 04:00-08:00 | 1,239.76 | 1,877.96 |
| 08:00-12:00 | 9,421.97 | 0.00     | 12:00-16:00 | 0.00     | 0.00     |
| 16:00-20:00 | 163.47   | 5,505.34 | 20:00-24:00 | 6,196.26 | 2,497.81 |
| 00:00 储电    | 7,001.50 |          | 24:00 储电    | 7,631.37 |          |

对应结果见表 39。

表 39 问题 4-2（2025-06-21）紧急购电

| 时段 | 电量   |
|----|------|
| 无  | 0.00 |

对应结果见表 40。

表 40 问题 4-2（2025-09-23）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 491.40            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 549.29 | 18:00-18:10 | 416.70            | 20:00-20:10 | 0.00 |
| 全天购电量 71,436.45 kWh |        |             | 全天购电费 46,451.24 元 |             |      |

对应结果见表 41。

表 41 问题 4-2（2025-09-23）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,585.50 | 0.00     | 04:00-08:00 | 760.67   | 5,830.96 |
| 08:00-12:00 | 5,687.53 | 1,896.62 | 12:00-16:00 | 3,858.02 | 528.46   |
| 16:00-20:00 | 253.12   | 6,023.40 | 20:00-24:00 | 6,035.33 | 2,472.31 |
| 00:00 储电    | 6,184.77 |          | 24:00 储电    | 6,633.87 |          |

对应结果见表 42。

表 42 问题 4-2（2025-09-23）紧急购电

| 时段          | 电量    |
|-------------|-------|
| 20:10-20:20 | 10.23 |

对应结果见表 43。

表 43 问题 4-2（2025-12-21）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 967.61            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 812.86 | 18:00-18:10 | 739.98            | 20:00-20:10 | 0.00 |
| 全天购电量 97,314.24 kWh |        |             | 全天购电费 73,263.64 元 |             |      |

对应结果见表 44。

表 44 问题 4-2（2025-12-21）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 5,056.07 | 143.74   | 04:00-08:00 | 1,743.90 | 2,155.89 |
| 08:00-12:00 | 5,755.95 | 7,219.80 | 12:00-16:00 | 6,952.06 | 2,243.11 |
| 16:00-20:00 | 24.16    | 5,708.86 | 20:00-24:00 | 5,755.02 | 2,845.49 |
| 00:00 储电    | 6,309.24 |          | 24:00 储电    | 6,493.37 |          |

对应结果见表 45。

表 45 问题 4-2（2025-12-21）紧急购电

| 时段          | 电量    |
|-------------|-------|
| 07:50-08:00 | 23.78 |

A.4-3 问题 4-3

对应结果见表 46。

表 46 问题 4-3（2025-03-20）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量    |
|---------------------|--------|-------------|-------------------|-------------|--------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 520.67            | 14:00-14:10 | 0.00   |
| 16:00-16:10         | 638.18 | 18:00-18:10 | 714.94            | 20:00-20:10 | 741.44 |
| 全天购电量 66,091.31 kWh |        |             | 全天购电费 43,656.33 元 |             |        |

对应结果见表 47。

表 47 问题 4-3（2025-03-20）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 3,255.66 | 249.34   | 04:00-08:00 | 1,840.75 | 5,937.64 |
| 08:00-12:00 | 3,254.96 | 2,698.33 | 12:00-16:00 | 6,863.61 | 254.72   |
| 16:00-20:00 | 291.94   | 6,696.45 | 20:00-24:00 | 5,561.08 | 1,554.74 |
| 00:00 储电    | 6,485.79 |          | 24:00 储电    | 6,123.41 |          |

对应结果见表 48。

表 48 问题 4-3（2025-03-20）紧急购电

| 时段          | 电量    |
|-------------|-------|
| 09:40-10:00 | 79.05 |

对应结果见表 49。

表 49 问题 4-3（2025-06-21）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 0.00              | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 120.43 | 18:00-18:10 | 0.00              | 20:00-20:10 | 0.00 |
| 全天购电量 34,998.69 kWh |        |             | 全天购电费 17,375.08 元 |             |      |

对应结果见表 50。

表 50 问题 4-3（2025-06-21）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 275.76   | 2,705.06 | 04:00-08:00 | 1,497.17 | 1,779.01 |
| 08:00-12:00 | 9,921.65 | 0.00     | 12:00-16:00 | 0.00     | 0.00     |
| 16:00-20:00 | 41.37    | 6,074.66 | 20:00-24:00 | 5,613.02 | 2,623.77 |
| 00:00 储电    | 5,257.18 |          | 24:00 储电    | 6,224.03 |          |

对应结果见表 51。

表 51 问题 4-3（2025-06-21）紧急购电

| 时段 | 电量   |
|----|------|
| 无  | 0.00 |

对应结果见表 52。

表 52 问题 4-3（2025-09-23）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 347.56            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 522.47 | 18:00-18:10 | 628.48            | 20:00-20:10 | 0.00 |
| 全天购电量 67,886.23 kWh |        |             | 全天购电费 46,606.18 元 |             |      |

对应结果见表 53。

表 53 问题 4-3（2025-09-23）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 4,249.37 | 22.22    | 04:00-08:00 | 984.97   | 6,967.82 |
| 08:00-12:00 | 4,885.84 | 1,675.86 | 12:00-16:00 | 6,081.90 | 461.66   |
| 16:00-20:00 | 110.65   | 5,980.56 | 20:00-24:00 | 5,668.05 | 2,563.12 |
| 00:00 储电    | 6,114.10 |          | 24:00 储电    | 6,262.09 |          |

对应结果见表 54。

表 54 问题 4-3（2025-09-23）紧急购电

| 时段          | 电量     |
|-------------|--------|
| 09:00-09:50 | 220.76 |
| 18:20-18:30 | 1.08   |
| 20:10-20:20 | 10.23  |

对应结果见表 55。

表 55 问题 4-3（2025-12-21）指定时段购电及全天费用

| 时间段                 | 购电量    | 时间段         | 购电量               | 时间段         | 购电量  |
|---------------------|--------|-------------|-------------------|-------------|------|
| 10:00-10:10         | 0.00   | 12:00-12:10 | 924.13            | 14:00-14:10 | 0.00 |
| 16:00-16:10         | 821.56 | 18:00-18:10 | 739.98            | 20:00-20:10 | 0.00 |
| 全天购电量 95,641.71 kWh |        |             | 全天购电费 72,890.05 元 |             |      |

对应结果见表 56。

表 56 问题 4-3（2025-12-21）储能运行

| 时间段         | 充电量      | 放电量      | 时间段         | 充电量      | 放电量      |
|-------------|----------|----------|-------------|----------|----------|
| 00:00-04:00 | 5,169.62 | 178.51   | 04:00-08:00 | 232.98   | 904.57   |
| 08:00-12:00 | 5,347.35 | 7,538.30 | 12:00-16:00 | 8,080.06 | 2,504.57 |
| 16:00-20:00 | 57.98    | 5,766.67 | 20:00-24:00 | 5,708.45 | 2,845.49 |
| 00:00 储电    | 6,215.16 |          | 24:00 储电    | 6,420.72 |          |

对应结果见表 57。

表 57 问题 4-3（2025-12-21）紧急购电

| 时段          | 电量    |
|-------------|-------|
| 07:50-08:00 | 33.89 |

## 附录 B 支撑文件与计算程序

支撑文件包括 result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx 五份结果表，逐时供需与储能轨迹、价格预测及模型参数，以及以下完整计算程序。结果表保留现有布局；第一问为周期运行，按原模板区间起点映射同一日轨迹，末行次日 00:00—00:10 对应重复日首区间。

原始附件只读。程序依次完成数据读取、预测、风险校准、计划优化、实时执行及结果输出；时间和单位转换在输入阶段统一完成。完整运行说明随支撑文件提供。

## src/q12.py

"""Causal forecasts, MILP day-ahead dispatch and independent physical audits.

Run from the repository: python -m src.q12 --data-root <C 题 directory> --stage full  
All power observations are interpreted as interval means at right-end labels.

"""

```
from __future__ import annotations
```

```
import argparse
from dataclasses import asdict, dataclass
import hashlib
import json
import os
from pathlib import Path
import platform
import subprocess
import time

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
import numpy as np
import pandas as pd
import scipy
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import lil_matrix
```

```
@dataclass(frozen=True)
class Battery:
    minimum: float = 1200.0
    maximum: float = 10800.0
    initial: float = 6000.0
    max_power: float = 5000.0
    eta_c: float = 0.9
    eta_d: float = 0.9
    dt: float = 1 / 6

    @property
    def limit(self):
        return self.max_power * self.dt
```

```
B = Battery()
N = 144
TOL = 1e-5
```

```
def dump_json(path, obj):
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
```

```
def interval_label(i):
    def clock(m):
        return f"{m // 60:02d}:{m % 60:02d}"
    return f"{clock(i * 10)}-{clock((i + 1) * 10)}"
```

```
def time_minutes(value):
    if hasattr(value, "hour"):
        return value.hour*60 + value.minute
    text = str(value).strip()
    next_day = "+1" in text
    hour, minute, *_ = text.replace("+1", "").split(":")
    return int(hour)*60 + int(minute) + 1440*int(next_day)
```

```
def read_inputs(root):
    root = Path(root)
```

```

day = pd.read_excel(root / "附件/附件 1.xlsx", sheet_name=0, header=0)
load = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="小区负载", header=0)
pv = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="光伏发电实际功率", header=0)
dates = pd.to_datetime(load.iloc[:, 0])
if day.shape != (144, 4) or load.shape != (365, 145) or pv.shape != (365, 145):
    raise ValueError("Unexpected input dimensions")
if not dates.equals(pd.to_datetime(pv.iloc[:, 0])):
    raise ValueError("Load/PV dates differ")
if list(dates) != list(pd.date_range("2025-01-01", "2025-12-31")):
    raise ValueError("Dates must cover the complete ordered year")
expected_minutes = list(range(10, 1441, 10))
if any([time_minutes(v) for v in labels] != expected_minutes
        for labels in (load.columns[1:], pv.columns[1:], day.iloc[:, 0])):
    raise ValueError("Input time labels do not match")
data = {
    "dates": pd.DatetimeIndex(dates), "price": day.iloc[:, 1].to_numpy(float),
    "q1_load": day.iloc[:, 2].to_numpy(float) * B.dt,
    "q1_pv": day.iloc[:, 3].to_numpy(float) * B.dt,
    "load": load.iloc[:, 1:].to_numpy(float) * B.dt,
    "pv": pv.iloc[:, 1:].to_numpy(float) * B.dt,
}
audit = {"shapes": {"attachment1": list(day.shape), "load": list(load.shape), "pv": list(pv.shape)},
         "date_start": str(dates.iloc[0].date()), "date_end": str(dates.iloc[-1].date()),
         "duplicates": int(dates.duplicated().sum()), "input_files": [], "variables": {}}
for key in ["price", "q1_load", "q1_pv", "load", "pv"]:
    a = data[key]
    if not np.isfinite(a).all() or (a < 0).any():
        raise ValueError(f"Nonfinite or negative data in {key}; investigate, do not silently impute")
    audit["variables"][key] = {"count": int(a.size), "missing": 0, "min": float(a.min()),
                              "max": float(a.max()), "zero_count": int((a == 0).sum())}
if (data["price"] <= 0).any():
    raise ValueError("This implementation assumes strictly positive fixed prices")
for relative in ["C 题.pdf", "附件/附件 1.xlsx", "附件/附件 2.xlsx"]:
    raw = (root / relative).read_bytes()
    audit["input_files"].append({"path": relative, "bytes": len(raw), "sha256":
hashlib.sha256(raw).hexdigest()})
    audit["data_treatment"] = "No missing/negative values found; no observations deleted or imputed. Unusual
peaks retained."
return data, audit

```

```

def optimize_day(load, pv, price, initial, terminal=6000.0, battery=B, integer=True):
    """Nominal minimum-cost schedule, g/c/d/w/S/z in kWh; terminal is a lower bound.

```

For Q1 initial=terminal and terminal\_equal=True is obtained by the explicit upper bound below when requested by a (value, 'equal') tuple.

```

"""
n = len(price)
eq_terminal = isinstance(terminal, tuple)
target = terminal[0] if eq_terminal else terminal
size = 6 * n + 1
gi, ci, di, wi, si, zi = 0, n, 2*n, 3*n, 4*n, 5*n+1
objective = np.zeros(size)
objective[:n] = price
lb, ub = np.zeros(size), np.full(size, np.inf)
ub[ci:wi] = battery.limit
lb[si:zi], ub[si:zi] = battery.minimum, battery.maximum
lb[si] = ub[si] = initial
lb[si+n] = target
if eq_terminal:
    ub[si+n] = target
ub[zi:] = 1
integrality = np.zeros(size)
integrality[zi:] = int(integer)
a = lil_matrix((4*n, size))
lower, upper = np.zeros(4*n), np.zeros(4*n)
for t in range(n):
    a[t, gi+t], a[t, ci+t], a[t, di+t], a[t, wi+t] = 1, -1, 1, -1
    lower[t] = upper[t] = load[t] - pv[t]
    a[n+t, si+t+1], a[n+t, si+t] = 1, -1
    a[n+t, ci+t], a[n+t, di+t] = -battery.eta_c, 1 / battery.eta_d
    a[2*n+t, ci+t], a[2*n+t, zi+t] = 1, -battery.limit
    lower[2*n+t], upper[2*n+t] = -np.inf, 0
    a[3*n+t, di+t], a[3*n+t, zi+t] = 1, battery.limit
    lower[3*n+t], upper[3*n+t] = -np.inf, battery.limit
start = time.perf_counter()
result = milp(objective, integrality=integrality, bounds=Bounds(lb, ub),

```



```

        constraints=LinearConstraint(a.tocsc(), lower, upper),
        options={"time_limit": 30.0, "mip_rel_gap": 1e-7})
elapsed = time.perf_counter() - start
if not result.success or result.x is None:
    raise RuntimeError(f"MILP failed: status={result.status}, {result.message}")
x = result.x
sol = {"plan_kwh": x[:n], "charge_kwh": x[ci:di], "discharge_kwh": x[di:wi],
      "spill_kwh": x[wi:si], "soc_start_kwh": x[si:si+n], "soc_end_kwh": x[si+1:zi],
      "emergency_kwh": np.zeros(n), "load_kwh": np.asarray(load), "pv_kwh": np.asarray(pv)}
frame = pd.DataFrame(sol)
frame["price"] = price
meta = {"status": int(result.status), "cost": float(np.dot(price, x[:n])),
      "seconds": elapsed, "mip_gap": float(getattr(result, "mip_gap", 0) or 0),
      "dual_bound": float(getattr(result, "mip_dual_bound", result.fun) or result.fun)}
if integer:
    audit_trace(frame, battery=battery, initial=initial)
return frame, meta

def execute_day(plan, load, pv, initial, price, battery=B):
    """Causal real-time balancing; does not read future observations.

    Assumes instantaneous regulation within a ten-minute constant-power interval.
    Surplus charges first; deficit exhausts feasible discharge before emergency.
    """
    s = float(initial)
    records = []
    for t, (g, l, v) in enumerate(zip(plan, load, pv)):
        surplus = float(g + v - l)
        c = min(max(surplus, 0), battery.limit, max(0, (battery.maximum-s)/battery.eta_c))
        d = min(max(-surplus, 0), battery.limit, max(0, (s-battery.minimum)*battery.eta_d))
        e = max(0, -surplus-d)
        w = max(0, surplus-c)
        end = s + battery.eta_c*c - d/battery.eta_d
        records.append((g, l, v, c, d, e, w, s, end, price[t]))
        s = end
    frame = pd.DataFrame(records, columns=["plan_kwh", "load_kwh", "pv_kwh", "charge_kwh", "discharge_kwh",
      "emergency_kwh", "spill_kwh", "soc_start_kwh", "soc_end_kwh",
"price"], dtype=float)
    audit_trace(frame, battery=battery, initial=initial)
    return frame

def audit_trace(frame, battery=B, initial=None):
    """Independent vectorized equations, also applicable to saved full-year traces."""
    f = frame
    balance = f.plan_kwh + f.pv_kwh + f.discharge_kwh + f.emergency_kwh - f.load_kwh - f.charge_kwh -
f.spill_kwh
    evolution = f.soc_end_kwh - f.soc_start_kwh - battery.eta_c*f.charge_kwh + f.discharge_kwh/battery.eta_d
    continuity = f.soc_start_kwh.to_numpy()[1:] - f.soc_end_kwh.to_numpy()[:-1]
    metrics = {"max_balance_residual_kwh": float(np.max(np.abs(balance))),
      "max_soc_residual_kwh": float(np.max(np.abs(evolution))),
      "max_soc_discontinuity_kwh": float(np.max(np.abs(continuity))) if len(f)>1 else 0,
      "simultaneous_intervals": int(((f.charge_kwh > TOL) & (f.discharge_kwh > TOL)).sum()),
      "min_soc_kwh": float(min(f.soc_start_kwh.min(), f.soc_end_kwh.min())),
      "max_soc_kwh": float(max(f.soc_start_kwh.max(), f.soc_end_kwh.max()))}
    checks = [np.isfinite(f.select_dtypes(include="number").to_numpy()).all(),
      (f[["plan_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh", "spill_kwh"]].to_numpy() >= -
TOL).all(),
      metrics["max_balance_residual_kwh"] < TOL, metrics["max_soc_residual_kwh"] < TOL,
      metrics["max_soc_discontinuity_kwh"] < TOL, metrics["simultaneous_intervals"] == 0,
      metrics["min_soc_kwh"] >= battery.minimum-TOL, metrics["max_soc_kwh"] <= battery.maximum+TOL,
      f.charge_kwh.max() <= battery.limit+TOL, f.discharge_kwh.max() <= battery.limit+TOL]
    if initial is not None:
        checks.append(abs(f.soc_start_kwh.iloc[0]-initial) < TOL)
    if not all(checks):
        raise AssertionError(f"Physical audit failed: {metrics}")
    return {**metrics, "pass": True}

def features(history, day, dates):
    """Features for all 144 targets on a day, using completed days only."""
    if day < 7:
        raise ValueError("Seven completed days required")
    past = history[day-7:day]

```

```

h = (np.arange(N)+0.5)/N
dow = dates[day].dayofweek
cols = [past[-1], past[-2], past[0], past[-3:].mean(0), past.mean(0), past.std(0),
        np.full(N, past[-1].mean()), np.full(N, past[-3:].mean())]
for k in (1, 2, 3):
    cols.extend([np.sin(2*np.pi*k*h), np.cos(2*np.pi*k*h)])
cols.extend([np.full(N, float(dow == k)) for k in range(7)])
return np.column_stack(cols)

def ridge_fit(x, y, alpha):
    """Centered ridge via linear solve; intercept unpenalized, no sklearn required."""
    mean, scale = x.mean(0), x.std(0)
    scale[scale < 1e-10] = 1
    z = (x-mean)/scale
    intercept = float(y.mean())
    coef = np.linalg.solve(z.T@z + alpha*np.eye(z.shape[1]), z.T@(y-intercept))
    return mean, scale, coef, intercept

def forecasts(data, method, alphas=(10.0, 10.0), stop=365):
    pred, fit_dates = {}, []
    for k, alpha in zip(("load", "pv"), alphas):
        values = data[k]
        out = np.zeros((stop, N))
        model, fitted_on = None, -1
        all_x = {d: features(values, d, data["dates"]) for d in range(7, stop)}
        for d in range(stop):
            if d == 0:
                out[d] = data[f"q1_{k}"]
            elif d < 7:
                out[d] = values[max(0,d-3):d].mean(0)
            elif method == "seasonal" or d < 14:
                out[d] = 0.5*values[d-1] + 0.5*values[d-7]
            else:
                if model is None or d-fitted_on >= 7:
                    training_days = range(max(7,d-60), d)
                    x = np.concatenate([all_x[j] for j in training_days])
                    y = values[max(7,d-60):d].ravel()
                    model = ridge_fit(x, y, alpha)
                    fitted_on = d
                fit_dates.append({"variable": k, "decision_date": str(data["dates"][d].date()),
                                "fit_until": str(data["dates"][d-1].date()), "alpha": alpha,
                                "training_rows": len(y)})
                mean, scale, coef, intercept = model
                out[d] = ((all_x[d]-mean)/scale)@coef+intercept
            out[d] = np.maximum(out[d], 0)
            if k == "pv" and d:
                out[d, values[max(0,d-7):d].max(0) == 0] = 0
        pred[k] = out
    return pred, fit_dates

def reserve(data, forecast, day, quantile):
    if quantile == 0 or day < 7:
        return np.zeros(N)
    start = max(1, day-28)
    errors = (data["load"][start:day]-data["pv"][start:day]
              -forecast["load"][start:day]+forecast["pv"][start:day])
    # Pool +/- three slots; no wrap at the day boundary, no future residuals.
    return np.array([max(0.0, float(np.quantile(errors[:,max(0,t-3):min(N,t+4)], quantile))) for t in
                     range(N)])

def summarize_day(f, date, name, seconds, gap=0.0):
    normal = float(np.dot(f.plan_kwh, f.price))
    emergency = float(5*np.dot(f.emergency_kwh, f.price))
    return {"date": str(date.date()), "strategy": name, "normal_cost_yuan": normal,
            "emergency_cost_yuan": emergency, "total_cost_yuan": normal+emergency,
            "plan_kwh": float(f.plan_kwh.sum()), "emergency_kwh": float(f.emergency_kwh.sum()),
            "emergency_intervals": int((f.emergency_kwh>TOL).sum()), "spill_kwh": float(f.spill_kwh.sum()),
            "charge_kwh": float(f.charge_kwh.sum()), "discharge_kwh": float(f.discharge_kwh.sum()),
            "soc_start_kwh": float(f.soc_start_kwh.iloc[0]), "soc_end_kwh": float(f.soc_end_kwh.iloc[-1]),
            "solve_seconds": seconds, "mip_gap": gap}

```

```

def backtest(data, forecast, start, stop, initial, quantile, name, retain=True):
    s, daily, frames = initial, [], []
    for d in range(start, stop):
        buffer = reserve(data, forecast, d, quantile)
        nominal, meta = optimize_day(forecast["load"][d]+buffer, forecast["pv"][d], data["price"], s)
        f = execute_day(nominal.plan_kwh.to_numpy(), data["load"][d], data["pv"][d], s, data["price"])
        daily.append(summarize_day(f, data["dates"][d], name, meta["seconds"], meta["mip_gap"]))
        if retain:
            f.insert(0, "interval_start", pd.date_range(data["dates"][d], periods=N, freq="10min"))
            f.insert(1, "date", str(data["dates"][d].date()))
            f.insert(2, "slot", np.arange(N))
            f["forecast_load_kwh"], f["forecast_pv_kwh"], f["reserve_kwh"] = forecast["load"][d],
forecast["pv"][d], buffer
            frames.append(f)
            s = float(f.soc_end_kwh.iloc[-1])
            if retain and (d == start or data["dates"][d].day == 1):
                print(f"{name}: {data['dates'][d].date()}, cumulative cost={sum(x['total_cost_yuan'] for x in
daily):.2f}", flush=True)
            combined = pd.concat(frames, ignore_index=True) if frames else None
            if combined is not None:
                audit_trace(combined, initial=initial)
            return pd.DataFrame(daily), combined, s

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-root", type=Path, required=True)
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--stage", choices=["q1", "smoke", "full"], default="full")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    start = time.perf_counter()
    data, input_audit = read_inputs(args.data_root)
    dump_json(args.out/"input_audit.json", input_audit)
    q1, q1_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"))
    lp, lp_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"),
integer=False)
    q1.insert(0, "slot", np.arange(N)); q1.insert(1, "interval", [interval_label(i) for i in range(N)])
    q1.to_csv(args.out/"q1_intervals.csv", index=False)
    no_storage_cost = float(np.dot(np.maximum(data["q1_load"]-data["q1_pv"], 0), data["price"]))
    q1_meta.update({"no_storage_cost": no_storage_cost, "saving_percent": 100*(no_storage_cost-
q1_meta["cost"])/no_storage_cost,
                    "lp_cost_lower_bound": lp_meta["cost"], "lp_gap_yuan": q1_meta["cost"]-lp_meta["cost"],
                    "audit": audit_trace(q1, initial=B.initial), "plan_kwh": float(q1.plan_kwh.sum())})
    dump_json(args.out/"q1_summary.json", q1_meta)
    print("Q1", json.dumps(q1_meta, ensure_ascii=False), flush=True)
    if args.stage == "q1":
        return
    seasonal, _ = forecasts(data, "seasonal")
    tuning = []
    for alpha in (1.0, 10.0, 100.0):
        pred, _ = forecasts(data, "ridge", (alpha, alpha), stop=31)
        row = {"alpha": alpha}
        for key in ("load", "pv"):
            row[key+"_mae_kw"] = float(np.abs(pred[key][21:31]-data[key][21:31]).mean())/B.dt)
        tuning.append(row)
    alpha_load = min(tuning, key=lambda r: r["load_mae_kw"])["alpha"]
    alpha_pv = min(tuning, key=lambda r: r["pv_mae_kw"])["alpha"]
    ridge, fit_log = forecasts(data, "ridge", (alpha_load, alpha_pv))
    dump_json(args.out/"forecast_fit_log.json", fit_log)
    pd.DataFrame(tuning).to_csv(args.out/"ridge_validation.csv", index=False)
    _, _, validation_soc = backtest(data, seasonal, 0, 21, B.initial, 0, "warmup", retain=False)
    warmup, _, evaluation_soc = backtest(data, seasonal, 21, 31, validation_soc, 0, "warmup", retain=False)
    warmup.to_csv(args.out/"warmup_last10days.csv", index=False)
    validation = []
    for method, pred in (("seasonal", seasonal), ("ridge", ridge)):
        for q in (0.0, 0.7, 0.8, 0.9):
            daily, _, _ = backtest(data, pred, 21, 31, validation_soc, q, method, retain=False)
            validation.append({"forecast": method, "quantile": q, "cost_yuan": float(daily.total_cost_yuan.sum()),
                              "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
                              "end_soc_kwh": float(daily.soc_end_kwh.iloc[-1])})
    selected = min(validation, key=lambda r: r["cost_yuan"])
    pd.DataFrame(validation).to_csv(args.out/"policy_validation.csv", index=False)
    stop = 38 if args.stage == "smoke" else 365
    strategies = [("seasonal_q0", "seasonal", 0.0), ("ridge_q0", "ridge", 0.0)]

```

```

for method in ("seasonal", "ridge"):
    best = min((v for v in validation if v["forecast"]==method), key=lambda r: r["cost_yuan"])
    if best["quantile"]:
        strategies.append((f"{method}_q{best['quantile']:g}", method, best["quantile"]))
selected_name = f"{selected['forecast']}_q{selected['quantile']:g}"
all_daily, metrics = [], []
for name, method, q in strategies:
    pred = {"seasonal": seasonal, "ridge": ridge}[method]
    daily, trace, _ = backtest(data, pred, 31, stop, evaluation_soc, q, name)
    all_daily.append(daily)
    trace.to_csv(args.out/f"{name}_intervals.csv.gz", index=False, float_format="%.17g")
    audit = audit_trace(trace, initial=evaluation_soc)
    row = {"strategy": name, "forecast": method, "quantile": q, "selected_on_january": name==selected_name,
          "days": len(daily), "total_cost_yuan": float(daily.total_cost_yuan.sum()),
          "normal_cost_yuan": float(daily.normal_cost_yuan.sum()),
          "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
          "emergency_kwh": float(daily.emergency_kwh.sum()), "emergency_intervals": int(daily.emergency_intervals.sum()),
          "plan_kwh": float(daily.plan_kwh.sum()), "spill_kwh": float(daily.spill_kwh.sum()),
          "solve_seconds": float(daily.solve_seconds.sum()), "initial_soc_kwh": evaluation_soc,
          "final_soc_kwh": float(daily.soc_end_kwh.iloc[-1]), "audit": audit}
    for key in ("load", "pv"):
        error = (pred[key][31:stop]-data[key][31:stop])/B.dt
        row[key+"_mae_kw"], row[key+"_rmse_kw"] = float(np.abs(error).mean()), float(np.sqrt(np.mean(error**2)))
    metrics.append(row)
    daily = pd.concat(all_daily, ignore_index=True)
    daily.to_csv(args.out/f"q2_daily_metrics.csv", index=False)

dump_json(args.out/"q2_summary.json", {"selection": selected, "selected_strategy": selected_name, "strategies": metrics})
command = f'python -m src.q12 --data-root "{args.data_root}" --out "{args.out}" --stage {args.stage}'
manifest = {"stage": args.stage, "battery": asdict(B), "time_interpretation": "interval mean power, right-end labels",
           "alpha_load": alpha_load, "alpha_pv": alpha_pv, "validation_start": "2025-01-22", "validation_end": "2025-01-31",
           "evaluation_start": "2025-02-01", "evaluation_end": str(data["dates"][stop-1].date()),
           "validation_initial_soc": validation_soc, "evaluation_initial_soc": evaluation_soc,
           "terminal_nominal_soc_minimum": 6000, "refit_period_days": 7, "training_window_days": 60,
           "residual_window_days": 28, "pool_halfwidth_slots": 3, "seed": 2026, "deterministic": True,
           "selected_strategy": selected_name, "command": command, "total_seconds": time.perf_counter()-start,
           "python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__, "pandas": pd.__version__,
           "platform": platform.platform(), "git_head_before_run": subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip(),
           "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(), "input_files": input_audit["input_files"]}
dump_json(args.out/"run_manifest.json", manifest)
print("COMPLETE", json.dumps({"selected": selected_name, "seconds": manifest["total_seconds"], "days": stop-31}), flush=True)

if __name__ == "__main__":
    main()

```

## src/q34\_data.py

```
"""Strict attachment 3/4 parsing and causal hourly forecast alignment."""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
import openpyxl
from src.q12 import read_inputs, time_minutes, dump_json

def read_extended(root, audit_path=None):
    root=Path(root); data, base_audit=read_inputs(root)
    wb=openpyxl.load_workbook(root/'附件/附件 3.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==1461 and len(rows[0])==26
    assert list(rows[0][2:]) == [f'预报{i}小时' for i in range(1,25)]
    hourly=np.empty((365,4,24)); filled=0
    for day in range(365):
        date=data['dates'][day]
        for version in range(4):
            row=rows[1+day*4+version]
            if version==0:
                assert pd.Timestamp(row[0])==date
            elif row[0] in (None,''):
                filled+=1
            else: assert pd.Timestamp(row[0])==date
            assert time_minutes(row[1])==version*360
            hourly[day,version]=np.asarray(row[2:],float)
    wb=openpyxl.load_workbook(root/'附件/附件 4.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==366 and len(rows[0])==145
    assert [time_minutes(v) for v in rows[0][1:]]==list(range(10,1441,10))
    assert pd.DatetimeIndex([r[0] for r in rows[1:]]).equals(data['dates'])
    prices=np.asarray([r[1:] for r in rows[1:]],float)
    checks={}
    for name, values in [(('hourly_pv_kw', hourly), ('actual_price', prices))]:
        assert np.isfinite(values).all(), f'{name}: missing/nonfinite'
        assert (values==0).all(), f'{name}: negative values need investigation'
        if name=='actual_price': assert (values>0).all()
        q1,q3=np.quantile(values, [.25, .75]); iqr=q3-q1
        checks[name]={ 'shape':list(values.shape), 'count':values.size, 'missing_nonfinite':0,
            'negative':0, 'zero':int((values==0).sum()), 'min':float(values.min()), 'max':float(values.max()),
            'iqr_flags_retained':int(((values<q1-1.5*iqr)|(values>q3+1.5*iqr)).sum()),
            'deleted':0, 'imputed_numeric':0, 'winsorized':0, 'smoothed_observations':0}
    # An hourly point is an instantaneous forecast. Integrate its piecewise-linear
    # curve exactly over 10-minute cells. At lead 0 use the newest completed PV
    # interval (causally known) as a boundary proxy; midnight Jan 1 uses zero.
    pv=np.full((365,4,144), np.nan)
    for day in range(365):
        for version in range(4):
            start=version*36
            anchor=(data['pv'][day,start-1]*6 if start else
                data['pv'][day-1,-1]*6 if day else 0.0)
            vals=np.r_[anchor, hourly[day,version]]
            edges=np.arange(145-start)/6
            powers=np.interp(edges, np.arange(25), vals)
            pv[day,version,start:]=(powers[:-1]+powers[1:])/12
    data.update(hourly_pv=hourly, pv_issued=pv, actual_price=prices)
    audit={'pass':True, 'checks':checks, 'date_forward_fills_structural_only':filled,
        'dates':365, 'unique_issues':1460, 'issue_hours':[0,6,12,18],
        'forecast_horizons_hours':list(range(1,25)),
        'alignment':'Hourly lead endpoints; piecewise-linear integration over 10-minute cells; lead-zero
        anchor from last completed observed PV interval; only current-day suffix traded.',
        'input_files':base_audit['input_files']+[
            {'path':name, 'sha256':hashlib.sha256((root/name).read_bytes()).hexdigest()}
            for name in ['附件/附件 3.xlsx', '附件/附件 4.xlsx']],
        'unusable_past_slots':'NaN intentionally marks targets before each issue; never supplied to a
        solver',
        'auditor_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    if audit_path: dump_json(audit_path, audit)
    return data, audit

if __name__=='__main__':
```

```
import argparse, json
p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
p.add_argument('--out',default='artifacts/q34/input_audit.json');args=p.parse_args()
Path(args.out).parent.mkdir(parents=True,exist_ok=True)
_,audit=read_extended(args.data_root,args.out)
print(json.dumps(audit,ensure_ascii=False,indent=2))
```

## src/q4\_forecast.py

"""Direct 24-hour price forecasts at four issue times, with monthly causal fits.

Seasonal, ridge and GRU see the same day/week lag curves and calendar. GRU predicts a residual around the seasonal curve, not future observed prices.

```
"""
from pathlib import Path
import argparse
import copy
import hashlib
import json
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from src.q12 import ridge_fit, dump_json
from src.q34_data import read_extended

torch.set_num_threads(1)

def calendar(indices):
    hours=(indices%144+.5)/144
    dow=(indices//144+2)%7 # 2025-01-01 Wednesday
    return np.stack([np.sin(2*np.pi*hours),np.cos(2*np.pi*hours),
                    np.sin(2*np.pi*dow/7),np.cos(2*np.pi*dow/7)],axis=-1)

def features_at(flat,origin):
    assert origin>=1008
    target=np.arange(origin,origin+144)
    return np.column_stack([flat[target-144],flat[target-1008],calendar(target)]).astype('float32')

def ridge_features(x):
    a,b=x[...,:0],x[...,:1]
    context=np.stack([a.mean(-1),b.mean(-1),a.std(-1),b.std(-1)],axis=-1)
    context=np.broadcast_to(context[...,:None,:],(*a.shape,4))
    return np.concatenate([x,np.stack([a-b,a*b],axis=-1),context],axis=-1)

class PriceGRU(nn.Module):
    def __init__(self):
        super().__init__();self.gru=nn.GRU(6,32,batch_first=True);self.head=nn.Linear(32,1)
        nn.init.zeros_(self.head.weight);nn.init.zeros_(self.head.bias)
    def forward(self,x):
        h,_=self.gru(x)
        return .5*(x[:, :, :0]+x[:, :, :1])+self.head(h).squeeze(-1)

def train_gru(x,y,origins,cutoff,seed,max_epochs=50):
    train=(origins+144<=cutoff-3*144);valid=(origins>=cutoff-3*144)
    assert train.sum()>0 and valid.sum()>0
    torch.manual_seed(seed);torch.use_deterministic_algorithms(True)
    mean=float(x[train,:, :2].mean());scale=max(float(x[train,:, :2].std()),1e-6)
    z=x.copy();z[:, :, :2]=(z[:, :, :2]-mean)/scale;target=(y-mean)/scale

    tx=torch.tensor(z[train]);ty=torch.tensor(target[train]);vx=torch.tensor(z[valid]);vy=torch.tensor(target[valid])
    model=PriceGRU();opt=torch.optim.Adam(model.parameters(),lr=.003)
    best=float('inf');weights=None;stale=0;history=[];rng=np.random.default_rng(seed)
    for epoch in range(max_epochs):
        model.train();order=rng.permutation(len(tx));losses=[]
        for left in range(0,len(order),32):
            idx=order[left:left+32];opt.zero_grad()
            loss=nn.functional.smooth_l1_loss(model(tx[idx]),ty[idx])
        loss.backward();nn.utils.clip_grad_norm_(model.parameters(),1.0);opt.step();losses.append(float(loss.detach()))
    model.eval()
```

```

        with torch.no_grad():score=float((model(vx)-vy).abs().mean())*scale
history.append({'epoch':epoch+1,'training_huber':float(np.mean(losses)), 'inner_validation_mae':score})
    if score<best-1e-7:best=score;weights=copy.deepcopy(model.state_dict());stale=0
    else:stale+=1
    if stale>=6:break
model.load_state_dict(weights);model.eval()
return model,mean,scale,history,int(train.sum()),int(valid.sum())

def same_day_errors(pred,actual,start,stop):
    values=[]
    for day in range(start,stop):
        for v in range(4):values.extend(pred[day,v,:144-v*36]-actual[day,v*36:])
    a=np.asarray(values)
    return {'mae':float(np.abs(a).mean()), 'rmse':float(np.sqrt(np.mean(a*a))), 'targets':len(a)}

def main():
    p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
    p.add_argument('--out',type=Path,default=Path('artifacts/q34'));p.add_argument('--
smoke',action='store_true')

args=p.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True);(out/'models').mkdir(exist_ok=True)
    data,_=read_extended(args.data_root);flat=data['actual_price'].ravel();started=time.perf_counter()
    stop=38 if args.smoke else 365
    origins=np.arange(1008,len(flat)-143,36)
    x=np.stack([features_at(flat,t) for t in origins]);y=np.stack([flat[t:t+144] for t in
origins]).astype('float32')
    # Fit on Jan 15/22 then each month boundary; never use targets ending after fit time.
    fit_days=[d for d in range(14,stop) if d in [14,21] or data['dates'][d].day==1]
    result={name:np.full((stop,4,144),np.nan,dtype='float64') for name in
['seasonal','ridge1','ridge10','ridge100','gru17','gru42','gru2026']}
    all_logs=[]
    for day in range(7,stop):
        for v in range(4):
            sample=features_at(flat,day*144+v*36)
            result['seasonal'][day,v]=sample[:,2].mean(-1)
    for fi,day in enumerate(fit_days):
        cutoff=day*144;end=fit_days[fi+1] if fi+1<len(fit_days) else stop
        choose=(origins+144<=cutoff)&(origins>=max(1008,cutoff-60*144))
        xx=x[choose];yy=y[choose];oo=origins[choose]
        assert (oo+144<=cutoff).all()
        future_x=np.stack([features_at(flat,d*144+v*36) for d in range(day,end) for v in range(4)])
        for alpha in [1.,10.,100.]:
            a=ridge_features(xx).reshape(-1,12);b=yy.ravel()
            model=ridge_fit(a,b,alpha);mean,scale,coef,intercept=model
            forecasts=((ridge_features(future_x)-mean)/scale)@coef+intercept
            result[f'ridge{alpha:g}'][day:end]=np.maximum(forecasts.reshape(end-day,4,144),.001)
        for seed in [17,42,2026]:
t=time.perf_counter();model,mean,scale,history,nt,nv=train_gru(xx,yy,oo,cutoff,seed+day,max_epochs=12 if
args.smoke else 50)
            z=future_x.copy();z[:,2,:]=z[:,2,:]-mean/scale
            with torch.no_grad():values=model(torch.tensor(z)).numpy()*scale+mean
            result[f'gru{seed}'][day:end]=np.maximum(values.reshape(end-day,4,144),.001)
            checkpoint=out/'models'/f'gru{seed}_fit{day:03d}.pt'
            torch.save({'state_dict':model.state_dict(),'mean':mean,'scale':scale,'fit_day':day,'seed':seed,
'training_last_target_exclusive':cutoff,'architecture':'GRU(6,32)+Linear(32,1)+seasonal
residual'},checkpoint)

log={'fit_day':day,'fit_date':str(data['dates'][day].date()),'seed':seed,'actual_rng_seed':seed+day,
'training_windows':nt,'inner_validation_windows':nv,'last_target_exclusive':int((oo+144).max()),
'seconds':time.perf_counter()-t,'history':history,'checkpoint':str(checkpoint)}
    all_logs.append(log)
    print(f'price GRU seed={seed} fit={day}: {len(history)} epochs, {log["seconds"]:.1f}s, best
inner MAE={min(r["inner_validation_mae"] for r in history):.4f}',flush=True)
    np.savez_compressed(out/'price_forecasts_checkpoint.npz',**result)
    dump_json(out/'price_fit_log.json',all_logs)
    for name in result:
        if name!='seasonal':result[name][7:14]=result['seasonal'][7:14]
        result['gru_mean']=np.mean([result[f'gru{s}'] for s in [17,42,2026]],axis=0)
        validation={name:same_day_errors(values,data['actual_price'],21,31) for name,values in result.items()}
        ridge_name=min(['ridge1','ridge10','ridge100'],key=lambda n:validation[n]['mae'])
        result['ridge']=result[ridge_name]
        chosen=min(['seasonal','ridge','gru_mean'],key=lambda n:validation[ridge_name if n=='ridge' else

```



```

n]['mae'])
    metrics={name:same_day_errors(result[name],data['actual_price'],31,stop) for name in
['seasonal','ridge','gru17','gru42','gru2026','gru_mean']}
    for name in ['seasonal','ridge','gru17','gru42','gru2026','gru_mean']:assert
np.isfinite(result[name][21:]).all()
    np.savez_compressed(out/'price_forecasts.npz',**result)

dump_json(out/'price_forecast_summary.json',{'validation':validation,'evaluation':metrics,'ridge_selected':r
idge_name,
        'selected_by_validation_price_mae':chosen,'seeds':[17,42,2026],'torch_version':torch.__version__,
        'epochs_max':12 if args.smoke else 50,'fit_days':fit_days,'elapsed_seconds':time.perf_counter()-
started,
        'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        'evaluation_note':'Same-day suffix only, at each issue time; repeated targets at different origins
counted separately.',
        'training_rule':'Whole 24h targets end before fit; last 3 days internal validation; refit monthly
from scratch, last 60 days; Jan15/22 initialization.',
        'clip_floor':.001,'architecture':'Direct aligned day/week lag sequences plus known calendar;
residual GRU(6,32), no future teacher forcing'})

if __name__=='__main__':main()

```

## src/v3\_model.py

```

"""Exact LP reformulation and causal forecast-corrected dispatch.

Energy in kWh. Same settlement, time axis and realtime regulation as V1.
No training or tuning is performed here; inputs explicitly carry decisions.
"""

from dataclasses import dataclass, asdict
from functools import lru_cache
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from src.q12 import B, TOL, audit_trace, execute_day

@lru_cache(maxsize=32)
def structure(n, revision):
    # g,c,d,w,S[,increase,decrease]; state has n+1 boundary values.
    size = (7 if revision else 5)*n+1
    a = lil_matrix(((3 if revision else 2)*n, size))
    for t in range(n):
        a[t,t], a[t,n+t], a[t,2*n+t], a[t,3*n+t] = 1,-1,1,-1
        a[n+t,4*n+t+1], a[n+t,4*n+t] = 1,-1
        a[n+t,n+t], a[n+t,2*n+t] = -B.eta_c,1/B.eta_d
        if revision:
            a[2*n+t,t], a[2*n+t,5*n+1+t], a[2*n+t,6*n+1+t] = 1,-1,1
    return a.tocsr()

def remove_cycles(charge, discharge, spill):
    """Preserve grid purchases and ALL SOC values, eliminate simultaneous flow.

    x=min(c,d/(eta_c*eta_d)); c'=c-x; d'=d-eta_c*eta_d*x;
    w'=w+(1-eta_c*eta_d)*x. Requires free unbounded curtailment.
    """
    rho=B.eta_c*B.eta_d
    x=np.minimum(charge,discharge/rho)
    return charge-x,discharge-rho*x,spill+(1-rho)*x

def solve(load,pv,price,initial,terminal=6000.,old=None,refund=True,equal=False):
    load,pv,price=map(lambda x:np.asarray(x,dtype=float),(load,pv,price))
    n=len(price);revision=old is not None;size=(7 if revision else 5)*n+1
    if not all(x.shape==(n,) and np.isfinite(x).all() for x in (load,pv,price)):
        raise ValueError('Nonfinite or misaligned dispatch inputs')
    if np.any(price<=0) or not B.minimum<=terminal<=B.maximum:
        raise ValueError('Positive prices and admissible terminal storage required')
    bounds=[(0,None)]*size
    bounds[n:3*n]=[(0,B.limit)]*(2*n)
    bounds[4*n:5*n+1]=[(B.minimum,B.maximum)]*(n+1)
    bounds[4*n]=(initial,initial);bounds[5*n]=(terminal,terminal if equal else B.maximum)
    cost=np.zeros(size)
    rhs=np.r_[load-pv,np.zeros(n)]
    if revision:
        old=np.asarray(old,dtype=float)
        assert old.shape==(n,) and np.isfinite(old).all() and old.min()>=-TOL
        cost[5*n+1:6*n+1]=1.5*price
        cost[6*n+1:]=(-.5 if refund else .5)*price
        bounds[6*n+1:]=[(0,max(float(v),0)) for v in old]
        rhs=np.r_[rhs,old]
    else:cost[:n]=price
    t=time.perf_counter()
    fit=linprog(cost,A_eq=structure(n,revision),b_eq=rhs,bounds=bounds,method='highs',
        options={'time_limit':30,'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
    if not fit.success:raise RuntimeError(f'LP status={fit.status}: {fit.message}')
    x=fit.x;c,d,w=remove_cycles(x[n:2*n],x[2*n:3*n],x[3*n:4*n])
    f=pd.DataFrame(dict(plan_kwh=x[:n],charge_kwh=c,discharge_kwh=d,spill_kwh=w,
        soc_start_kwh=x[4*n:5*n],soc_end_kwh=x[4*n+1:5*n+1],load_kwh=load,pv_kwh=pv,
        emergency_kwh=np.zeros(n),price=price))
    physical=audit_trace(f,initial=initial)
    return f,dict(objective=float(fit.fun),seconds=time.perf_counter()-t,status=int(fit.status),

```

```

iterations=int(fit.nit),physical=physical)

@dataclass(frozen=True)
class Policy:
    quantile:float=.7
    terminal:float=6000.
    mask:int=0
    beta:float=0.
    refund:bool=True
    lookback:int=18
    decay:int=36

def corrected_load(data,pred,beta,lookback=18,decay=36):
    """At each release, use only the just-completed <=3h load residuals.

    No observed PV is used to correct load; no current/future interval enters.
    Correction decays with lead, preventing all-day propagation of a short shock.
    """
    out=np.repeat(pred['load'][:,None,:],4,axis=1).copy()
    for v in range(1,4):
        start=36*v;left=max(0,start-lookback)
        err=(data['load'][:,left:start]-pred['load'][:,left:start]).mean(axis=1)
        lead=np.arange(144-start)+.5
        out[:,v,start:]=np.maximum(0,out[:,v,start:]+beta*err[:,None]*np.exp(-lead[None,:]/decay))
    return out

def risk_buffers(data,load_pred,pv_pred,q):
    out=np.zeros_like(load_pred)
    if q==0:return out
    for day in range(7,len(load_pred)):
        left=max(1,day-28)
        error=data['load'][:,left:day,None,:]-data['pv'][:,left:day,None,:]-load_pred[left:day]+pv_pred[left:day]
        for v in range(4):
            start=36*v
            # Historical same-release residuals, not errors from unreleased versions.
            for slot in range(start,144):
                out[day,v,slot]=max(0,float(np.quantile(error[:,v,max(start,slot-3):min(144,slot+4)],q)))
    return out

def run(data,load_pred,pv_pred,buffers,policy,*,start,stop,initial,name,prices=None,retain=True):
    state=float(initial);frames=[];days=[];ledger=[];solves=[]
    for day in range(start,stop):
        date=str(data['dates'][day].date());s0=state
        realized=data['price'] if prices is None else data['actual_price'][day]
        p0=data['price'] if prices is None else prices[day,0,:144]
        sol,meta=solve(load_pred[day,0]+buffers[day,0],pv_pred[day,0],p0,state,policy.terminal)
        solves.append({'date':date,'version':0,**{k:meta[k] for k in
['seconds','status','iterations','objective']})
        plan=np.maximum(sol.plan_kwh.to_numpy(),0);original=plan.copy()
        up=np.zeros(144);down=np.zeros(144);blocks=[]
        for v in range(4):
            first=v*36;last=first+36
            if v and policy.mask & (1<<(v-1)):
                p=data['price'][first:] if prices is None else prices[day,v,:144-first]
                old=plan[first:].copy()
                sol,meta=solve(load_pred[day,v,first:]+buffers[day,v,first:],pv_pred[day,v,first:],
                    p,state,policy.terminal,original[first:],policy.refund)
                solves.append({'date':date,'version':v,**{k:meta[k] for k in
['seconds','status','iterations','objective']})
                new=np.maximum(sol.plan_kwh.to_numpy(),0);delta=new-old
                up[first:]+=np.maximum(delta,0);down[first:]+=np.maximum(-delta,0);plan[first:]=new
                if retain:
                    for j,slot in enumerate(range(first,144)):
                        ledger.append([date,v,first,slot,float(old[j]),float(new[j]),max(float(delta[j]),0),
                            max(-float(delta[j]),0),float(p[j]),float(load_pred[day,v,slot]),
                                float(pv_pred[day,v,slot]),float(buffers[day,v,slot]),state)])
    f=execute_day(plan[first:last],data['load'][day,first:last],data['pv'][day,first:last],state,realized[first:
last])
    state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

```

```

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',np.arange(144))
# Publication versions control execution; the final bill is relative to
# the midnight contract, not the sum of intermediate revision movements.
up=np.maximum(plan-original,0);down=np.maximum(original-plan,0)
f['original_plan_kwh']=original;f['increase_kwh']=up;f['decrease_kwh']=down
f['original_cost_yuan']=realized*original;f['increase_cost_yuan']=realized*1.5*up
f['decrease_cost_yuan']=realized*(-.5 if policy.refund else .5)*down
f['emergency_cost_yuan']=realized*5*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
np.testing.assert_allclose(f.plan_kwh,original+up-down,atol=TOL,rtol=0)
item={'date':date,'strategy':name,'soc_start_kwh':s0,'soc_end_kwh':state}
for c in f:
    if c.endswith('_yuan') or c in ['original_plan_kwh','plan_kwh','increase_kwh','decrease_kwh',
        'emergency_kwh','spill_kwh','charge_kwh','discharge_kwh']:item[c]=float(f[c].sum())
    item['emergency_intervals']=int((f.emergency_kwh>TOL).sum());days.append(item)
    if retain:frames.append(f)
daily=pd.DataFrame(days);trace=pd.concat(frames,ignore_index=True) if retain else None
versions=pd.DataFrame(ledger,columns=['date','version','effective_slot','slot','old_kwh','new_kwh',
'increase_kwh','decrease_kwh','forecast_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision'
_soc_kwh'])
summary={'strategy':name,'policy':asdict(policy),'days':stop-
start,'initial_soc':initial,'final_soc':state,
'solver_seconds':sum(x['seconds'] for x in solves),'solver_calls':len(solves),
'physical':audit_trace(trace,initial=initial) if retain else None,
**{c:float(daily[c].sum()) for c in daily if c.endswith('_yuan') or c in
['emergency_kwh','spill_kwh']}}
return daily,trace,versions,summary,pd.DataFrame(solves)

```

## src/v3\_q1.py

```
"""Q1 template-start sampling with explicit periodic midnight closure."""
from pathlib import Path
from dataclasses import replace
import argparse
import numpy as np
import pandas as pd
from src.q12 import read_inputs, optimize_day, B, dump_json, interval_label, audit_trace
from src.v3_model import solve

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    data,_=read_inputs(a.data_root); out=Path('artifacts/v3'); out.mkdir(parents=True, exist_ok=True)
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    f,meta=solve(L,V,p,6000,equal=True);_,milp=optimize_day(L,V,p,6000,(6000,'equal'))
    assert abs(meta['objective']-milp['cost'])<1e-6
    f.insert(0,'slot',range(144)); f.insert(1,'interval',[interval_label(i) for i in range(144)])
    f.to_csv(out/'q1_intervals.csv',index=False,float_format='%.17g')
    sensitivity=[]
    for eta in [.8,.85,.9,np.sqrt(.9),1.]:
        b=replace(B,eta_c=eta,eta_d=eta); ff,mm=optimize_day(L,V,p,6000,(6000,'equal'),battery=b)
        audit_trace(ff,battery=b,initial=6000)
        sensitivity.append({'single_efficiency':eta,'roundtrip_efficiency':eta*eta,'cost':mm['cost']})
    pd.DataFrame(sensitivity).to_csv(out/'q1_efficiency.csv',index=False)
    dump_json(out/'q1_summary.json',{'cost':meta['objective'],'no_storage_cost':float(p@np.maximum(L-V,0)),
    'plan_kwh':float(f.plan_kwh.sum()),'charge_kwh':float(f.charge_kwh.sum()),'discharge_kwh':float(f.discharge_
    kwh.sum()),
    'milp':milp,'lp':meta,'specified':{interval_label(i):float(f.plan_kwh.iloc[i]) for i in
    [60,72,84,96,108,120]}},
    'alignment':'Template start timestamps; Q1 periodic 24:00 sample fills 00:00 and all signals are
    jointly rotated; solve again with actual 00:00=24:00 SOC=6000.',
    'historical_series':'Q2-Q4 retained interval-average/right-end convention; no same-day final sample
    copied into their midnight histories.')}
    print('Q1',meta['objective'],milp['cost'])

if __name__=='__main__':main()
```

## src/v3\_experiments.py

```

"""Final-versus-midnight settlement and paired price-information experiments."""
from pathlib import Path
import argparse,json,time,hashlib
import numpy as np
import pandas as pd
from src.q12 import forecasts,dump_json
from src.q34_data import read_extended
from src.v3_model import Policy,corrected_load,risk_buffers,run,solve

OUT=Path('artifacts/v3')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args()
    OUT.mkdir(parents=True,exist_ok=True);beg=time.perf_counter()
    data,audit=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.))
    pp=np.load('artifacts/q34/price_forecasts.npz');ll=corrected_load(data,pred,0)
    pv2=np.repeat(pred['pv'][:,None,:],4,axis=1);pv3=data['pv_issued']
    cache={}
    def calc(mode,policy,start,stop,label,prices=None,save=False):
        pv=pv2 if mode==2 else pv3;k=(mode,policy.quantile,stop)
        if k not in cache:cache[k]=risk_buffers(data,ll[:stop],pv[:stop],policy.quantile)
        r=run(data,ll,pv,cache[k],policy,start=start,stop=stop,
            initial=6244.256891388887 if start==21 else 9902.811287870369,
            name=label,prices=prices,retain=save)
        if save:
            for obj,suffix in
zip([r[0],r[1],r[2],r[4]],['daily.csv','intervals.csv.gz','versions.csv.gz','solvers.csv.gz']):
                if len(obj):obj.to_csv(OUT/f'{label}_{suffix}',index=False,float_format='%.17g')

dump_json(OUT/f'{label}_summary.json',r[3]);print(label,round(r[3]['total_cost_yuan'],2),flush=True)
        return r
    # Fixed design preceding the new evaluation: exact Q3 mechanism, three risks,
    # all update subsets; Q4 fair price comparison with unchanged physical policy.
    design={'q3_quantiles':[0,.5,.7],'q3_masks':list(range(8)),'q4_models':['seasonal','ridge','gru_mean'],
        'q4_2_policy':{'quantile':.7,'terminal':6000},'settlement':'final vs midnight; no fees for
intermediate reversals',
        'scope':'same-year revision; no claim of untouched external test set'}
    dump_json(OUT/'design.json',design);rows=[]
    for q in design['q3_quantiles']:
        for mask in range(8):
            r=calc(3,Policy(quantile=q,mask=mask),21,31,'validation')
            rows.append({'q':q,'mask':mask,'cost':r[3]['total_cost_yuan']})
    pd.DataFrame(rows).to_csv(OUT/'q3_validation.csv',index=False)
    best=min(rows,key=lambda x:(x['cost'],x['mask'],x['q']));pol=Policy(quantile=best['q'],mask=best['mask'])
    pvals=[]
    for mode in [2,3]:
        for m in design['q4_models']:
            r=calc(mode,Policy() if mode==2 else pol,21,31,'validation',pp[m])
            pvals.append({'mode':mode,'model':m,'cost':r[3]['total_cost_yuan']})
    selected={str(mode):min([r for r in pvals if r['mode']==mode],key=lambda x:x['cost'])['model'] for mode
in [2,3]}

dump_json(OUT/'selection.json',{'q3':best,'prices':selected});pd.DataFrame(pvals).to_csv(OUT/'q4_validation.
csv',index=False)
    summaries=[]
    for mask in
range(8):summaries.append(calc(3,Policy(quantile=pol.quantile,mask=mask),31,365,f'q3_m{mask}',save=True)[3])
        for mode in [2,3]:
            for m in design['q4_models']:
                summaries.append(calc(mode,Policy() if mode==2 else pol,31,365,f'q4_{mode}_{m}',pp[m],True)[3])
    # Actual execution comparison with clairvoyant prices only (NOT a bound on
    # an unknown global stochastic optimum): future demand/PV remain forecasts.
    oracle=np.full((365,4,144),np.nan)
    for v in range(4):oracle[:,v,:144-v*36]=data['actual_price'][:,v*36:]
    for mode in [2,3]:summaries.append(calc(mode,Policy() if mode==2 else
pol,31,365,f'q4_{mode}_oracle',oracle,True)[3])
    dump_json(OUT/'summary.json',summaries)
    # Provable nominal-stage bound: identical 334 forecast feasible sets and
    # initial states for every price predictor, evaluated with true prices.
    states=pd.read_csv(OUT/f'q4_2_{selected["2"]}daily.csv').soc_start_kwh.to_numpy()
    rr=cache[(2,.7,365)];regrets=[]
    for i,day in enumerate(range(31,365)):
        ptrue=data['actual_price'][day];L=ll[day,0]+rr[day,0];V=pv2[day,0]
        f,meta=solve(L,V,ptrue,states[i]);lower=float(ptrue@f.plan_kwh)
        for model in ['seasonal','ridge','gru_mean']:
            plan,_=solve(L,V,pp[model][day,0],states[i]);cost=float(ptrue@plan.plan_kwh)

```

```

        assert cost>=lower-1e-5

regrets.append({'date':str(data['dates'][day].date()),'model':model,'nominal_cost':cost,'oracle_lower':lower
,'regret':cost-lower})
    pd.DataFrame(regrets).to_csv(OUT/'nominal_price_bound.csv',index=False,float_format='%.17g')
    dump_json(OUT/'run_manifest.json',{'seconds':time.perf_counter()-beg,'input_audit':audit,
        'sources':{f:hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest() for f in
['src/v3_model.py','src/v3_experiments.py']},
        'bound_scope':'fixed common nominal demand/PV forecasts, risk allowance, initial SOC and terminal
constraints; no emergency execution claim'})
    print('COMPLETE',round(time.perf_counter()-beg,2),flush=True)

if __name__=='__main__':main()

```

## src/v3\_verify.py

```

"""Reconstruct final contracts independently and compare all physical traces."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import pandas as pd
from src.q12 import audit_trace, dump_json, TOL
from src.q34_data import read_extended

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    out=Path('artifacts/v3'); data,_=read_extended(a.data_root); checks={}
    manifest=json.loads((out/'run_manifest.json').read_text(encoding='utf8'))
    for f,h in manifest['sources'].items(): assert
    hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest()==h
    summaries=json.loads((out/'summary.json').read_text(encoding='utf8'))
    for s in summaries:

name=s['strategy']; f=pd.read_csv(out/f'{name}_intervals.csv.gz'); daily=pd.read_csv(out/f'{name}_daily.csv')
    assert len(f)==334*144 and len(daily)==334
    prices=np.tile(data['price'],334) if name.startswith('q3') else data['actual_price'][:31:].reshape(-1)
    for col,expected in [(('load_kwh',data['load'][:31:].reshape(-1)),('pv_kwh',data['pv'][:31:].reshape(-1))),('price',prices)]:
        np.testing.assert_allclose(f[col],expected,atol=1e-10,rtol=0)
        physical=audit_trace(f,initial=9902.811287870369)
        vp=out/f'{name}_versions.csv.gz'; vs=pd.read_csv(vp) if vp.exists() else pd.DataFrame()
        groups={d:g for d,g in vs.groupby('date')} if len(vs) else {}
        for date,g in f.groupby('date',sort=False):
            plan=g.original_plan_kwh.to_numpy().copy()
            if date in groups:
                for ver,v in groups[date].groupby('version',sort=True):

slots=v.slot.to_numpy(int); np.testing.assert_array_equal(slots,np.arange(int(ver)*36,144))
        np.testing.assert_allclose(v.old_kwh,plan[slots],atol=TOL,rtol=0)

np.testing.assert_allclose(v.decision_soc_kwh,g.soc_start_kwh.iloc[int(ver)*36],atol=TOL,rtol=0)
        plan[slots]=v.new_kwh
        np.testing.assert_allclose(plan,g.plan_kwh,atol=TOL,rtol=0)
        # Independent formula uses retained quantity + positive penalties, not
        # the solver's original +/- accounting expression.
        x=f.original_plan_kwh.to_numpy(); y=f.plan_kwh.to_numpy(); u=np.maximum(x-y,0); v=np.maximum(y-x,0)

np.testing.assert_allclose(f.decrease_kwh,u,atol=TOL,rtol=0); np.testing.assert_allclose(f.increase_kwh,v,atol=TOL,rtol=0)
        total=prices*(np.minimum(x,y)+.5*u+.5*v+.5*f.emergency_kwh.to_numpy())
        np.testing.assert_allclose(total,f.total_cost_yuan,atol=TOL,rtol=0)
        np.testing.assert_allclose(total.reshape(334,144).sum(1),daily.total_cost_yuan,atol=1e-6,rtol=0)
        assert abs(total.sum()-s['total_cost_yuan'])<1e-5
        for col in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']:
            assert abs(f[col].sum()-s[col])<1e-5
            solver=pd.read_csv(out/f'{name}_solvers.csv.gz'); assert (solver.status==0).all()

checks[name]={ 'physical':physical, 'cost':float(total.sum()), 'versions':len(vs), 'solver_calls':len(solver)}
    assert len(checks)==16
    q1=pd.read_csv(out/'q1_intervals.csv'); audit_trace(q1,initial=6000)
    for col,key in
[(('load_kwh','q1_load'),('pv_kwh','q1_pv'),('price','price')): np.testing.assert_allclose(q1[col],np.roll(data[key],1),atol=1e-10)
a[key],1),atol=1e-10)
        b=pd.read_csv(out/'nominal_price_bound.csv'); assert len(b)==1002 and b.regret.min()>-1e-5
        # Paired GRU vs seasonal actual savings with temporal block dependence.
        df0=pd.read_csv(out/'q4_3_seasonal_daily.csv'); df1=pd.read_csv(out/'q4_3_gru_mean_daily.csv')
        diff=(df0.total_cost_yuan-df1.total_cost_yuan).to_numpy(); rng=np.random.default_rng(20260911); cis={}
        for length in [7,14,28]:
            starts=rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/length))))
            ids=((starts[:,:,:None]+np.arange(length))%len(diff)).reshape(2000,-1)[:,:len(diff)]
            cis[str(length)]=np.quantile(diff[ids].sum(1),[.025,.975]).tolist()

dump_json(out/'verification.json',{'pass':True,'traces':checks,'q1_pass':True,'nominal_bound_cases':len(b),
'gru_saving_yuan':float(diff.sum()),'gru_saving_block95':cis,'daily_series_exact_order':True})
    print('PASS:16 full-year traces, final settlements, Q1 alignment, 1002 paired nominal cases',cis)

if __name__=='__main__':main()

```



## src/deliver\_q12.py

"""Build the Q1/Q2 manuscript, plotting inputs, figures and Excel payload.

This never refits forecasts or changes dispatch decisions. Inputs are saved results.

"""

```
from __future__ import annotations
import argparse
from dataclasses import replace
import hashlib
import json
from pathlib import Path
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from src.q12 import B, TOL, audit_trace, dump_json, interval_label, optimize_day
```

```
LABELS = {"seasonal_q0": "同期预测 无余量", "ridge_q0": "岭回归 无余量",
          "seasonal_q0.7": "同期预测 70%余量", "ridge_q0.7": "岭回归 70%余量"}
COLORS = {"load": "#3E608D", "pv": "#E8A33D", "pv_fill": "#F0C284", "grid": "#4D779B",
          "soc": "#8074C8", "emergency": "#992224", "forecast": "#7895C1", "actual": "#CD3B42", "base": "#9D9EA3"}
DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
```

```
def md_table(headers, rows):
    def fmt(x):
        return f"{x:,.2f}" if isinstance(x, (float, np.floating)) else str(x)
    return "\n".join([" | "+" | ".join(headers)+" |", " | "+" | ".join(["---"]*len(headers))+ " |"+
                      [" | "+" | ".join(fmt(x) for x in r)+" |" for r in rows])
```

```
def blocks(f):
    return [[f"{h:02d}:{00}-{h+4:02d}:{00}", float(f.iloc[h*6:(h+4)*6].charge_kwh.sum()),
            float(f.iloc[h*6:(h+4)*6].discharge_kwh.sum())] for h in range(0, 24, 4)]
```

```
def emergency_runs(f):
    positive = f.emergency_kwh.to_numpy() > TOL
    result = []; i = 0
    while i < len(f):
        if not positive[i]:
            i += 1; continue
        j = i + 1
        while j < len(f) and positive[j]: j += 1
        label = interval_label(i).split("-")[0] + "-" + interval_label(j-1).split("-")[1]
        result.append([label, float(f.emergency_kwh.iloc[i:j].sum())]); i = j
    return result
```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--skill-root", type=Path, default=Path.home()/.codex/skills/math-modeling")
    args = parser.parse_args(); out = args.out
    sys.path.insert(0, str(args.skill_root/"tools/figure/scripts"))
    from export_figure import export_figure
    from visual_qa import audit_layout
    q1 = pd.read_csv(out/"q1_intervals.csv")
    a = json.loads((out/"q1_summary.json").read_text(encoding="utf-8"))
    summary = json.loads((out/"q2_summary.json").read_text(encoding="utf-8"))
    manifest = json.loads((out/"run_manifest.json").read_text(encoding="utf-8"))
    if manifest["stage"] != "full": raise ValueError("Full-year outputs required")
    selected = summary["selected_strategy"]
    f = pd.read_csv(out/f"{selected}_intervals.csv.gz")
    f["interval_start"] = pd.to_datetime(f.interval_start)
    daily = pd.read_csv(out/"q2_daily_metrics.csv")
    chosen_daily = daily[daily.strategy == selected].set_index("date")
    val = pd.read_csv(out/"policy_validation.csv")
```

```

metrics={r["strategy"]:r for r in summary["strategies"]}
m=metrics[selected];base=metrics["seasonal_q0"];r0=metrics["ridge_q0"]
audit={"q1":audit_trace(q1,initial=6000),"q2":{}},"accounting":{}}
for name,row in metrics.items():
    trace=pd.read_csv(out/f"{name}_intervals.csv.gz")
    audit["q2"][name]=audit_trace(trace,initial=manifest["evaluation_initial_soc"])
    cost=float(np.dot(trace.plan_kwh+5*trace.emergency_kwh,trace.price))
    if abs(cost-row["total_cost_yuan"])>0.02:raise AssertionError("Saved trace accounting differs")
    audit["accounting"][name]={recomputed_cost_yuan:cost,difference_yuan:cost-row["total_cost_yuan"]}
    if len(trace)!=334*144:raise AssertionError("Incomplete trace")
    dump_json(out/"saved_output_audit.json",audit)
    f[f.date.isin(DATES)].to_csv(out/"selected_dates_plot_data.csv",index=False)
    q1_sensitivity=[]
    for eta in (0.9,float(np.sqrt(0.9))):
        _,meta=optimize_day(q1.load_kwh.to_numpy(),q1.pv_kwh.to_numpy(),q1.price.to_numpy(),6000,(6000,"equal"),
            battery=replace(B,eta_c=eta,eta_d=eta))

    q1_sensitivity.append({"eta_charge":eta,"eta_discharge":eta,"round_trip":eta**2,"cost_yuan":meta["cost"]})
    pd.DataFrame(q1_sensitivity).to_csv(out/"q1_efficiency_sensitivity.csv",index=False)
    payload={"q1":{"plan":[[interval_label(i),float(g)] for i,g in enumerate(q1.plan_kwh)],
        "storage":r+["00:00",6000] if i==0 else ["24:00",6000] if i==1 else [None,None]
        for i,r in enumerate(blocks(q1))]},
        "q2":{"headers":["日期\时间"]+[interval_label(i) for i in range(144)]+"全天购电量","全天购电费",
        "plan":[],"storage":[],"emergency":[]}}
    for date,g in f.groupby("date",sort=True):
        s=chosen_daily.loc[date]

    payload["q2"]["plan"].append([date]+g.plan_kwh.astype(float).tolist()+[float(s.plan_kwh),float(s.total_cost_yuan)])
    for i,row in enumerate(blocks(g)):
        payload["q2"]["storage"].append([date if i==0 else None]+row+
            ([ "00:00",float(s.soc_start_kwh)] if i==0 else ["24:00",float(s.soc_end_kwh)] if i==1 else
            [None,None]))
        runs=emergency_runs(g)
        if not runs:runs=[["无紧急购电",0.0]]
        payload["q2"]["emergency"].extend([[date if i==0 else None]+row for i,row in enumerate(runs)])
    dump_json(out/"xlsx_payload.json",payload)
    pd.DataFrame({"slot":np.arange(144),"input_right_endpoint":[interval_label(i).split("-")[1] for i in
    range(144)],
        "normalized_output_interval":[interval_label(i) for i in
    range(144)]}).to_csv(out/"time_mapping.csv",index=False)
    figdir=Path("figures/q12");figdir.mkdir(parents=True,exist_ok=True)
    plt.rcParams.update({"font.sans-serif":["Microsoft YaHei","SimHei","DeJaVu
    Sans"],"axes.unicode_minus":False,
    "font.size":10,"axes.spines.top":False,"axes.spines.right":False,"svg.fonttype":"none"})
    figure_audit={}
    def save(fig,name):
        issues=audit_layout(fig)
        if any(level in ("FAIL","WARN") for level,_ in issues):raise RuntimeError(f"Figure layout: {issues}")
        figure_audit[name]=issues

    export_figure(fig,str(figdir/name),formats=["png","svg"],dpi=300,size_inches=tuple(fig.get_size_inches()),
        grayscale_preview=False,tight=False)
    svg=figdir/(name+".svg")
    svg.write_text("\n".join(line.rstrip() for line in svg.read_text(encoding="utf-
    8")).splitlines()+"\n",
        encoding="utf-8",newline="\n")
    plt.close(fig)
    x=(np.arange(144)+0.5)/6

    fig,axs=plt.subplots(2,1,figsize=(7.2,5.8),sharex=True,layout="constrained",gridspec_kw={"height_ratios":[2,
    1]})
    for key,label,color in [("load_kwh","负载",COLORS["load"]),("pv_kwh","光伏",COLORS["pv"]),("plan_kwh","计
    划购电",COLORS["grid"])]):
        axs[0].plot(x,q1[key]*6,label=label,color=color,lw=1.5,ls="--" if key=="plan_kwh" else "-")
        axs[0].set_ylim(0,1.25*q1[["load_kwh","pv_kwh","plan_kwh"]].to_numpy().max()*6)
        axs[0].set_ylabel("功率 / kW");axs[0].legend(ncol=3,loc="upper left");axs[0].set_title("(a) 单日供需与购电",loc="left",fontsize=11)
        axs[1].plot(np.arange(145)/6,np.r_[q1.soc_start_kwh.iloc[0],q1.soc_end_kwh],color=COLORS["soc"])
        axs[1].axhline(1200,color=COLORS["base"],ls=":");axs[1].axhline(10800,color=COLORS["base"],ls=":")
        axs[1].set_ylabel("储电量 / kWh");axs[1].set_xlabel("时刻 / h");axs[1].set_xticks(np.arange(0,25,4));axs[1].set_xlim(0,24)
        axs[1].set_title("(b) 储能状态",loc="left",fontsize=11);save(fig,"q1_dispatch")
    fig,axs=plt.subplots(2,1,figsize=(7.2,5.4),sharex=True,layout="constrained")
    g=f[f.date==DATES[0]]
    for ax,key,label in zip(axs,("load","pv"),("负载","光伏")):

```

```

ax.plot(x,g[key+"_kwh"]*6,color=COLORS["actual"],label="实际值",lw=1.4)
ax.plot(x,g["forecast_"+key+"_kwh"]*6,color=COLORS["forecast"],label="日前预测",ls="--",lw=1.5)
ax.set_ylabel(label+" /
kw");ax.set_ylim(0,1.25*g[[key+"_kwh"],"forecast_"+key+"_kwh"].to_numpy().max()*6)
axs[0].legend(ncol=2,loc="upper left");axs[0].set_title("2025-03-20 岭回归日前预测",loc="left",fontsize=11)
axs[1].set_xlabel("时刻 / h");axs[1].set_xticks(np.arange(0,25,4));axs[1].set_xlim(0,24);save(fig,"q2_forecast")
names=list(metrics);idx=np.arange(len(names))
fig,ax=plt.subplots(figsize=(7.2,4.5),layout="constrained")
normal=np.array([metrics[n]["normal_cost_yuan"] for n in names])/1e4
emerg=np.array([metrics[n]["emergency_cost_yuan"] for n in names])/1e4
ax.bar(idx,normal,color=COLORS["grid"],label="计划购电费",width=.55)
ax.bar(idx,emerg,bottom=normal,color=COLORS["emergency"],label="紧急购电费",width=.55)
ax.set_xticks(idx,[LABELS[n].replace(" ","\n") for n in names]);ax.set_ylabel("2-12月费用 / 万元")
ax.set_ylim(0,max(normal+emerg)*1.23);ax.legend(ncol=2,loc="upper right")
for i,total in enumerate(normal+emerg):ax.text(i,total+20,f"{total:,.1f}",ha="center",fontsize=10)
save(fig,"q2_cost_comparison")
fig,axs=plt.subplots(1,2,figsize=(7.2,3.5),layout="constrained")
for method,color in [{"seasonal",COLORS["base"]},("ridge",COLORS["load"])]):
    v=val[val.forecast==method]
    label="同期预测" if method=="seasonal" else "岭回归"
    axs[0].plot(range(4),v.cost_yuan/1e4,color=color,marker="o",ms=4,label=label)
    axs[1].plot(range(4),v.emergency_cost_yuan/1e4,color=color,marker="s",ms=4,label=label)
for ax in axs:
    ax.set_xticks(range(4),["无余量","70%","80%","90%"]);ax.set_xlabel("历史误差分位数");ax.set_ylim(bottom=0)
axs[0].set_ylabel("验证期总费用 / 万元");axs[1].set_ylabel("验证期紧急费用 / 万元")
axs[0].legend(loc="lower left",fontsize=9);save(fig,"q2_validation")
dump_json(out/"figure_layout_audit.json",figure_audit)
q1table=md_table(["时段","计划购电量 / kWh"],[[interval_label(h*6),float(q1.plan_kwh.iloc[h*6])] for h in (10,12,14,16,18,20)])
q1storage=md_table(["时段","充电量 / kWh","放电量 / kWh"],blocks(q1))
comparison=md_table(["策略","计划费 / 万元","紧急费 / 万元","总费用 / 万元","紧急购电 / kWh"],
[[LABELS[n],metrics[n]["normal_cost_yuan"]/1e4,metrics[n]["emergency_cost_yuan"]/1e4,metrics[n]["total_cost_yuan"]/1e4,metrics[n]["emergency_kwh"]] for n in names])
forecast_table=md_table(["预测器","负载 MAE / kW","负载 RMSE / kW","光伏 MAE / kW","光伏 RMSE / kW"],
[["同期预测",base["load_mae_kw"],base["load_rmse_kw"],base["pv_mae_kw"],base["pv_rmse_kw"]],
["岭回归",m["load_mae_kw"],m["load_rmse_kw"],m["pv_mae_kw"],m["pv_rmse_kw"]]])
date_summary=md_table(["日期","全天计划购电 / kWh","全天总购电费 / 元","紧急购电 / kWh","日初 SOC / kWh","日末 SOC / kWh"],
[[d,*[float(chosen_daily.loc[d,k]) for k in ["plan_kwh","total_cost_yuan","emergency_kwh","soc_start_kwh","soc_end_kwh"]] for d in DATES])
# Build text from computed evidence. Mathematics stays editable in Markdown.
text=rfr'# 考虑预测误差的微网日前购电与储能调度

```

第一问与第二问初稿

## 摘要

针对小区光伏发电、负载需求与分时电价共同影响的微网购电问题，建立统一的能量平衡与储能状态模型。在给定单日曲线时，以计划购电费最小为目标，采用混合整数线性规划约束充放电互斥，并要求日初日末储能电量相等。对于未来供需未知的情形，比较同期预测和岭回归，使用历史净负载误差的分位数构造非负余量，再通过实际运行仿真评估计划费用与五倍电价的紧急购电费用。

在充电、放电效率分别为 90%，输入功率视为十分钟区间平均值的假设下，第一问全天计划购电  $\{a['plan\_kwh']\} \sim .2f$  kWh，购电费  $\{a['cost']\} \sim .2f$  元，相对无储能基线降低  $\{a['saving\_percent']\} \sim .2f\%$ 。第二问利用一月份验证数据选择岭回归与 70% 分位数余量，随后在 2-12 月 334 天进行顺序回溯。该策略总费用为  $\{m['total\_cost\_yuan']/1e4\} \sim .2f$  万元，相对同期预测无余量基线下降  $\{100*(1-m['total\_cost\_yuan']/base['total\_cost\_yuan'])\} \sim .2f\%$ ；相对岭回归无余量策略下降  $\{100*(1-m['total\_cost\_yuan']/r0['total\_cost\_yuan'])\} \sim .2f\%$ 。结果表明，预测改进与适量风险余量均能降低费用，但更高余量虽可进一步减少紧急购电，也可能增加总成本。本稿仅回答前两问，其结论限于所列信息、时间、效率与运行假设。

关键词：微网调度；混合整数线性规划；岭回归；风险余量；时间顺序回溯

## 1 问题分析

题面及附件提供了微网的物理参数、供需曲线与结果模板 [1]。第一问的电价、负载和光伏预测曲线已给定，需决定各时段的购电与储能安排，属于确定性优化问题，不需要拟合预测模型。储能的经济作用来自跨时段转移电量，必须同时考虑功率上限、容量边界和能量损失。

第二问要求每天午夜制定整天计划。实际用电和发电变化后，普通购电仍按计划量付费，剩余供电缺口通过五倍电价紧急补购。因此，

少买与多买均存在代价。模型分为日前预测、计划优化和实时执行三个部分；历史真实值用于过去信息与事后评价，不能用于提前决定当天购电量。

## ## 2 数据处理与模型假设

附件 1 含 144 个十分钟时刻，附件 2 的负载与光伏各含  $365 \times 144 = 52,560$  个观测。实际读取未发现缺失值、负数或日期重复，保留全部样本，不对波峰进行无依据的删除或平滑。附件 1 部分时间单元格为字符串、部分为时间类型，读取后统一转换为分钟数，检查为 10、20、...、1440 的完整序列。

采用下列假设，并将它们与题面直接给定的参数区分：

1. 每个功率值表示以其时间标签为右端点的十分钟区间平均功率。电量为功率乘  $1/6$  小时。模板的区间标签存在疑似错位，本稿按内部区间起止时间汇总，导出副本同步采用规范标签，原始附件保持不变。
2. 充电和放电效率各取 0.9，往返效率为 0.81；功率上限定义在微网母线侧。另检验总往返效率 0.9 的解释。
3. 不设置题面未给出的售电收入，允许无法利用的剩余电量被弃用。计划富余电量仍付费。
4. 实时控制采用每十分钟内功率恒定且可即时平衡的近似。当前区间缺口可以被电池和紧急购电补足；这不是提前知道整天真实曲线。
5. 第二问实际 SOC 跨日连续，日前预测轨迹的日末 SOC 下限取 6000 kWh，作为避免耗空电池的策略设置，不强制实际日末回到此值。评价期初值由统一的一月份因果热身策略得到，不从二月起每天重置。

记区间负载与光伏电量为  $L_t, V_t$ ，计划购电为  $g_t$ ，紧急购电为  $e_t$ ，母线侧充电与放电为  $c_t, d_t$ ，未利用电量为  $w_t$ ，区间起点储电量为  $S_t$ 。上述电量单位均为 kWh，电价  $p_t$  单位为元/kWh。

## ## 3 第一问的确定性优化模型

### ### 3.1 目标与约束

最小化全天购电费用：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t.$$

第一问不设置紧急购电，能量平衡与储能状态满足：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad w_t \geq 0,$$

$$S_{t+1} = S_t + 0.9c_t - \frac{d_t}{0.9}, \quad 1200 \leq S_t \leq 10800.$$

令  $z_t \in \{0, 1\}$  表示充电状态，母线侧每区间最大充放电量为  $M = 5000/6$  kWh，规定

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad g_t \geq 0.$$

取  $S_1 = S_{145} = 6000$  kWh。该等式避免利用初始存量净放电制造虚假的节省。模型共有 865 个变量，其中 144 个为二元变量；利用 SciPy 的 MILP 接口调用 HiGHS 求解 [2]。

### ### 3.2 求解结果

全天计划购电量为  $\{a['plan\_kwh']\}$  kWh，购电费为  $\{a['cost']\}$  元。无储能基线按正净负载直接购电，费用为  $\{a['no\_storage\_cost']\}$  元，采用储能后降低  $\{a['saving\_percent']\}$ %。表 1 给出题目指定区间，表 2 给出四小时充放电汇总。图 1 展示供需、购电与储能轨迹。

表 1 第一问指定区间计划购电量

{q1table}

表 2 第一问储能运行汇总

{q1storage}

00:00 与 24:00 的储电量均为 6000.00 kWh。充电量记录进入电池前的母线电量，放电量记录电池返回母线的电量，二者不应直接按无损储能作差。

![图 1 第一问单日调度与储电量](../figures/q12/q1\_dispatch.png)

图 1 第一问单日供需、购电与 SOC 轨迹

### ### 3.3 结果检验与效率敏感性

整数模型的目标值与 LP 松弛下界之差为  $\{a['lp\_gap\_yuan']:.8f\}$  元，求解器报告相对间隙为  $\{a['mip\_gap']:.2g\}$ 。因此，在当前输入与约束口径下，第一问已获得数值精度内的全局最优解。最大能量平衡残差约  $\{a['audit']['max\_balance\_residual\_kwh']:.2e\}$  kWh，未出现同时充放电。

若“效率 90%”解释为总往返效率，将两侧效率均设为  $\sqrt{0.9}$ ，第一问费用变为  $\{q1\_sensitivity[1]['cost\_yuan']:.2f\}$  元。由此可见，效率定义会影响数值结论，后续整篇论文应保持同一口径，不混用两组结果。

## ## 4 第二问的预测与风险余量模型

### ### 4.1 时间顺序与预测器

以 1 月 22–31 日为验证区间，模型只用各预测时点之前已结束的日期训练。岭回归从积累足够历史后开始，每七天重新拟合一次，最多使用最近 60 天。样本不足时采用简单历史曲线。验证期选择完成后，将规则固定并应用到 2–12 月，后续已观测数据可以进入下一次更新，但不回写历史预测。

同期基线使用昨日和上周同日相同时段的均值。岭回归使用昨日、前日、上周同日的同槽电量，最近三日与七日同槽均值、七日标准差、过去日均值，三阶日内正余弦项以及星期指示变量，共 21 个特征。所有标准化参数均由当前训练段计算，预测当日的真实负载与光伏不进入特征。

对负载和光伏分别拟合带截距的岭回归 [3]：

$$\min_{\{\beta, b\}} \sum_i (y_i - b - x_i^T \beta)^2 + \lambda \|\beta\|_2^2.$$

截距不参与惩罚。验证比较  $\lambda \in \{1, 10, 100\}$ ，两种变量均选择  $\lambda = 1$ 。预测值截断为非负；如果某时槽过去七天的光伏均为零，则其预测置零。该规则只利用历史，但对日出日落变化的适应存在滞后。

### ### 4.2 风险余量

令净负载预测误差为

$$\varepsilon_{d,t} = (L_{d,t} - V_{d,t}) - (\hat{L}_{d,t} - \hat{V}_{d,t}).$$

对决策日之前最近 28 天、目标时槽前后三个十分钟槽内的历史误差取经验分位数，并截断为非负，得到余量  $r_{d,t}(q)$ 。不足 28 天时只使用已有历史，时槽池化不跨越日界。初稿比较无余量以及  $q = 0.7, 0.8, 0.9$ 。

将  $\hat{L} + r(q)$  与  $\hat{V}$  代入第一问的物理模型，以计划购电费最小求得日前计划，同时将预测轨迹日末储电量约束为至少 6000 kWh。这里的分位数余量是一种可解释的风险控制近似，\*\*不是精确求解全年随机最优控制，也不等于以概率  $q$  保证全日无缺口\*\*。

### ### 4.3 实时执行与费用

当天计划确定后不再倒改。每一十分钟区间，令计划购电与实际光伏减去实际负载的差额为  $u_t$ 。当  $u_t \geq 0$  时，在功率和容量允许范围内充电，剩余部分记为未用电量；当  $u_t < 0$  时，先按功率和 SOC 限制尽可能放电，剩余缺口记为紧急购电。所有区间均满足供电平衡，实际 SOC 连续传递至次日。

实际评价费用为

$$C_2 = \sum_t p_t g_t^{\text{plan}} + \sum_t p_t e_t.$$

普通购电按计划量收费，未利用的计划电量不退款。余量参数按验证期的该项总费用选择，而不是只按预测误差或紧急购电量选择。

## 5 第二问结果与分析

### 5.1 验证期的费用权衡

图 2 比较不同余量的验证费用。岭回归配合 70% 分位数余量的十天总费用为 {summary['selection']['cost\_yuan']:, .2f} 元，紧急费用为 {summary['selection']['emergency\_cost\_yuan']:, .2f} 元，在候选方案中最低。岭回归配合 90% 余量的验证期紧急费用为零，但总费用为 {float(val[(val.forecast=='ridge') & (val['quantile']==0.9)].cost\_yuan.iloc[0]):, .2f} 元，更高的计划支出抵消了避免紧急购电的收益。

![图 2 验证期余量与费用](../figures/q12/q2\_validation.png)

图 2 一月验证期风险余量与费用的关系

四种策略在评价期采用相同初始 SOC: {manifest['evaluation\_initial\_soc']:, .2f} kWh，来源于从 1 月 1 日 6000 kWh 出发的共同热身运行。不同策略在后续产生各自的 SOC 轨迹，没有按天重置。验证期与全年比较均报告期末存量，避免忽略存量差异。

### 5.2 样本外预测与调度效果

表 3 为 334 天共 48,096 个时段上的预测误差。相对同期基线，岭回归对负载的改善较明显，光伏改善较小。图 3 展示题目指定的 3 月 20 日预测结果，单日图用于说明误差形态，整体评价以全期指标为准。

表 3 第二问样本外预测误差

{forecast\_table}

![图 3 指定日期的负载与光伏预测](../figures/q12/q2\_forecast.png)

图 3 2025 年 3 月 20 日的日前预测与实际观测

表 4 与图 4 给出各策略的费用。岭回归无余量相对同期无余量已降低成本；在相同岭回归预测上增加 70% 余量，普通购电费增加，但紧急费用下降更多，最终总费用进一步降低 {100\*(1-m['total\_cost\_yuan']/r0['total\_cost\_yuan']):, .2f}%。这说明本题需要评价预测与调度的组合，而非只评价预测器。

表 4 第二问 2-12 月策略比较

{comparison}

![图 4 全期费用分解](../figures/q12/q2\_cost\_comparison.png)

图 4 不同预测与余量策略的全期费用分解

选定策略累计紧急购电 {m['emergency\_kwh']:, .2f} kWh，涉及 {m['emergency\_intervals']} 个十分钟区间。未利用电量为 {m['spill\_kwh']:, .2f} kWh，其中可能同时包含光伏富余和未吸收计划电量，当前账本不将其全部解释为弃光。该结果没有实现全年零紧急购电，也没有证明其为所有可行策略中的全局最优。

岭回归无余量与带余量策略在 12 月 31 日结束时均为 {m['final\_soc\_kwh']:, .2f} kWh，因此二者的主要费用差异不来自不同的期末电池存量。同期两组的期末 SOC 分别为 {base['final\_soc\_kwh']:, .2f} 与 {metrics['seasonal\_q0.7']['final\_soc\_kwh']:, .2f} kWh；跨预测器的费用比较仍需注意这种存量差异。

### 5.3 指定日期结果

表 5 为题目指定四日的汇总。“全天购电量”在本稿及导出表中指普通计划购电量；紧急购电另列。“全天购电费”包含普通与紧急两部分。后续各表给出指定十分钟槽与四小时充放电量。

表 5 第二问指定日期汇总

{date\_summary}

...

table\_number=6

```

for date in DATES:
    g=f[f.date==date]
    purchases=[[interval_label(h*6),float(g.plan_kwh.iloc[h*6])] for h in (10,12,14,16,18,20)]
    text+=f"### {date}\n\n 表 {table_number} 给出指定时段购电量, 表 {table_number+1} 为储能汇总, 表
{table_number+2} 为连续紧急购电区间.\n\n"
    text+=f"表 {table_number} {date} 指定区间计划购电\n\n"+md_table(["时段","计划购电 /
kWh"],purchases)+"\n\n"
    text+=f"表 {table_number+1} {date} 四小时充放电\n\n"+md_table(["时段","充电 / kWh","放电 /
kWh"],blocks(g))+"\n\n"
    text+=f"表 {table_number+2} {date} 紧急购电\n\n"+md_table(["时段","紧急购电 / kWh"],emergency_runs(g)
or [{"无",0.0}])+"\n\n"
    table_number+=3
    text+=rf'""## 6 模型评价与后续改进

```

第一问的混合整数模型可直接对应题面物理约束, 并通过 LP 下界和数值残差检验最优性与可行性。第二问将预测、风险余量和实际执行分开, 保持历史信息边界, 并以真实账本评价经济性, 具有可解释和易复现的优点。

当前局限包括: 一月份验证区间较短, 无法充分代表全部季节; 经验分位数没有给出联合概率保证; 贪心实时平衡不一定是最优电池控制; 名义日末 6000 kWh 的目标尚未系统调优; 效率、时间标签及实时观测近似会影响结果。较优的全年表现来自本次实际回测, 不应推广为任意未来数据下的保证。

在本机的一次运行中, 包括预测候选验证、共同热身、策略筛选和四组全年回测的计算约为 {manifest['total\_seconds']:.1f} 秒。每组全年共求解 334 个小规模日模型; 该时间不包括代码开发、排错、写作与图表导出, 也不代表第三、四问或更大场景模型的运行耗时。

后续可在不改变费用和物理口径的前提下, 比较更充分的季节验证、终端储能目标、分位数预测与场景优化, 并将第三问的日内预报更新接入同一调度框架。本稿不包含第三、四问结果。

## ## 参考文献

[1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略. 用户提供题面及附件 1、2、5.

[2] SciPy Developers. scipy.optimize.milp. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html> . 本文实际运行版本 1.17.1; 在线文档用于接口与方法说明。

[3] scikit-learn Developers. Ridge. [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.Ridge.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html) . 本文按带截距的 L2 正则化目标以 NumPy 实现, 未调用 scikit-learn 软件包。

```

'''
    # The abstract already identifies scope; keep project control notes outside manuscript.
    reportdir=Path("reports");reportdir.mkdir(exist_ok=True)
    (reportdir/"Q1_Q2 初稿.md").write_text(text,encoding="utf-8")
    dump_json(out/"delivery_manifest.json",{ "manuscript": "reports/Q1_Q2 初稿.md", "selected_strategy": selected,
        "figure_count": 4, "table_count": table_number-
1, "builder_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        "skill_repo": "XiaoMaColtAI/math-modeling-
skill", "skill_commit": "3527fad922660397834a6167fc2b4c29ee64ba17"})
    print("Manuscript and plotting evidence built.",flush=True)

```

```

if __name__=="__main__":main()

```

### src/v3\_payload.py

```
"""Exact result-cell payloads; preserve populated V2 workbook layouts."""
from pathlib import Path
import json
import pandas as pd
from src.q12 import dump_json
from src.deliver_q12 import blocks, emergency_runs

def main():
    out=Path('artifacts/v3');sel=json.loads((out/'selection.json').read_text());payload={}
    chosen={'3':f'q3_m{sel["q3"]["mask"]}', '4-2':f'q4_2_{sel["prices"]["2"]}', '4-3':f'q4_3_{sel["prices"]["3"]}' }
    for key,name in chosen.items():

f=pd.read_csv(out/f'{name}_intervals.csv.gz');plans=[];adjusted=[];storage=[];emergency=[];ledger=[];version_rows=[]
        for date,g in f.groupby('date',sort=False):

plans.append([date,*g.original_plan_kwh.to_list(),float(g.original_plan_kwh.sum()),float(g.total_cost_yuan.sum() if key=='4-2' else g.original_cost_yuan.sum())])

adjusted.append([date,*g.plan_kwh.to_list(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
        for i,b in enumerate(blocks(g)):
            storage.append([date if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
            for i,b in enumerate(emergency_runs(g) or [['无',0.]):emergency.append([date if i==0 else None,*b])
            ledger.append([date,*[float(g[c].sum()) for c in ['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
        vp=out/f'{name}_versions.csv.gz'
        if vp.exists():
            v=pd.read_csv(vp)
            for (date,ver),g in v.groupby(['date','version'],sort=True):
                values=[None]*144
                for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                version_rows.append([date,f'{int(ver)*6:02d}:00',*values])

payload[key]={ 'plan':plans, 'adjusted':adjusted, 'storage':storage, 'emergency':emergency, 'ledger':ledger, 'versions':version_rows}
        q1=pd.read_csv(out/'q1_intervals.csv');b=blocks(q1)
        # Official Q1 template is ordered 00:10,...,24:00. Its last slot is the
        # first slot of the repeated day. Keep every original template label.
        payload['1']={ 'plan':[[float(q1.plan_kwh.iloc[i%144])] for i in range(1,145)],
            'storage':[[*b[i][1:],float(q1.soc_start_kwh.iloc[0]) if i==0 else float(q1.soc_end_kwh.iloc[-1]) if i==1 else None] for i in range(6)]}

dump_json(out/'xlsx_payload.json',payload);dump_json(out/'delivery_selection.json',{'1':'q1_intervals.csv','2':'../v2/result2.xlsx',**chosen})

if __name__=='__main__':main()
```



## scripts/export\_v3\_xlsx.mjs

```
import fs from 'node:fs/promises';
import path from 'node:path';
import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
const root=process.argv[2];if(!root)throw Error('Original data root required');
const out='artifacts/v3',p=JSON.parse(await fs.readFile(`${out}/xlsx_payload.json`,`utf8`));
const dates=rows=>rows.map(r=>[r[0]?new Date(`${r[0]}T00:00:00Z`):null,...r.slice(1)]);
await fs.mkdir(`${out}/v3/xlsx',{recursive:true});const audit=[];
for(const key of ['1','3','4-2','4-3']){
  const source=key==='1'?path.join(root,'附件/附件 5/result1.xlsx'):`artifacts/v2/result${key}.xlsx`;
  const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(source));
  if(key==='1'){
    wb.worksheets.getItem('计划购电量').getRange('B2:B145').values=p[key].plan;
    const s=wb.worksheets.getItem('充放电量');s.getRange('B2:C7').values=p[key].storage.map(r=>r.slice(0,2));
    s.getRange('E2:E3').values=p[key].storage.slice(0,2).map(r=>r[2]);
  }else{
    for(const [sheet,field,last] of [['计划购电量','plan','EQ'],['调整购电量','adjusted','EQ'],['充放电量','storage','F'],['紧急购电量','emergency','C'],['费用分解','ledger','F'],['计划版本','versions','EP']]){
      if((sheet==='调整购电量' || sheet==='计划版本')&&key==='4-2')continue;
      const s=wb.worksheets.getItem(sheet),rows=p[key][field];
      // Only data cells are overwritten. Header text, interval labels in the
      // existing V2 filled templates, widths and formatting remain untouched.
      if(field==='storage' || field==='emergency' || field==='versions'){
        const previous=(await s.getUsedRange().values).length;
        if(previous>1)s.getRange(`A2:${last}${previous}`).clear({applyTo:'contents'});
      }
      s.getRange(`A2:${last}${rows.length+1}`).values=dates(rows);
    }
    const note=wb.worksheets.getItem('口径说明');
    if(key!=='4-2')note.getRange('B4:B5').values=[['午夜原计划与最终有效量结算：保留量 1 倍、取消量 0.5 倍、新增量 1.5 倍、紧急 5 倍；中途版本不重复收费'],['最终有效普通计划：每次已执行区间冻结，版本只用于追溯']];
  }
  wb.recalculate();await (await SpreadsheetFile.exportXlsx(wb)).save(`${out}/result${key}.xlsx`);
  const img=await wb.render({sheetName:'计划购电量',range:key==='1'?`A59:B65`:`A1:F5`,scale:1.5,format:'png'});
  await fs.writeFile(`${out}/v3/xlsx/${key}.png`,new Uint8Array(await img.arrayBuffer()));
  audit.push({key,source,template_headers_written:false});console.log(`Exported ${key}`);
}
await fs.copyFile('artifacts/v2/result2.xlsx',`${out}/result2.xlsx`);
await fs.writeFile(`${out}/xlsx_build.json`,JSON.stringify(audit,null,2));
```

## src/v5\_model.py

```
"""Past-only, lead-specific online load residual calibration (energy in kWh)."""
import numpy as np
```

```
def adaptive_load(data, midnight, window=28, shrink=.2):
    """Fit one nonnegative slope per release/lead-hour using completed days.

    x: last three observed hours' mean midnight-forecast error.
    y: error in the target hour, on historical days only.
    Ridge penalty = shrink * sum(x**2); no future-day normalization.
    Within each target hour the correction is constant. Midnight is unchanged.
    """
    truth=data['load']; count=len(midnight)
    out=np.repeat(midnight[:,None,:],4,axis=1); records=[]
    error=truth[:count]-midnight
    for day in range(14,count):
        left=max(7,day-window)
        for v in range(1,4):
            start=v*36
            x=error[left:day,start-18:start].mean(axis=1)
            current=float(error[day,start-18:start].mean())
            denom=float(x*x)*(1+shrink)
            for slot in range(start,144,6):
                y=error[left:day,slot:slot+6].mean(axis=1)
                beta=float(np.clip(x@y/denom,0,1.5)) if denom>1e-12 else 0.
                out[day,v,slot:slot+6]=np.maximum(0,midnight[day,slot:slot+6]+beta*current)
                records.append(dict(day=day,issue_hour=v*6,target_slot=slot,train_first_day=left,
                                    train_last_day=day-1,n_train=day-left,beta=beta,observed_residual_kwh=current))
    return out,records
```

## src/v5\_experiments.py

```

"""Frozen two-candidate audit experiment; no full-year strategy reselection."""
from pathlib import Path
import argparse,hashlib,time,json
import numpy as np
import pandas as pd
from src.q12 import forecasts,dump_json
from src.q34_data import read_extended
from src.v3_model import Policy,run,risk_buffers
from src.v5_model import adaptive_load

OUT=Path('artifacts/v5')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args()
    OUT.mkdir(parents=True,exist_ok=True);tick=time.perf_counter()
    design={'candidates':['baseline','adaptive'],'validation_days':[21,31],'evaluation_days':[31,365],
            'window_days':28,'shrink':.2,'slope_bounds':[0,1.5],'observation_slots':18,'target_block_slots':6,
            'quantile':.5,'mask':7,'terminal':6000,'price_model':'seasonal',
            'selection':'January 22-31 realized cost separately for Q3 and Q4 rolling; baseline wins ties',
            'scope':'Same-dataset development; previous annual results known; no untouched-test claim'}
    dump_json(OUT/'design_before_evaluation.json',design)
    data,audit=read_extended(a.data_root);pred,fits=forecasts(data,'ridge',(1.,1.))
    base=np.repeat(pred['load'][:,None,:],4,axis=1)
    adaptive,records=adaptive_load(data,pred['load']);loads={'baseline':base,'adaptive':adaptive}
    pd.DataFrame(records).to_csv(OUT/'load_calibration.csv.gz',index=False,float_format='%.17g')
    prices=np.load('artifacts/q34/price_forecasts.npz')['seasonal'];pol=Policy(quantile=.5,mask=7)
    # Complete validation and record selections BEFORE building evaluation buffers.
    validation=[];chosen={}
    for mode in [3,4]:
        for name,ll in loads.items():
            rr=risk_buffers(data,ll[:31],data['pv_issued'][:31],.5)
            r=run(data,ll,data['pv_issued'],rr,pol,start=21,stop=31,initial=6244.256891388887,
                name=f'q{mode}_{name}',prices=prices if mode==4 else None,retain=False)
    validation.append(dict(mode=mode,candidate=name,cost=r[3]['total_cost_yuan'],final_soc=r[3]['final_soc']))
    chosen[str(mode)]=min([x for x in validation if x['mode']==mode],key=lambda
x:(x['cost'],x['candidate']!= 'baseline'))['candidate']
    pd.DataFrame(validation).to_csv(OUT/'validation.csv',index=False)
    dump_json(OUT/'selection.json',chosen);print('January selection',chosen,validation,flush=True)
    summaries=[];daily_results={}
    for name,ll in loads.items():
        rr=risk_buffers(data,ll,data['pv_issued'],.5)
        for mode in [3,4]:
            label=f'q{mode}_{name}'
            r=run(data,ll,data['pv_issued'],rr,pol,start=31,stop=365,initial=9902.811287870369,
                name=label,prices=prices if mode==4 else None,retain=True)
            for obj,suffix in
zip([r[0],r[1],r[2],r[4]],['daily.csv','intervals.csv.gz','versions.csv.gz','solvers.csv.gz']):
                obj.to_csv(OUT/f'{label}_{suffix}',index=False,float_format='%.17g')
            dump_json(OUT/f'{label}_summary.json',r[3]);summaries.append(r[3]);daily_results[label]=r[0]
            print(label,round(r[3]['total_cost_yuan'],2),flush=True)
    dump_json(OUT/'summary.json',summaries)
    metrics=[]
    for name,ll in loads.items():
        for v in [1,2,3]:
            e=ll[31:,v,36*v:]-data['load'][31:,36*v:]
    metrics.append(dict(candidate=name,issue_hour=6*v,mae_kwh=float(abs(e).mean()),rmse_kwh=float(np.sqrt((e*e).
mean()))),n_pairs=e.size))
    pd.DataFrame(metrics).to_csv(OUT/'load_errors.csv',index=False)
    pairs=[];rng=np.random.default_rng(20260912)
    for mode in [3,4]:
b=daily_results[f'q{mode}_baseline'];c=daily_results[f'q{mode}_adaptive'];diff=b.total_cost_yuan.to_numpy()-
c.total_cost_yuan.to_numpy()
        for block in [7,14,28]:
            starts=rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/block))))
            indices=((starts[:,None]+np.arange(block))%len(diff)).reshape(2000,-1)[:,:len(diff)]
            lo,hi=np.quantile(diff[indices].sum(axis=1),[.025,.975])
    pairs.append(dict(mode=mode,block_days=block,saving_yuan=float(diff.sum()),ci_low=float(lo),ci_high=float(hi)
),
        final_soc_difference_kwh=float(c.soc_end_kwh.iloc[-1]-b.soc_end_kwh.iloc[-1]))
    pd.DataFrame(pairs).to_csv(OUT/'paired_comparison.csv',index=False)

```

```

    dump_json(OUT/'run_manifest.json',dict(seconds=time.perf_counter()-
tick,input_audit=audit,forecast_fits=fits,
    sources={f:hashlib.sha256(Path(f).read_bytes().replace(b'\r\n',b'\n')).hexdigest() for f in
        ['src/v5_model.py','src/v5_experiments.py','src/v3_model.py','src/q12.py','src/q34_data.py']}),
    price_cache_sha256=hashlib.sha256(Path('artifacts/q34/price_forecasts.npz').read_bytes()).hexdigest())
if __name__=='__main__':main()

```

## src/v5\_online.py

```

"""Monthly past-only validation of load correction, with continuous live SOC."""
from pathlib import Path
import argparse,json
import numpy as np
import pandas as pd
from src.q12 import dump_json,forecasts
from src.q34_data import read_extended
from src.v3_model import Policy,run,risk_buffers
from src.v5_model import adaptive_load

OUT=Path('artifacts/v5')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args()
    dump_json(OUT/'online_design_before_evaluation.json',{'update':'first day of every
month','validation_days':28,
    'candidates':['baseline','adaptive'],'risk':.5,'terminal':6000,'mask':7,
    'selection':'common initial SOC from live state at start of historical validation window; realized
historical costs; baseline wins ties',
    'february':'January selection, unchanged','price_model':'seasonal',
    'scope':'Predefined online selection extension; not an untouched external test'})

    data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.));prices=np.load('artifacts/q34/price
_forecasts.npz')['seasonal']

    loads={'baseline':np.repeat(pred['load'][:,None,:],4,axis=1),'adaptive':adaptive_load(data,pred['load'])[0]}
    buffers={k:risk_buffers(data,v,data['pv_issued'],.5) for k,v in
loads.items()};policy=Policy(quantile=.5,mask=7)
    choices=[];summaries=[]
    for mode in [3,4]:
        state=9902.811287870369;states={31:state};pieces=[[],[],[],[]]
        for month in range(2,13):
            indices=np.flatnonzero(data['dates'].month==month);start=int(indices[0]);stop=int(indices[-1]+1)
            name='baseline'
            if month>2:
                left=start-28;vals=[]
                for candidate in ['baseline','adaptive']:
                    r=run(data,loads[candidate],data['pv_issued'],buffers[candidate],policy,start=left,stop=start,
                        initial=states[left],name='validation',prices=prices if mode==4 else
None,retain=False)
                    vals.append(dict(mode=mode,month=month,candidate=candidate,validation_first_day=left,
                        validation_last_day=start-
1,initial_soc=states[left],final_soc=r[3]['final_soc'],cost=r[3]['total_cost_yuan']))
                    name=min(vals,key=lambda x:(x['cost'],x['candidate']!='baseline'))['candidate']
                    choices.extend([{*v,'selected':v['candidate']==name} for v in vals])

            r=run(data,loads[name],data['pv_issued'],buffers[name],policy,start=start,stop=stop,initial=state,
                name=f'q{mode}_online',prices=prices if mode==4 else None,retain=True)
            state=r[3]['final_soc']
            for day,val in zip(range(start,stop),r[0].soc_start_kwh):states[day]=float(val)
            for dest,obj in zip(pieces,[r[0],r[1],r[2],r[4]]):dest.append(obj)
            print('online',mode,month,name,round(r[3]['total_cost_yuan'],2),flush=True)
            combined=[pd.concat(p,ignore_index=True) for p in pieces];daily=combined[0]
            for obj,suffix in zip(combined,['daily.csv','intervals.csv.gz','versions.csv.gz','solvers.csv.gz']):
                obj.to_csv(OUT/f'q{mode}_online_{suffix}',index=False,float_format='%17g')
            from src.q12 import audit_trace
            summary=dict(strategy=f'q{mode}_online',days=334,initial_soc=9902.811287870369,final_soc=state,

total_cost_yuan=float(daily.total_cost_yuan.sum()),emergency_kwh=float(daily.emergency_kwh.sum()),

emergency_cost_yuan=float(daily.emergency_cost_yuan.sum()),physical=audit_trace(combined[1],initial=9902.811
287870369))
            dump_json(OUT/f'q{mode}_online_summary.json',summary);summaries.append(summary)
            pd.DataFrame(choices).to_csv(OUT/'online_selection.csv',index=False,float_format='%17g')
            dump_json(OUT/'online_summary.json',summaries)
if __name__=='__main__':main()

```

## src/v5\_audit.py

```

"""Independent workbook, cumulative-inventory LP, dual certificate and trace audit."""
from pathlib import Path
import argparse,hashlib,json
import numpy as np
import pandas as pd
import openpyxl
from scipy.optimize import linprog
from src.q12 import B,forecasts,optimize_day,dump_json,audit_trace
from src.q34_data import read_extended
from src.v3_model import solve
from src.v5_model import adaptive_load

OUT=Path('artifacts/v5')
def independent_lp(load,pv,price,initial,terminal=6000,old=None,equal=False):
    """Eliminate state variables; revision uses a two-line convex epigraph."""
    n=len(price);rev=old is not None;k=(5 if rev else
4)*n;I=np.eye(n);Z=np.zeros((n,n));T=np.tril(np.ones((n,n)))
    eq=np.hstack([I,-I,I,-I]+([Z] if rev else []));rhs=np.asarray(load)-pv
    soc=np.hstack([Z,B.eta_c*T,-T/B.eta_d,Z]+([Z] if rev else []))
    ub=[soc,-soc];br=[np.full(n,B.maximum-initial),np.full(n,initial-B.minimum)]
    if equal:eq=np.vstack([eq,soc[-1:]];rhs=np.r_[rhs,terminal-initial]
    else:ub.append(-soc[-1:]);br.append(np.array([initial-terminal]))
    c=np.zeros(k);bounds=[(0,None)]*k;bounds[n:3*n]=[(0,B.limit)]*(2*n)
    if rev:
        c[4*n:]=1;bounds[4*n:]=[(None,None)]*n
        for slope in [.5,1.5]:
            row=np.zeros((n,k));row[:,n]=np.diag(slope*price);row[:,4*n:]=-I
            ub.append(row);br.append(slope*price*old)
        else:c[:n]=price
        A=np.vstack(ub);b=np.concatenate(br)
        fit=linprog(c,A_ub=A,b_ub=b,A_eq=eq,b_eq=rhs,bounds=bounds,method='highs')
        assert fit.success,fit.message
        stationarity=c-eq.T@fit.eqlin.marginals-A.T@fit.ineqlin.marginals-fit.lower.marginals-
fit.upper.marginals
        dual=float(rhs@fit.eqlin.marginals+b@fit.ineqlin.marginals)
        comp=[np.max(abs(fit.ineqlin.residual*fit.ineqlin.marginals))]
        for label,idx in [('lower',0),('upper',1)]:
            valid=np.array([v[idx] is not None for v in bounds]);v=np.array([x[idx] or 0 for x in
bounds]);m=getattr(fit,label).marginals
            dual+=float(v[valid]@m[valid]);comp.append(float(np.max(abs((fit.x[valid]-v[valid])*m[valid]))) if
valid.any() else 0)
            sign_violation=float(max(0,fit.ineqlin.marginals.max(),-
fit.lower.marginals.min(),fit.upper.marginals.max()))
            cert=dict(objective=float(fit.fun),dual_objective=dual,gap=abs(float(fit.fun)-
dual),stationarity=float(abs(stationarity).max()),dual_sign_violation=sign_violation,
            complementarity=max(comp),equality_residual=float(abs(eq@fit.x-
rhs).max()),inequality_violation=float(max(0,np.max(A@fit.x-b))))
            assert max(cert[x] for x in
['gap','stationarity','complementarity','equality_residual','inequality_violation','dual_sign_violation'])<1
e-5
        return cert

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-
root',required=True);a=ap.parse_args();root=Path(a.data_root)
    OUT.mkdir(parents=True,exist_ok=True);data,source=read_extended(root);books=[]
    for index in range(1,5):
        path=root/'附件'/f'附件{index}.xlsx';wb=openpyxl.load_workbook(path,data_only=False)
        sheets=[]
        for ws in wb:
            cells=list(ws.values);formula=sum(isinstance(v,str) and v.startswith('=') for row in cells for v
in row)
            sheets.append(dict(name=ws.title,rows=ws.max_row,columns=ws.max_column,state=ws.sheet_state,
            formulas=formula,hidden_rows=sum(bool(v.hidden) for v in
ws.row_dimensions.values()),hidden_columns=sum(bool(v.hidden) for v in ws.column_dimensions.values()),
            merges=[str(v) for v in ws.merged_cells.ranges]))
    books.append(dict(file=path.name,sha256=hashlib.sha256(path.read_bytes()).hexdigest(),sheets=sheets));wb.clo
se()
    # Independent openpyxl extraction compared against the production pandas parser.
    wb=openpyxl.load_workbook(root/'附件/附件 2.xlsx',data_only=True,read_only=True)
    for name,ws in zip(['load','pv'],wb):
        vals=np.array([r[1:] for r in list(ws.values)[1:]],float)/6
        np.testing.assert_allclose(vals,data[name],atol=3e-13,rtol=0)
    wb.close()

```

```

pred,_=forecasts(data,'ridge',(1.,1.));ll,records=adaptive_load(data,pred['load']);changed=dict(data)
altered=data['load'].copy();altered[20,72:]+=12345;altered[21:]+=98765;changed['load']=altered
ll2,_=adaptive_load(changed,pred['load'])
np.testing.assert_array_equal(ll[:20],ll2[:20]);np.testing.assert_array_equal(ll[20,:3],ll2[20,:3])
certificates=[];rng=np.random.default_rng(20260912)
# Randomized and adverse cases, including low prices and redundant PV.
for case in range(36):
    n=[6,24,144][case%3];L=rng.uniform(0,800,n);V=rng.uniform(0,1000,n);p=rng.uniform(.001,2,n)
    s0=6000.;old=rng.uniform(0,900,n) if case%2 else None
    ref=independent_lp(L,V,p,s0,old=old);f,m=solve(L,V,p,s0,old=old)
    assert abs(ref['objective']-m['objective'])<1e-5
    ref.update(case=f'random_{case}',difference=abs(ref['objective']-
m['objective']));certificates.append(ref)
    if old is None:
        _,mm=optimize_day(L,V,p,s0,integer=True);assert abs(mm['cost']-m['objective'])<1e-5
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    cert=independent_lp(L,V,p,6000,equal=True);f,m=solve(L,V,p,6000,equal=True)
    cert.update(case='q1',difference=abs(cert['objective']-m['objective']));certificates.append(cert)
# Actual published revision instances, not only synthetic examples.
for name in ['q3_baseline','q4_baseline','q3_adaptive','q4_adaptive','q3_online','q4_online']:
    versions=pd.read_csv(OUT/f'{name}_versions.csv.gz');trace=pd.read_csv(OUT/f'{name}_intervals.csv.gz')
    for date in ['2025-03-20','2025-06-21','2025-09-23','2025-12-21']:
        for ver in [1,2,3]:
            v=versions[(versions.date==date)&(versions.version==ver)];g=trace[(trace.date==date)&(trace.slot>=ver*36)]

            L=(v.forecast_load_kwh+v.reserve_kwh).to_numpy();V=v.forecast_pv_kwh.to_numpy();p=v.forecast_price.to_numpy(
);old=g.original_plan_kwh.to_numpy();state=float(v.decision_soc_kwh.iloc[0])
            cert=independent_lp(L,V,p,state,old=old);_,m=solve(L,V,p,state,old=old)
            cert.update(case=f'{name}_{date}_{ver}',difference=abs(cert['objective']-
m['objective']));assert cert['difference']<1e-5;certificates.append(cert)
            pd.DataFrame(certificates).to_csv(OUT/'solver_certificates.csv',index=False)
            traces=[]
            for name in ['q3_baseline','q4_baseline','q3_adaptive','q4_adaptive','q3_online','q4_online']:
                f=pd.read_csv(OUT/f'{name}_intervals.csv.gz');d=pd.read_csv(OUT/f'{name}_daily.csv');v=pd.read_csv(OUT/f'{na
me}_versions.csv.gz')
                audit=audit_trace(f,initial=9902.811287870369)
                for col,key in
[('load_kwh','load'),('pv_kwh','pv')]:np.testing.assert_allclose(f[col],data[key][31:].reshape(-1),atol=1e-
10,rtol=0)
                expected_price=np.tile(data['price'],334) if name.startswith('q3') else
data['actual_price'][31:].reshape(-1)
                np.testing.assert_allclose(f.price,expected_price,atol=1e-12,rtol=0)

            independent_cost=f.price*(np.minimum(f.original_plan_kwh,f.plan_kwh)+.5*np.maximum(f.original_plan_kwh-
f.plan_kwh,0)+1.5*np.maximum(f.plan_kwh-f.original_plan_kwh,0)+5*f.emergency_kwh)
            np.testing.assert_allclose(independent_cost,f.total_cost_yuan,atol=1e-7,rtol=0)
            np.testing.assert_allclose(independent_cost.groupby(f.date).sum(),d.total_cost_yuan,atol=1e-6,rtol=0)
            for date,g in f.groupby('date',sort=False):
                plan=g.original_plan_kwh.to_numpy().copy()
                for ver,vg in v[v.date==date].groupby('version',sort=True):
                    slots=vg.slot.to_numpy(int);np.testing.assert_array_equal(slots,np.arange(int(ver)*36,144));np.testing.asser
t_allclose(vg.old_kwh,plan[slots],atol=1e-7)
                    np.testing.assert_allclose(vg.decision_soc_kwh,g.soc_start_kwh.iloc[int(ver)*36],atol=1e-
7);plan[slots]=vg.new_kwh
                    np.testing.assert_allclose(plan,g.plan_kwh,atol=1e-7)

            traces.append(dict(strategy=name,intervals=len(f),versions=len(v),cost=float(independent_cost.sum()),**audit
))
            for new,old in [('q3_baseline','q3_m7'),('q4_baseline','q4_3_seasonal')]:
                f=pd.read_csv(OUT/f'{new}_daily.csv');g=pd.read_csv(f'artifacts/v3/{old}_daily.csv')
                np.testing.assert_allclose(f.total_cost_yuan,g.total_cost_yuan,atol=1e-6,rtol=0)
                q2=pd.read_csv(f'artifacts/q12/ridge_q0.7_intervals.csv.gz')
                q2_physical=audit_trace(q2,initial=9902.811287870369)
                for col,key in
[('load_kwh','load'),('pv_kwh','pv')]:np.testing.assert_allclose(q2[col],data[key][31:].reshape(-1),atol=1e-
10,rtol=0)
                np.testing.assert_allclose(q2.price,np.tile(data['price'],334),atol=1e-12,rtol=0)
                q2_cost=float((q2.price*(q2.plan_kwh+5*q2.emergency_kwh)).sum());assert abs(q2_cost-
14132612.15924625)<1e-5
                q1=pd.read_csv(f'artifacts/v3/q1_intervals.csv');q1_physical=audit_trace(q1,initial=6000)
                for col,key in
[('load_kwh','q1_load'),('pv_kwh','q1_pv'),('price','price')]:np.testing.assert_allclose(q1[col],np.roll(dat
a[key],1),atol=1e-10,rtol=0)
                assert abs(float(q1.price@q1.plan_kwh)-certificates[36]['objective'])<1e-5
                choices=pd.read_csv(OUT/'online_selection.csv')
                for (mode,month),g in choices.groupby(['mode','month']):

```

```

        start=int(np.flatnonzero(data['dates'].month==month)[0]);assert (g.validation_last_day==start-
1).all() and (g.validation_first_day==start-28).all()
        live=pd.read_csv(OUT/f'q{mode}_online_daily.csv');assert abs(g.initial_soc.iloc[0]-
live.soc_start_kwh.iloc[start-28-31])<1e-7
        assert g[g.selected].cost.iloc[0]==g.cost.min()
        result=dict(status='PASS',workbooks=books,input_audit=source,independent_attachment2_values=105120,
        future_perturbation_test='PASS: future loads do not change earlier
releases',solver_cases=len(certificates),
        q1_retained_trace=q1_physical,q2_retained_trace=q2_physical,q2_independent_cost=q2_cost,
        monthly_selection_windows=20,milp_comparisons=18,max_solver_difference=max(x['difference'] for x in
certificates),
        max_duality_gap=max(x['gap'] for x in certificates),traces=traces,
        limitations=['Structural audit does not prove that sensors are error-free','Reference Modex is a
supplied comparison manuscript, not independently certified results'])
        dump_json(OUT/'verification.json',result);print(json.dumps({k:result[k] for k in
['status','solver_cases','milp_comparisons','max_solver_difference','max_duality_gap']}),ensure_ascii=False))
if __name__=='__main__':main()

```



## src/v5\_summary.py

```
from pathlib import Path
import json
import numpy as np
import pandas as pd
from src.q12 import dump_json

def main():
    out=Path('artifacts/v5');rows=[];months=[];rng=np.random.default_rng(20260912)
    for mode in [3,4]:
        b=pd.read_csv(out/f'q{mode}_baseline_daily.csv');c=pd.read_csv(out/f'q{mode}_online_daily.csv')
        diff=b.total_cost_yuan.to_numpy()-c.total_cost_yuan.to_numpy()

    row=dict(mode=mode,baseline_cost=float(b.total_cost_yuan.sum()),v5_cost=float(c.total_cost_yuan.sum()),savin
g=float(diff.sum()),

    saving_percent=float(diff.sum()/b.total_cost_yuan.sum()*100),baseline_emergency=float(b.emergency_kwh.sum()),
        v5_emergency=float(c.emergency_kwh.sum()),initial_soc_difference=float(c.soc_start_kwh.iloc[0]-
b.soc_start_kwh.iloc[0]),
        final_soc_difference=float(c.soc_end_kwh.iloc[-1]-b.soc_end_kwh.iloc[-1]),bootstrap=[]
        for block in [7,14,28]:

    start=rng.integers(0,334,(2000,int(np.ceil(334/block)))));ix=((start[:, :,None]+np.arange(block))%334).reshape
(2000,-1)[:,:334]

    lo,hi=np.quantile(diff[ix].sum(axis=1),[.025,.975]);row['bootstrap'].append(dict(block_days=block,low=float(
lo),high=float(hi)))
        rows.append(row)
        for month in range(2,13):
            mask=pd.to_datetime(b.date).dt.month==month

    months.append(dict(mode=mode,month=month,baseline=float(b.loc[mask,'total_cost_yuan'].sum()),online=float(c.
loc[mask,'total_cost_yuan'].sum()),saving=float(diff[mask].sum()))

    dump_json(out/'improvement.json',rows);pd.DataFrame(months).to_csv(out/'monthly_comparison.csv',index=False)
    from src.q12 import B
    pmax=json.loads((out/'verification.json').read_text(encoding='utf-
8'))['input_audit']['checks']['actual_price']['max']
    selections=pd.read_csv(out/'online_selection.csv');sensitivity=[]
    for (mode,month),g in selections.groupby(['mode','month']):
        a=g[g.selected].iloc[0];b=g[~g.selected].iloc[0]
        gap=float(b.cost-a.cost);value=float(5*pmax/B.eta_c*abs(b.final_soc-a.final_soc))

    sensitivity.append(dict(mode=int(mode),month=int(month),cost_gap=gap,max_inventory_adjustment=value,unchange
d=gap>value))
    pd.DataFrame(sensitivity).to_csv(out/'selection_inventory_sensitivity.csv',index=False)
    print(json.dumps(rows,ensure_ascii=False,indent=2))
if __name__=='__main__':main()
```

## src/v5\_delivery.py

```
"""Write V5 data cells into V3 layouts and read every written cell back."""
from pathlib import Path
from datetime import datetime
import json,shutil,math,hashlib
from copy import copy
import pandas as pd
import openpyxl
from src.q12 import dump_json
from src.deliver_q12 import blocks,emergency_runs

OUT=Path('artifacts/v5')
SOURCES={'1':'artifacts/v3/q1_intervals.csv','2':'artifacts/q12/ridge_q0.7_intervals.csv.gz',
'3':str(OUT/'q3_online_intervals.csv.gz'),'4-2':'artifacts/v3/q4_2_ridge_intervals.csv.gz','4-3':str(OUT/'q4_online_intervals.csv.gz')}

def main():
    audit=[]
    for key in ['1','2','4-2']:shutil.copy2(f'artifacts/v3/result{key}.xlsx',OUT/f'result{key}.xlsx')
    for key,label in [('3','q3_online'),('4-3','q4_online')]:
        f=pd.read_csv(SOURCES[key]);v=pd.read_csv(OUT/f'{label}_versions.csv.gz');payload={k:[] for k in ['计划购电量','调整购电量','充放电量','紧急购电量','费用分解','计划版本']}
        for date,g in f.groupby('date',sort=False):
            dt=datetime.strptime(date,'%Y-%m-%d')
            payload['计划购电量'].append([dt,*g.original_plan_kwh.tolist(),float(g.original_plan_kwh.sum()),float(g.original_cost_yuan.sum())])
            payload['调整购电量'].append([dt,*g.plan_kwh.tolist(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
            for i,b in enumerate(blocks(g)):
                payload['充放电量'].append([dt if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
            for i,b in enumerate(emergency_runs(g) or [['无',0]]):payload['紧急购电量'].append([dt if i==0 else None,*b])
            payload['费用分解'].append([dt,*[float(g[c].sum()) for c in ['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
            for (date,ver),g in v.groupby(['date','version'],sort=True):
                values=[None]*144
                for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                payload['计划版本'].append([datetime.strptime(date,'%Y-%m-%d'),f'{int(ver)*6:02d}:00',*values])
            wb=openpyxl.load_workbook(f'artifacts/v3/result{key}.xlsx');assert not any(c.data_type=='f' for ws in wb for row in ws for c in row),'Formula-bearing template needs recalculation'
            headers={ws.title:tuple(c.value for c in ws[1]) for ws in wb};sheets=wb.sheetnames
            for name,rows in payload.items():
                ws=wb[name];end=ws.max_row;columns=len(rows[0])
                for row in ws.iter_rows(min_row=2,max_row=max(end,len(rows)+1),max_col=columns):
                    for cell in row:cell.value=None
                for i,row in enumerate(rows,2):
                    for j,value in enumerate(row,1):
                        cell=ws.cell(i,j);cell.value=value
                        if i>end:cell._style=copy(ws.cell(2,j)._style)
            note=wb['口径说明'];note['B2']='V5: 每月以前 28 天共同初始 SOC 验证, 选择基础或自适应负载预测; 实际 SOC 连续跨月。'
            dest=OUT/f'result{key}.xlsx';wb.save(dest);wb.close()
            checked=openpyxl.load_workbook(dest,read_only=True,data_only=True);count=0
            assert checked.sheetnames==sheets
            for name in sheets:assert tuple(next(checked[name].values))==headers[name]
            for name,rows in payload.items():
                for actual,expected in zip(checked[name].iter_rows(min_row=2,max_row=len(rows)+1,max_col=len(rows[0]),values_only=True),rows):
                    for a,e in zip(actual,expected):
                        count+=1
                        if isinstance(e,(int,float)):assert isinstance(a,(int,float)) and math.isclose(a,e,abs_tol=1e-7,rel_tol=1e-11),(name,a,e)
                        else:assert a==e,(name,a,e)
```

```

        for row in checked[name].iter_rows(min_row=len(rows)+2,values_only=True):assert all(v is None
for v in row)

checked.close();audit.append(dict(key=key,cells_compared=count,headers_preserved=True,formula_cells=0,sha256
=hashlib.sha256(dest.read_bytes()).hexdigest()))
    dump_json(OUT/'delivery_selection.json',{ 'sources':SOURCES,'main':'Q3 and Q4 rolling use causal monthly
selection; Q1/Q2/Q4 ahead retained','copied_byte_identical':['1','2','4-2']})

dump_json(OUT/'xlsx_audit.json',dict(status='PASS',updated=audit,copied={k:hashlib.sha256((OUT/f'result{k}.x
lsx').read_bytes()).hexdigest() for k in ['1','2','4-2']}));print(audit)
if __name__=='__main__':main()

```

## src/v5\_price\_audit.py

```
"""Replay stored price checkpoints and verify release/target cutoffs."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import torch
from src.q34_data import read_extended
from src.q4_forecast import features_at, PriceGRU, ridge_features
from src.q12 import dump_json, ridge_fit

def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()

    data,_=read_extended(a.data_root); flat=data['actual_price'].ravel(); cache=np.load('artifacts/q34/price_forecasts.npz')
    logs=json.loads(Path('artifacts/q34/price_fit_log.json').read_text(encoding='utf-8')); fits=sorted({r['fit_day'] for r in logs}); records=[]
    for day in range(7,365):
        for v in range(4):
            origin=day*144+36*v; targets=np.arange(origin,origin+144)
            assert (targets-144<origin).all() and (targets-1008<origin).all()
            np.testing.assert_array_equal(features_at(flat,origin)[:,:2].mean(-1),cache['seasonal'][day,v])
            origins=np.arange(1008,len(flat)-143,36)
            for fi,day in enumerate(fits):
                end=fits[fi+1] if fi+1<len(fits) else 365
                xx=np.stack([features_at(flat,d*144+36*v) for d in range(day,end) for v in range(4)])
                # Refit the selected linear predictor from raw history, without using cache labels.
                oo=origins[(origins+144<=day*144)&(origins>=max(1008,day*144-60*144))]
                trainx=np.stack([features_at(flat,o) for o in oo]); trainy=np.stack([flat[o:o+144] for o in oo]).astype('float32')
                mean,scale,coef,intercept=ridge_fit(ridge_features(trainx).reshape(-1,12),trainy.ravel(),1.)
                pred=np.maximum(((ridge_features(xx)-mean)/scale)@coef+intercept,.001).reshape(end-day,4,144)
                np.testing.assert_allclose(pred,cache['ridge'][day:end],atol=1e-10,rtol=0)
                for seed in [17,42,2026]:
                    log=next(x for x in logs if x['fit_day']==day and x['seed']==seed)
                    assert log['last_target_exclusive']<=day*144 and log['actual_rng_seed']==day+seed
                    path=Path(f'artifacts/q34/models/gru{seed}_fit{day:03d}.pt')
                    saved=torch.load(path,map_location='cpu',weights_only=True); assert
                    saved['training_last_target_exclusive']==day*144
                    model=PriceGRU(); model.load_state_dict(saved['state_dict']); model.eval()
                    z=xx.copy(); z[:, :, :2]=(z[:, :, :2]-saved['mean'])/saved['scale']
                    with
                    torch.no_grad(): values=np.maximum(model(torch.tensor(z)).numpy()*saved['scale']+saved['mean'],.001).reshape(
                    end-day,4,144)
                    diff=float(abs(values-cache[f'gru{seed}'][day:end]).max()); assert diff<1e-6

    records.append(dict(fit_day=day,base_seed=seed,actual_seed=day+seed,max_replay_error=diff,checkpoint_sha256=
    hashlib.sha256(path.read_bytes()).hexdigest()))
    np.testing.assert_allclose(cache['gru_mean'][14:],np.mean([cache[f'gru{s}'][14:] for s in
    [17,42,2026]],axis=0),atol=1e-12,rtol=0)
    result=dict(status='PASS',seasonal_release_vectors=358*4,ridge_refits=13,gru_checkpoint_replays=39,
    seed_reporting_correction='17,42,2026 are base seeds; actual seed is base seed plus fit-day index',
    max_replay_error=max(x['max_replay_error'] for x in records),records=records,
    cache_sha256=hashlib.sha256(Path('artifacts/q34/price_forecasts.npz').read_bytes()).hexdigest())
    dump_json('artifacts/v5/price_replay_audit.json',result); print({k:v for k,v in result.items() if
    k!='records'})
    if __name__=='__main__':main()
```

## src/v5\_retained\_audit.py

```
"""Rerun retained Q1, Q2 and Q4-ahead decisions from raw inputs."""
from pathlib import Path
import argparse
import numpy as np
import pandas as pd
from src.q12 import forecasts, backtest, optimize_day, dump_json
from src.q34_data import read_extended
from src.v3_model import run, Policy, risk_buffers

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);a=ap.parse_args();out=Path('artifacts/v5')
    data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.))
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    _,m=optimize_day(L,V,p,6000,(6000,'equal'));assert abs(m['cost']-35126.84858928963)<1e-7
    daily,_=state=backtest(data,pred,31,365,9902.811287870369,.7,'q2_reproduced',retain=False)
    old=pd.read_csv('artifacts/q12/ridge_q0.7_intervals.csv.gz')
    old_cost=(old.price*(old.plan_kwh+5*old.emergency_kwh)).groupby(old.date).sum().to_numpy()
    diff2=float(abs(daily.total_cost_yuan.to_numpy()-old_cost).max());assert diff2<1e-6

    ll=np.repeat(pred['load'][:,None,:],4,axis=1);pv=np.repeat(pred['pv'][:,None,:],4,axis=1);rr=risk_buffers(data,ll,pv,.7)

    r=run(data,ll,pv,rr,Policy(),start=31,stop=365,initial=9902.811287870369,name='q4_ahead_reproduced',prices=np.load('artifacts/q34/price_forecasts.npz')['ridge'],retain=False)
    old4=pd.read_csv('artifacts/v3/q4_2_ridge_daily.csv');diff4=float(abs(r[0].total_cost_yuan.to_numpy()-old4.total_cost_yuan.to_numpy()).max());assert diff4<1e-6

    result=dict(status='PASS',q1_milp_cost=m['cost'],q2_cost=float(daily.total_cost_yuan.sum()),q2_max_daily_difference=diff2,

    q4_ahead_cost=r[3]['total_cost_yuan'],q4_max_daily_difference=diff4,recomputed_days_per_stochastic_policy=334)
    dump_json(out/'retained_reproduction.json',result);print(result)
    if __name__=='__main__':main()
```

## src/v5b\_value.py

```

"""Continuous convex piecewise-linear storage values; no SOC discretization."""
from dataclasses import dataclass
import numpy as np
import pandas as pd
from src.q12 import B, audit_trace

@dataclass
class Value:
    x: np.ndarray
    y: np.ndarray
    def at(self, s): return np.interp(s, self.x, self.y)
    @property
    def slopes(self): return np.diff(self.y)/np.diff(self.x)

def restrict(f, lo=B.minimum, hi=B.maximum):
    lo = max(lo, float(f.x[0])); hi = min(hi, float(f.x[-1]))
    assert lo <= hi + 1e-7
    x = np.unique(np.r_[lo, f.x[(f.x > lo + 1e-8) & (f.x < hi - 1e-8)], hi])
    y = f.at(x)
    if len(x) > 2:
        slopes = np.diff(y)/np.diff(x); keep = np.r_[True, abs(np.diff(slopes)) > 1e-8, True]
        x, y = x[keep], y[keep]
    return Value(x, y)

def convolve(f, h):
    """Convex infimal convolution by merging segment slopes and lengths."""
    lengths = np.r_[np.diff(f.x), np.diff(h.x)]; slopes = np.r_[f.slopes, h.slopes]
    order = np.argsort(slopes, kind='stable'); lengths = lengths[order]; slopes = slopes[order]
    x = np.r_[f.x[0] + h.x[0], f.x[0] + h.x[0] + np.cumsum(lengths)]
    y = np.r_[f.y[0] + h.y[0], f.y[0] + h.y[0] + np.cumsum(lengths * slopes)]
    return restrict(Value(x, y))

def purchase_stage(net, p):
    # v = s_before - s_after; negative v charges, positive v discharges.
    lo = -B.eta_c * B.limit; hi = B.limit / B.eta_d
    zero = net / B.eta_d if net >= 0 else net * B.eta_c
    x = np.unique(np.r_[lo, hi, np.clip(0, lo, hi), np.clip(zero, lo, hi)])
    y = p * np.maximum(0, net + np.where(x >= 0, -B.eta_d * x, -x / B.eta_c))
    return Value(x, y)

def deterministic(load, pv, price, initial=6000, terminal=6000):
    n = len(price); values = [None] * (n + 1); values[n] = Value(np.array([float(terminal)]), np.array([0.]))
    stages = [purchase_stage(l - v, p) for l, v, p in zip(load, pv, price)]
    for t in range(n - 1, -1, -1): values[t] = convolve(values[t + 1], stages[t])
    s = float(initial); records = []
    for t, h in enumerate(stages):
        future = values[t + 1]; lo = max(future.x[0], s - h.x[-1]); hi = min(future.x[-1], s - h.x[0])
        choices = np.unique(np.clip(np.r_[lo, hi, future.x, s - h.x], lo, hi))
        costs = future.at(choices) + h.at(s - choices); end = float(choices[np.argmin(costs)])
        c = max(0, (end - s) / B.eta_c); d = max(0, (s - end) * B.eta_d); g = max(0, load[t] - pv[t] + c - d); w = g + pv[t] + d - load[t] - c
        records.append([g, c, d, max(0, w), s, end, load[t], pv[t], 0., price[t]]); s = end
    f = pd.DataFrame(records, columns=['plan_kwh', 'charge_kwh', 'discharge_kwh', 'spill_kwh', 'soc_start_kwh', 'soc_end_kwh', 'load_kwh', 'pv_kwh', 'emergency_kwh', 'price'])
    audit_trace(f, initial=initial)
    assert abs(float(price[f.plan_kwh]) - float(values[0].at(initial))) < 1e-6
    return f, values

def recourse_values(net, price, terminal_rate):
    """A historical path's perfect-information value; used only as an approximation.

    Emergency supply serves deficits only; it cannot charge storage. Surplus
    charges greedily, which is optimal for these nonincreasing future values.

```

```

"""
n=len(net);out=[None]*(n+1)
out[n]=Value(np.array([B.minimum,6000.,B.maximum]),terminal_rate*np.array([4800.,0.,0.]))
for t in range(n-1,-1,-1):
    if net[t]>0:
        top=min(float(net[t]),B.limit)/B.eta_d
        h=Value(np.array([0.,top]),5*price[t]*np.array([net[t],net[t]-B.eta_d*top]))
        out[t]=convolve(out[t+1],h)
    else:
        a=min(-float(net[t]),B.limit)*B.eta_c;f=out[t+1]
        x=np.unique(np.r_[B.minimum,np.clip(f.x-a,B.minimum,B.maximum),B.maximum])
        out[t]=restrict(Value(x,f.at(np.minimum(B.maximum,x+a))))
return out

def execute_value(plan,load,pv,initial,price,paths,terminal_rate):
    all_values=[recourse_values(net,price,terminal_rate) for net in paths]
    s=float(initial);rows=[]
    for t,(g,l,v,p) in enumerate(zip(plan,load,pv,price)):
        net=float(1-v-g);c=d=e=w=0.
        if net<=0:
            c=min(-net,B.limit,(B.maximum-s)/B.eta_c);w=-net-c;end=s+B.eta_c*c
        else:
            lo=max(B.minimum,s-min(net,B.limit)/B.eta_d);hi=s
            choices=np.unique(np.r_[lo,hi,*[f[t+1].x[(f[t+1].x>lo)&(f[t+1].x<hi)] for f in all_values]])
            costs=5*p*(net-B.eta_d*(s-choices))+np.mean([f[t+1].at(choices) for f in all_values],axis=0)
            end=float(choices[np.argmin(costs)]);d=B.eta_d*(s-end);e=max(0,net-d)
            rows.append([g,l,v,c,d,e,w,s,end,p]);s=end
f=pd.DataFrame(rows,columns=['plan_kwh','load_kwh','pv_kwh','charge_kwh','discharge_kwh','emergency_kwh','spill_kwh','soc_start_kwh','soc_end_kwh','price'])
audit_trace(f,initial=initial);return f

```

## src/v5b\_experiments.py

```

"""Screenshot-inspired tests: exact deterministic DP and causal Q2 recourse."""
from pathlib import Path
import argparse,time,json,hashlib
import numpy as np
import pandas as pd
from src.q12 import forecasts,optimize_day,reserve,execute_day,summarize_day,dump_json,audit_trace,B
from src.q34_data import read_extended
from src.v5b_value import deterministic,execute_value

OUT=Path('artifacts/v5b')
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-
root',required=True);a=ap.parse_args();OUT.mkdir(parents=True,exist_ok=True)
    design=dict(candidates=['greedy','value_zero','value_night'],scenario_days=28,risk_quantile=.7,
        terminal_rates={'value_zero':0,'value_night':'first-slot fixed price / charging efficiency'},
        validation=[21,31],evaluation=[31,365],selection='January realized fee; greedy wins ties',
        scenario='Paired historical load and PV residuals from completed days; nonnegative reconstructed
levels',
        note='Mean historical perfect-information path values approximate continuation cost; causal action
is not exact stochastic DP')
    dump_json(OUT/'design_before_evaluation.json',design)
    data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.))
    L,V,p=[np.roll(data[k],1) for k in ['q1_load','q1_pv','price']]
    f,vals=deterministic(L,V,p);f.to_csv(OUT/'q1_dp_intervals.csv',index=False,float_format='%.17g')
    _,m=optimize_day(L,V,p,6000,(6000,'equal'));assert abs(m['cost']-p@f.plan_kwh)<1e-6
    checks=[]
    for s in [1200.,2400.,3600.,4800.,6000.,7200.,8400.,9600.,10800.]:
        _,ref=optimize_day(L,V,p,s,(6000,'equal'))
        val=float(vals[0].at(s));assert abs(val-ref['cost'])<1e-5
        checks.append(dict(initial_soc=s,dp_cost=val,milp_cost=ref['cost']))
    pd.DataFrame(checks).to_csv(OUT/'q1_state_value_check.csv',index=False)

pd.DataFrame({'soc_kwh':vals[0].x,'future_cost_yuan':vals[0].y}).to_csv(OUT/'q1_initial_value.csv',index=Fa
se)
    print('Q1 continuous DP',float(p@f.plan_kwh),len(vals[0].x),'breakpoints',flush=True)
    def calc(name,start,stop,retain):
        s=6244.256891388887 if start==21 else 9902.811287870369;days=[];frames=[];tick=time.perf_counter()
        for day in range(start,stop):
            r=reserve(data,pred,day,.7);nominal,_=optimize_day(pred['load'][day]+r,pred['pv'][day],data['price'],s)
            plan=nominal.plan_kwh.to_numpy()
            if name=='greedy':f=execute_day(plan,data['load'][day],data['pv'][day],s,data['price'])
            else:
                left=max(7,day-28);l=np.maximum(0,pred['load'][day]+data['load'][left:day]-
pred['load'][left:day]);v=np.maximum(0,pred['pv'][day]+data['pv'][left:day]-pred['pv'][left:day])
                paths=l-v-plan
                rate=0. if name=='value_zero' else float(data['price'][0]/B.eta_c)
                f=execute_value(plan,data['load'][day],data['pv'][day],s,data['price'],paths,rate)
                s=float(f.soc_end_kwh.iloc[-1]);days.append(summarize_day(f,data['dates'][day],name,0))
            if
retain:f.insert(0,'date',str(data['dates'][day].date()));f.insert(1,'slot',range(144));frames.append(f)
            if retain and data['dates'][day].day==1:print(name,str(data['dates'][day].date()),flush=True)

        daily=pd.DataFrame(days);summary=dict(candidate=name,cost=float(daily.total_cost_yuan.sum()),emergency_kwh=f
loat(daily.emergency_kwh.sum()),final_soc=s,seconds=time.perf_counter()-tick)
        if retain:
            trace=pd.concat(frames,ignore_index=True);summary['physical']=audit_trace(trace,initial=9902.811287870369)

        daily.to_csv(OUT/f'q2_{name}_daily.csv',index=False,float_format='%.17g');trace.to_csv(OUT/f'q2_{name}_inter
vals.csv',index=False,float_format='%.17g')
        return summary
        validation=[calc(name,21,31,False) for name in design['candidates']];best=min(validation,key=lambda
r:(r['cost'],r['candidate']!= 'greedy'))['candidate']

    dump_json(OUT/'validation.json',validation);dump_json(OUT/'selection.json',{'candidate':best});print('Januar
y',validation,best,flush=True)
    results=[calc(name,31,365,True) for name in
design['candidates']];dump_json(OUT/'summary.json',results);print(results,flush=True)

    dump_json(OUT/'source_manifest.json',{str(f):hashlib.sha256(f.read_bytes().replace(b'\r\n',b'\n')).hexdigest
() for f in [Path(__file__),Path('src/v5b_value.py')])
    if __name__=='__main__':main()

```



## src/v5b\_rolling.py

```

"""State-value control in the remaining three cases, with causal prices."""
from pathlib import Path
import argparse,json,time
import numpy as np
import pandas as pd
from src.q12 import B,forecasts,audit_trace,execute_day,dump_json
from src.q34_data import read_extended
from src.v3_model import solve,risk_buffers
from src.v5_model import adaptive_load
from src.v5b_value import recourse_values

OUT=Path('artifacts/v5b')
def execute_prefix(plan,load,pv,initial,actual_price,forecast_price,paths,steps,release_aware=False):
    """Future values use forecast prices; only executed slots read real prices."""
    continuation_price=np.array(forecast_price,copy=True)
    # Beyond the next release, additional ordinary contracts cost 1.5p rather
    # than 5p. This is a value-function approximation, not a new billing rule.
    if release_aware:continuation_price[steps:]*=.3
    values=[recourse_values(path,continuation_price,0.) for path in paths];s=float(initial);rows=[]
    for t in range(steps):
        g,l,v,p=plan[t],load[t],pv[t],actual_price[t];net=float(1-v-g);c=d=e=w=0.
        if net<=0:c=min(-net,B.limit,(B.maximum-s)/B.eta_c);w=-net-c;end=s+B.eta_c*c
        else:
            lo=max(B.minimum,s-
min(net,B.limit)/B.eta_d);choices=np.unique(np.r_[lo,s,*[f[t+1].x[(f[t+1].x>lo)&(f[t+1].x<s)] for f in
values]])
            costs=5*p*(net-B.eta_d*(s-choices))+np.mean([f[t+1].at(choices) for f in values],axis=0)
            end=float(choices[np.argmin(costs)]);d=B.eta_d*(s-end);e=max(0,net-d)
            rows.append([g,l,v,c,d,e,w,s,end,p]);s=end

f=pd.DataFrame(rows,columns=['plan_kwh','load_kwh','pv_kwh','charge_kwh','discharge_kwh','emergency_kwh','sp
ill_kwh','soc_start_kwh','soc_end_kwh','price'])
    audit_trace(f,initial=initial);return f

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);ap.add_argument('--
case',choices=['q3','q42','q43'],required=True);ap.add_argument('--extra-
only',action='store_true');a=ap.parse_args()
    case=a.case;rolling=case!='q42';q=.5 if rolling else .7
    controls=['greedy','value','value_release'] if rolling else ['greedy','value']
    dump_json(OUT/f'{case}_design.json',dict(candidates=controls,selection='January cost; greedy wins ties',
    quantile=q,terminal=6000,scenario_days=28,terminal_value_rate=0.,
    load_selector='Retain earlier V5 causal monthly load selections as a shared forecast input',
    price_information='Only the executed slot sees actual prices; future value uses current-release
price forecast',
    release_extension='Added after January frozen-contract value performed poorly; assumes future
shortage can enter a 1.5p adjusted contract after next issue. Actual bills unchanged.))

data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1,1));base=np.repeat(pred['load'][:,None,
:],4,axis=1)
    loads={'baseline':base,'adaptive':adaptive_load(data,pred['load'])[0]}
    pv=data['pv_issued'] if rolling else np.repeat(pred['pv'][:,None,:],4,axis=1)
    rr={k:risk_buffers(data,ll,pv,q) for k,ll in loads.items()}
    pp=None if case=='q3' else np.load('artifacts/q34/price_forecasts.npz')['ridge' if case=='q42' else
'seasonal']
    selections=pd.read_csv('artifacts/v5/online_selection.csv');mode=3 if case=='q3' else 4
    choices={int(r.month):r.candidate for r in
selections[(selections['mode']==mode)&selections.selected].itertuples())
    def run(control,start,stop,retain):
        state=6244.25689138887 if start==21 else
9902.811287870369;frames=[];daily=[];versions=[];tick=time.perf_counter()
        for day in range(start,stop):
            date=str(data['dates'][day].date());month=int(data['dates'][day].month);name=choices.get(month,'baseline')
            if rolling else 'baseline'
                ll=loads[name];buffer=rr[name];p0=data['price'] if pp is None else
pp[day,0,:144];actual=data['price'] if pp is None else data['actual_price'][day]

nominal,_=solve(ll[day,0]+buffer[day,0],pv[day,0],p0,state);original=nominal.plan_kwh.to_numpy().copy();plan
=original.copy();blocks=[]
                for ver in (range(4) if rolling else [0]):
                    first=ver*36;steps=36 if rolling else 144;p=data['price'][first:] if pp is None else
pp[day,ver,:144-first]

```

```

        if ver:

old=plan[first:].copy();nominal,_=solve(ll[day,ver,first:]+buffer[day,ver,first:],pv[day,ver,first:],p,state
,old=original[first:]);plan[first:]=np.maximum(nominal.plan_kwh,0)
        if retain:
            for j,t in
enumerate(range(first,144)):versions.append([date,ver,first,t,float(old[j]),float(plan[t]),float(p[j]),float
(ll[day,ver,t]),float(pv[day,ver,t]),float(buffer[day,ver,t]),state])
            if
control=='greedy':f=execute_day(plan[first:first+steps],data['load'][day,first:first+steps],data['pv'][day,f
irst:first+steps],state,actual[first:first+steps])
            else:
                left=max(7,day-28)
                L=np.maximum(0,ll[day,ver,first:]+data['load'][left:day,first:]-ll[left:day,ver,first:])
                V=np.maximum(0,pv[day,ver,first:]+data['pv'][left:day,first:]-pv[left:day,ver,first:])

f=execute_prefix(plan[first:],data['load'][day,first:],data['pv'][day,first:],state,actual[first:],p,L-V-
plan[first:],steps,control=='value_release')
                state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',range(144));f['original_plan
_kwh']=original
                f['increase_kwh']=np.maximum(plan-original,0);f['decrease_kwh']=np.maximum(original-plan,0)

f['original_cost_yuan']=actual*original;f['increase_cost_yuan']=1.5*actual*f.increase_kwh;f['decrease_cost_y
uan']=-.5*actual*f.decrease_kwh;f['emergency_cost_yuan']=5*actual*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
                daily.append(dict(date=date,soc_start_kwh=float(f.soc_start_kwh.iloc[0]),soc_end_kwh=state,
                **{k:float(f[k].sum()) for k in
['total_cost_yuan','original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','pla
n_kwh','emergency_kwh','charge_kwh','discharge_kwh','spill_kwh']}))
                if retain:frames.append(f)
                if retain and data['dates'][day].day==1:print(case,control,date,flush=True)

d=pd.DataFrame(daily);s=dict(case=case,control=control,cost=float(d.total_cost_yuan.sum()),final_soc=state,e
mergency_kwh=float(d.emergency_kwh.sum()),seconds=time.perf_counter()-tick)
                if retain:

f=pd.concat(frames,ignore_index=True);s['physical']=audit_trace(f,initial=9902.811287870369);d.to_csv(OUT/f'
{case}_{control}_daily.csv',index=False,float_format='%.17g');f.to_csv(OUT/f'{case}_{control}_intervals.csv.
gz',index=False,float_format='%.17g')
                if
versions=pd.DataFrame(versions,columns=['date','version','effective_slot','slot','old_kwh','new_kwh','foreca
st_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision_soc_kwh']).to_csv(OUT/f'{case}_{cont
rol}_versions.csv.gz',index=False,float_format='%.17g')
                return s
                prior=json.loads((OUT/f'{case}_validation.json').read_text(encoding='utf-8'))['validation'] if
a.extra_only else []
                selected_controls=['value_release'] if a.extra_only else controls
                val=prior+[run(c,21,31,False) for c in selected_controls];chosen=min(val,key=lambda
x:(x['cost'],x['control']!='greedy'))['control']

dump_json(OUT/f'{case}_validation.json',{'validation':val,'selected':chosen});print(case,'January',val,chose
n,flush=True)
                prior=json.loads((OUT/f'{case}_summary.json').read_text(encoding='utf-8')) if a.extra_only else []
                results=prior+[run(c,31,365,True) for c in
selected_controls];dump_json(OUT/f'{case}_summary.json',results);print(case,results,flush=True)
if __name__=='__main__':main()

```

## src/v5b\_windows.py

```
"""Training-window sensitivity, separated from policy selection."""
from pathlib import Path
import inspect, argparse
import numpy as np
import pandas as pd
import src.q12 as q12
from src.q34_data import read_extended

def window_forecasts(data, window):
    # Reuse the full checked forecaster, changing exactly one training horizon.
    source=inspect.getsource(q12.forecasts).replace('d-60', 'd-'+str(int(window)))
    namespace=dict(vars(q12));exec(compile(source, '<window_forecasts>', 'exec'), namespace)
    return namespace['forecasts'](data, 'ridge', (1., 1.))[0]

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root', required=True);a=ap.parse_args()
    out=Path('artifacts/v5b');data,_=read_extended(a.data_root);rows=[]
    q12.dump_json(out/'window_design.json', {'windows_days': [28, 56, 60, 84], 'same_model': 'ridge alpha 1; weekly
refit; all other features unchanged',
    'interpretation': 'Sensitivity only; January cannot distinguish these windows because training starts
on day 7. Do not pick lowest annual result as a validated policy.'})
    for window in [28, 56, 60, 84]:
        pred=window_forecasts(data, window)
        val=q12.backtest(data, pred, 21, 31, 6244.256891388887, .7, 'validation', retain=False)[0]
        daily=q12.backtest(data, pred, 31, 365, 9902.811287870369, .7, 'window', retain=False)[0]

    row=dict(window_days=window, validation_cost=float(val.total_cost_yuan.sum()), annual_cost=float(daily.total_c
ost_yuan.sum()),
        load_mae_kwh=float(abs(pred['load'])[31:]-
data['load'])[31:]).mean()), pv_mae_kwh=float(abs(pred['pv'])[31:]-data['pv'])[31:]).mean()))
        rows.append(row);print(row, flush=True)
    pd.DataFrame(rows).to_csv(out/'window_sensitivity.csv', index=False)
if __name__=='__main__':main()
```

## src/v5b\_analysis.py

```

"""Recompute bills from raw inputs, audit revisions and quantify paired gains."""
from pathlib import Path
import argparse, json, hashlib
import numpy as np
import pandas as pd
from src.q12 import audit_trace, dump_json
from src.q34_data import read_extended
from src.v5_audit import independent_lp
OUT=Path('artifacts/v5b')
SELECTED={'2':'q2_value_zero', '3':'q3_value_release', '4-2':'q42_value', '4-3':'q43_value_release'}
def canonical(name):
    f=pd.read_csv(OUT/f'{name}_intervals.csv.gz')
    if 'original_plan_kwh' not in f:
        f['original_plan_kwh']=f.plan_kwh
        f['original_cost_yuan']=f.price*f.plan_kwh
        f['increase_cost_yuan']=0.; f['decrease_cost_yuan']=0.
        f['emergency_cost_yuan']=5*f.price*f.emergency_kwh
        f['total_cost_yuan']=f.original_cost_yuan+f.emergency_cost_yuan
    return f
def main():
    ap=argparse.ArgumentParser(); ap.add_argument('--data-root', required=True); a=ap.parse_args()
    data,_=read_extended(a.data_root); audits=[]; certs=[]
    names=[p.name.removesuffix('_intervals.csv.gz') for p in OUT.glob('*_intervals.csv.gz')]
    for name in sorted(names):
        if name.endswith('_selected'): continue
        f=canonical(name); assert len(f)==334*144
        assert not f[['date', 'slot']].duplicated().any()
        assert f.date.tolist()==np.repeat(data['dates'][31:].strftime('%Y-%m-%d'), 144).tolist()
        np.testing.assert_array_equal(f.slot, np.tile(np.arange(144), 334))
        for col, key in
            [ ('load_kwh', 'load'), ('pv_kwh', 'pv') ]: np.testing.assert_allclose(f[col], data[key][31:].reshape(-1), rtol=0, atol=1e-10)
        p=np.tile(data['price'], 334) if name.startswith(('q2_', 'q3_')) else
        data['actual_price'][31:].reshape(-1)
        np.testing.assert_allclose(f.price, p, atol=1e-12, rtol=0)
        fee=p*(np.minimum(f.original_plan_kwh, f.plan_kwh)+.5*np.maximum(f.original_plan_kwh-
        f.plan_kwh, 0)+1.5*np.maximum(f.plan_kwh-f.original_plan_kwh, 0)+5*f.emergency_kwh)
        np.testing.assert_allclose(fee, f.total_cost_yuan, atol=1e-7, rtol=0)

    d=pd.read_csv(OUT/f'{name}_daily.csv'); np.testing.assert_allclose(fee.groupby(f.date).sum(), d.total_cost_yuan, atol=1e-6, rtol=0)
    physical=audit_trace(f, initial=9902.811287870369)
    vp=OUT/f'{name}_versions.csv.gz'
    if vp.exists():
        v=pd.read_csv(vp)
        for date, g in f.groupby('date', sort=False):
            plan=g.original_plan_kwh.to_numpy().copy()
            for ver in [1, 2, 3]:
                z=v[(v.date==date)&(v.version==ver)]; slots=z.slot.to_numpy(int)
                np.testing.assert_array_equal(slots, np.arange(ver*36, 144))
                np.testing.assert_allclose(plan[slots], z.old_kwh, atol=1e-8, rtol=0)
                assert abs(z.decision_soc_kwh.iloc[0]-g.soc_start_kwh.iloc[ver*36])<1e-8
                plan[slots]=z.new_kwh
                np.testing.assert_allclose(plan, g.plan_kwh, atol=1e-8, rtol=0)
            if name in SELECTED.values():
                for date in ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']:
                    for ver in [1, 2, 3]:
                        z=v[(v.date==date)&(v.version==ver)]; g=f[(f.date==date)&(f.slot>=ver*36)]

    c=independent_lp((z.forecast_load_kwh+z.reserve_kwh).to_numpy(), z.forecast_pv_kwh.to_numpy(), z.forecast_price.to_numpy(), float(z.decision_soc_kwh.iloc[0]), old=g.original_plan_kwh.to_numpy())
    # The recorded future plan is a nominal optimizer; subsequent
    # execution and reoptimizations need not follow its SOC path.
    delta=z.new_kwh.to_numpy()-g.original_plan_kwh.to_numpy()
    recorded=float(z.forecast_price.to_numpy())@(1.5*np.maximum(delta, 0)-.5*np.maximum(-delta, 0))
    assert abs(recorded-c['objective'])<1e-5
    c.update(case=name, date=date, version=ver, recorded_difference=abs(recorded-c['objective'])); certs.append(c)
    audits.append(dict(name=name, cost=float(fee.sum()), intervals=len(f), physical=physical))
    gains=[]; monthly=[]; rng=np.random.default_rng(20260912)
    for key, name in SELECTED.items():
        base=canonical(name.split('_')[0]+'_greedy'); f=canonical(name)
        assert abs(f.soc_end_kwh.iloc[-1]-base.soc_end_kwh.iloc[-1])<1e-7
        diff=(base.total_cost_yuan-f.total_cost_yuan).groupby(f.date).sum().to_numpy(); boots=[]
        for block in [7, 14, 28]:

```

```

idx=(rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/block))))[:, :,None]+np.arange(block))%len(diff)
sums=diff[idx.reshape(2000,-1)[:, :len(diff)]].sum(axis=1)
lo,hi=np.quantile(sums,[.025,.975]);boots.append(dict(block=block,low=float(lo),high=float(hi)))

record=dict(question=key,selected=name,baseline_cost=float(base.total_cost_yuan.sum()),cost=float(f.total_cost_yuan.sum()),saving=float(diff.sum()),percent=float(diff.sum()/base.total_cost_yuan.sum()*100),bootstrap=boots,initial_soc=float(f.soc_start_kwh.iloc[0]),final_soc=float(f.soc_end_kwh.iloc[-1]))
for label,frame in [('before',base),('after',f)]:
    record[label]={c:float(frame[c].sum()) for c in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','emergency_kwh','spill_kwh']}
    gains.append(record)
    for month in range(2,13):
        mask=pd.to_datetime(f.date).dt.month==month
        monthly.append(dict(question=key,month=month,saving=float((base.total_cost_yuan-f.total_cost_yuan)[mask].sum()))))
        f.to_csv(OUT/f'{name}_selected_intervals.csv.gz',index=False,float_format='%.17g')
    periodic=[]
    for key in ['load','pv']:
        x=data[key]
        for lag in
[1,7,14,28]:periodic.append(dict(series=key,lag_days=lag,slot_correlation=float(np.corrcoef(x[lag:].ravel(),x[:-lag].ravel())[0,1]),daily_energy_correlation=float(np.corrcoef(x[lag:].sum(1),x[:-lag].sum(1))[0,1])))
    pd.DataFrame(periodic).to_csv(OUT/'periodicity.csv',index=False)

pd.DataFrame(monthly).to_csv(OUT/'monthly_savings.csv',index=False);pd.DataFrame(certs).to_csv(OUT/'new_solver_certificates.csv',index=False)
dump_json(OUT/'improvement.json',gains)
dump_json(OUT/'verification.json',dict(status='PASS',traces=audits,total_intervals=sum(x['intervals'] for x in audits),new_independent_lp_cases=len(certs),max_duality_gap=max(x['gap'] for x in certs),inputs={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in Path(a.data_root,'附件').glob('附件*.xlsx')},sources={str(p):hashlib.sha256(p.read_bytes().replace(b'\r\n',b'\n')).hexdigest() for p in Path('src').glob('v5b_*.py')}))
print('PASS',len(audits),'full traces;',len(certs),'independent LP certificates');print([(r['question'],r['cost'],r['saving']) for r in gains])
if __name__=='__main__':main()

```

## src/v5b\_delivery.py

```
"""Write V5 data cells into V3 layouts and read every written cell back."""
from pathlib import Path
from datetime import datetime
import json,shutil,math,hashlib
from copy import copy
import pandas as pd
import openpyxl
from src.q12 import dump_json
from src.deliver_q12 import blocks,emergency_runs

OUT=Path('artifacts/v5b')
from src.v5b_analysis import SELECTED
SOURCES={'1':'artifacts/v3/q1_intervals.csv','**{k:str(OUT/f'{v}_selected_intervals.csv.gz') for k,v in
SELECTED.items()}}

def main():
    audit=[]
    for key in ['1']:shutil.copy2(f'artifacts/v3/result{key}.xlsx',OUT/f'result{key}.xlsx')
    for key,label in SELECTED.items():
        f=pd.read_csv(SOURCES[key]);v=pd.read_csv(OUT/f'{label}_versions.csv.gz') if key in ['3','4-3'] else
pd.DataFrame();payload={k:[] for k in (['计划购电量','调整购电量','充放电量','紧急购电量','费用分解','计划版本']
if key in ['3','4-3'] else ['计划购电量','充放电量','紧急购电量']+['费用分解'] if key=='4-2' else []))
        for date,g in f.groupby('date',sort=False):
            dt=datetime.strptime(date,'%Y-%m-%d')
            payload['计划购电量']
            '.append([dt,*g.original_plan_kwh.tolist(),float(g.original_plan_kwh.sum()),float(g.original_cost_yuan.sum(
)) if key in ['3','4-3'] else float(g.total_cost_yuan.sum())])
            if '调整购电量' in payload:payload['调整购电量']
            '.append([dt,*g.plan_kwh.tolist(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
            for i,b in enumerate(blocks(g)):
                payload['充放电量'].append([dt if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else
None,float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else None])
                for i,b in enumerate(emergency_runs(g) or [['无',0]]):payload['紧急购电量'].append([dt if i==0
else None,*b])
                if '费用分解' in payload:payload['费用分解'].append([dt,*[float(g[c].sum()) for c in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
                for (date,ver),g in (v.groupby(['date','version'],sort=True) if len(v) else []):
                    values=[None]*144
                    for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                    payload['计划版本'].append([datetime.strptime(date,'%Y-%m-%d'),f'{int(ver)*6:02d}:00',*values])
                    wb=openpyxl.load_workbook(f'artifacts/v5/result{key}.xlsx');assert not any(c.data_type=='f' for ws
in wb for row in ws for c in row),'Formula-bearing template needs recalculation'
                    headers={ws.title:tuple(c.value for c in ws[1]) for ws in wb};sheets=wb.sheetnames
                    for name,rows in payload.items():
                        ws=wb[name];end=ws.max_row;columns=len(rows[0])
                        for row in ws.iter_rows(min_row=2,max_row=max(end,len(rows)+1),max_col=columns):
                            for cell in row:cell.value=None
                        for i,row in enumerate(rows,2):
                            for j,value in enumerate(row,1):
                                cell=ws.cell(i,j);cell.value=value
                                if i>end:cell._style=copy(ws.cell(2,j)._style)
                    if '口径说明' in wb:wb['口径说明']['B2']='V5 复核更新: 过去 28 日配对残差价值控制; 日前按固定合同, 滚动按后续
可调单成本估计库存价值; 实际费用规则不变。'
                    dest=OUT/f'result{key}.xlsx';wb.save(dest);wb.close()
                    checked=openpyxl.load_workbook(dest,read_only=True,data_only=True);count=0
                    assert checked.sheetnames==sheets
                    for name in sheets:assert tuple(next(checked[name].values))==headers[name]
                    for name,rows in payload.items():
                        for actual,expected in
zip(checked[name].iter_rows(min_row=2,max_row=len(rows)+1,max_col=len(rows[0]),values_only=True),rows):
                            for a,e in zip(actual,expected):
                                count+=1
                                if isinstance(e,(int,float)):assert isinstance(a,(int,float)) and
math.isclose(a,e,abs_tol=1e-7,rel_tol=1e-11),(name,a,e)
                                else:assert a==e,(name,a,e)
                            for row in checked[name].iter_rows(min_row=len(rows)+2,values_only=True):assert all(v is None
for v in row)

checked.close();audit.append(dict(key=key,cells_compared=count,headers_preserved=True,formula_cells=0,sha256
=hashlib.sha256(dest.read_bytes()).hexdigest()))
    dump_json(OUT/'delivery_selection.json',{'sources':SOURCES,'main':'January-selected historical-path
value controllers; Q1 unchanged','copied_byte_identical':['1']})

dump_json(OUT/'xlsx_audit.json',dict(status='PASS',updated=audit,copied={k:hashlib.sha256((OUT/f'result{k}.x
```

```
lsx').read_bytes()).hexdigest() for k in ['1']});print(audit)
if __name__=='__main__':main()
```

## src/v7\_audit.py

```

"""Unified time convention, independent value audit and operational flexibility."""
from pathlib import Path
from dataclasses import replace
import argparse, json, hashlib
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from src.q12 import B, read_inputs, optimize_day, audit_trace, dump_json, interval_label
from src.v3_model import solve
from src.v5b_value import deterministic, recourse_values
from src.v5_audit import independent_lp
OUT=Path('artifacts/v7')
def path_lp(net,p,s,terminal_rate):
    n=len(net);I=np.eye(n);Z=np.zeros((n,n));T=np.tril(np.ones((n,n)))
    # c,d,e,w plus terminal shortfall; fixed surplus/deficit sign rules.
    eq=np.hstack([-I,I,I,-I,np.zeros((n,1))]);soc=np.hstack([B.eta_c*T,-T/B.eta_d,Z,Z,np.zeros((n,1))])
    ub=np.vstack([soc,-soc,np.r_-soc[-1,-1],-1.])
    rhs=np.r_[np.full(n,B.maximum-s),np.full(n,s-B.minimum),s-6000]
    bounds=[]
    for variable in range(4):
        for a in net:
            maximum=(min(-a,B.limit) if a<=0 else 0) if variable==0 else (min(a,B.limit) if a>0 else 0) if
variable==1 else (max(a,0) if variable==2 else max(-a,0))
            bounds.append((0,maximum))
    bounds.append((0,None));c=np.r_[np.zeros(2*n),5*p,np.zeros(n),terminal_rate]
    r=linprog(c,A_eq=eq,b_ub=net,A_ub=ub,b_ub=rhs,bounds=bounds,method='highs');assert r.success
    return float(r.fun)
def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-
root',required=True);a=ap.parse_args();OUT.mkdir(exist_ok=True,parents=True)
    D,inputs=read_inputs(a.data_root);L,V,p=[D[k] for k in ['q1_load','q1_pv','price']]
    f,m=solve(L,V,p,6000,equal=True);dp,val=deterministic(L,V,p)
    _,milp=optimize_day(L,V,p,6000,(6000,'equal'));cert=independent_lp(L,V,p,6000,equal=True)
    assert max(abs(m['objective']-x) for x in [p@dp.plan_kwh,milp['cost'],cert['objective']])<1e-6
    f.insert(0,'slot',range(144));f.insert(1,'interval',[interval_label(i) for i in
range(144)]);f.to_csv(OUT/'q1_intervals.csv',index=False,float_format='%.17g')
    dp.to_csv(OUT/'q1_dp_intervals.csv',index=False,float_format='%.17g')
    pd.DataFrame({'soc_kwh':val[0].x,'cost_yuan':val[0].y}).to_csv(OUT/'q1_value.csv',index=False)
    no=float(p@np.maximum(L-V,0));savings=no-m['objective'];states=[]
    for s in np.arange(1200,10801,600):
        _,r=optimize_day(L,V,p,float(s),(6000,'equal'))
        error=abs(float(val[0].at(s))-r['cost']);assert error<1e-
5;states.append(dict(initial_soc=s,dp=float(val[0].at(s)),milp=r['cost'],error=error))
    pd.DataFrame(states).to_csv(OUT/'state_checks.csv',index=False)
    # Power remains 5000 kW. Increase usable capacity symmetrically around the
    # same 6000-kWh initial and terminal state, avoiding free initial energy.
    capacity=[]
    for width in [0.,2400.,4800.,7200.,9600.,12000.]:
        b=replace(B,minimum=6000-width/2,maximum=6000+width/2)
        _,r=optimize_day(L,V,p,6000,(6000,'equal'),battery=b)
        capacity.append(dict(usable_capacity_kwh=width,cost=r['cost'],flexibility_value=no-r['cost']))
    pd.DataFrame(capacity).to_csv(OUT/'capacity_value.csv',index=False)
    assert all(capacity[i]['cost']>=capacity[i+1]['cost']-1e-6 for i in range(len(capacity)-1))
    assert abs(capacity[0]['cost']-no)<1e-6
    slopes=np.diff([r['flexibility_value'] for r in capacity])/2400
    assert np.all(np.diff(slopes)<=1e-6)
    efficiency=[]
    for eta in [.8,.85,.9,np.sqrt(.9),1.]:
        _,r=optimize_day(L,V,p,6000,(6000,'equal'),battery=replace(B,eta_c=eta,eta_d=eta))
        efficiency.append(dict(single_efficiency=eta,roundtrip_efficiency=eta**2,cost=r['cost']))
    pd.DataFrame(efficiency).to_csv(OUT/'q1_efficiency.csv',index=False)
    rng=np.random.default_rng(20260912);path_checks=[]
    for i in range(48):
        n=[3,12,24,144][i%4];net=rng.uniform(-
1300,1600,n);price=rng.uniform(.0076,1.7936,n);s=float(rng.uniform(1200,10800));rate=[0.,.5,2.][i%3]
        value=float(recourse_values(net,price,rate)[0].at(s));ref=path_lp(net,price,s,rate);error=abs(value-
ref)
        assert error<1e-
5,(i,error);path_checks.append(dict(case=i,n=n,initial=s,terminal_rate=rate,dp=value,lp=ref,error=error))
    pd.DataFrame(path_checks).to_csv(OUT/'path_value_checks.csv',index=False)

summary=dict(cost=m['objective'],no_storage_cost=no,flexibility_value=savings,saving_percent=100*savings/no,
plan_kwh=float(f.plan_kwh.sum()),
initial_marginal_value=float((val[0].at(6000)-
val[0].at(6001))),breakpoints=len(val[0].x),physical=audit_trace(f,initial=6000),lp_certificate=cert,
interval_convention='All questions: input timestamp is right endpoint of a ten-minute mean interval;

```



```

no cyclic input rotation. Q1 export alone maps its periodic actions to unchanged template start labels.',
    previous_v5_cost=35126.84858928963,time_convention_cost_change=m['objective']-35126.84858928963)
    dump_json(OUT/'q1_summary.json',summary)

dump_json(OUT/'model_audit.json',dict(status='PASS',q1_state_tests=len(states),random_mixed_path_tests=len(p
ath_checks),max_path_error=max(x['error'] for x in path_checks),

raw_inputs=inputs['input_files'],capacity_concavity_pass=True,all_existing_stochastic_strategies='Unchanged
V5b traces; reevaluated
separately',sources={str(Path(__file__)):hashlib.sha256(Path(__file__).read_bytes().replace(b'\r\n',b'\n')).
hexdigest()})
    print(summary);print('48 mixed-sign historical-path LP checks passed')
if __name__=='__main__':main()

```

## src/v7\_delivery.py

```
"""Preserve workbook layouts; explicitly map Q1 periodic template labels."""
from pathlib import Path
import shutil,hashlib
import pandas as pd
import openpyxl
from src.q12 import dump_json
from src.deliver_q12 import blocks
OUT=Path('artifacts/v7')
SOURCES={'1':str(OUT/'q1_intervals.csv'),'2': 'artifacts/v5b/q2_value_zero_selected_intervals.csv.gz',
'3': 'artifacts/v5b/q3_value_release_selected_intervals.csv.gz','4-2': 'artifacts/v5b/q42_value_selected_intervals.csv.gz','4-3': 'artifacts/v5b/q43_value_release_selected_intervals.csv.gz'}
def main():
    copied={}
    for key in ['2','3','4-2','4-3']:
        source=Path(f'artifacts/v5b/result{key}.xlsx');dest=OUT/source.name;shutil.copy2(source,dest);copied[key]=hashlib.sha256(dest.read_bytes()).hexdigest()
        f=pd.read_csv(SOURCES['1']);wb=openpyxl.load_workbook('artifacts/v5b/result1.xlsx');ws=wb['计划购电量'];labels=[r[0].value for r in ws.iter_rows(min_row=2)]
        expected=[]
        for i in range(1,145):
            value=float(f.plan_kwh.iloc[i%144]);ws.cell(i+1,2).value=value;expected.append(value)
        ws=wb['充放电量']
        for i,b in enumerate(blocks(f),2):
            ws.cell(i,2).value=b[1];ws.cell(i,3).value=b[2]
            ws.cell(2,5).value=float(f.soc_start_kwh.iloc[0]);ws.cell(3,5).value=float(f.soc_end_kwh.iloc[-1])
        wb.save(OUT/'result1.xlsx');wb.close();w=openpyxl.load_workbook(OUT/'result1.xlsx',read_only=True,data_only=True)
        rows=list(w['计划购电量'].values)[1:];assert [r[0] for r in rows]==labels
        for row,value in zip(rows,expected):assert abs(row[1]-value)<1e-8
        assert abs(sum(r[1] for r in rows)-f.plan_kwh.sum())<1e-7
        b=blocks(f)
        for i,row in enumerate(list(w['充放电量'].values)[1:]):assert abs(row[1]-b[i][1])<1e-7 and abs(row[2]-b[i][2])<1e-7
        w.close();dump_json(OUT/'delivery_selection.json',dict(sources=SOURCES,Q1='Unified right-end interval means; unchanged template start labels map slots 1..143 then repeated slot0',other_questions='byte-identical V5b selected workbooks'))
    dump_json(OUT/'xlsx_audit.json',dict(status='PASS',q1_plan_cells_checked=144,q1_storage_cells_checked=14,unchanged_workbooks=copied,q1_sha256=hashlib.sha256((OUT/'result1.xlsx').read_bytes()).hexdigest()))
    print('Five V7 workbooks checked')
if __name__=='__main__':main()
```

## src/v7\_figures.py

```
"""Figures aligned with the final model; quantities, timing and valuation."""
from pathlib import Path
import sys, json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from src.q12 import dump_json
from src.v3_figures import C
sys.path.insert(0, str(Path.home()/.codex/skills/math-modeling/tools/figure/scripts'))
from export_figure import export_figure
from visual_qa import audit_layout
OUT=Path('figures/v7')
def main():
    OUT.mkdir(exist_ok=True, parents=True); audits=[]
    plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei'], 'axes.unicode_minus': False, 'font.size': 9, 'axes.spines.top': False, 'axes.spines.right': False, 'svg.fonttype': 'none', 'pdf.fonttype': 42})
    def save(fig, name, sources, claim):
        fig.canvas.draw(); a=audit_layout(fig); export_figure(fig, str(OUT/name), formats=['png', 'svg'], dpi=340, grayscale_preview=True); plt.close(fig)
        audits.append(dict(name=name, sources=sources, claim=claim, layout=a))
    def grid(ax): ax.grid(color='e6e9ed', lw=.5); ax.set_axisbelow(True)
    f=pd.read_csv('artifacts/v7/q1_intervals.csv'); edges=np.arange(145)/6
    fig, ax=plt.subplots(2, 1, figsize=(6.3, 3.1), sharex=True, layout='constrained')
    for col, label, color in [('load_kwh', '负载', C['load']), ('pv_kwh', '光伏', C['pv'])]:
        ax[col].stairs(f[col]*6, edges, baseline=None, label=label, color=color)
        ax[0].set_ylabel('功率 / kW'); ax[0].legend(frameon=False, ncol=2)
        ax[1].stairs(f[col].price, edges, baseline=None, color=C['price']); ax[1].set_ylabel='电价 / 元每 kWh', xlabel='时刻 / h', xticks=range(0, 25, 4), xlim=(0, 24))
    for a in ax: grid(a)
    save(fig, 'q1_inputs', ['artifacts/v7/q1_intervals.csv'], '统一终点区间均值的第一问输入')
    fig, ax=plt.subplots(3, 1, figsize=(6.3, 4.5), sharex=True, layout='constrained', height_ratios=[1.4, 1, .7])
    ax[0].stairs(f.plan_kwh*6, edges, baseline=None, color=C['grid'], label='普通购电')
    ax[0].stairs(f.charge_kwh*6, edges, fill=True, color=C['charge'], alpha=.4, label='充电')
    ax[0].stairs(-f.discharge_kwh*6, edges, fill=True, color=C['discharge'], alpha=.5, label='放电')
    ax[0].set_ylabel('功率 / kW'); ax[0].legend(frameon=False, ncol=3, loc='upper right')
    ax[1].plot(edges, np.r_[f.soc_start_kwh.iloc[0], f.soc_end_kwh], color=C['soc'])
    for y in [1200, 10800]: ax[1].axhline(y, color='#999', ls='--', lw=.6)
    ax[1].set_ylabel='储电量 / kWh', yticks=[1200, 6000, 10800], ylim=(500, 11500))
    ax[2].stairs(f.price, edges, baseline=None, color=C['price']); ax[2].set_ylabel='电价\n元 / kWh', xlabel='时刻 / h', xticks=range(0, 25, 4), xlim=(0, 24))
    for a in ax: grid(a)
    save(fig, 'q1_dispatch', ['artifacts/v7/q1_intervals.csv'], '第一问最优购电与可执行储能轨迹')
    v=pd.read_csv('artifacts/v7/q1_value.csv'); cap=pd.read_csv('artifacts/v7/capacity_value.csv')
    fig, ax=plt.subplots(1, 2, figsize=(6.3, 2.8), layout='constrained')
    ax[0].plot(v.soc_kwh/1000, v.cost_yuan/10000, 'o-', color=C['soc'], ms=3); ax[0].set(xlabel='初始储电量 / MWh', ylabel='后续购电费 / 万元', title='(a) 固定期末 6 MWh')
    ax[1].plot(cap.usable_capacity_kwh/1000, cap.flexibility_value/10000, 's-', color=C['grid'], ms=4)
    ax[1].scatter([9.6], [cap.flexibility_value.iloc[4]/10000], color=C['price'], zorder=3, s=30)
    ax[1].set(xlabel='可用容量 / MWh', ylabel='单日灵活性价值 / 万元', title='(b) 固定初末 6 MWh', ylim=(0, 1.6))
    for a in ax: grid(a)
    save(fig, 'storage_value', ['artifacts/v7/q1_value.csv', 'artifacts/v7/capacity_value.csv'], '库存价值与可用容量的边际收益递减')
    fig, ax=plt.subplots(2, 1, figsize=(6.3, 3.8), sharex=True, layout='constrained')
    for prefix, label, color in [('q2', '库存价值控制', C['grid'])]:
        b=pd.read_csv(f'artifacts/v5b/{prefix}_greedy_daily.csv'); n=pd.read_csv(f'artifacts/v5b/{prefix}_{"value_zero" if prefix=="q2" else "value"}_daily.csv')
        ax[0].plot(pd.to_datetime(n.date), (b.total_cost_yuan-n.total_cost_yuan).cumsum()/10000, label=label, color=color)
        ax[1].plot(pd.to_datetime(n.date), (n.emergency_kwh-b.emergency_kwh).cumsum(), label=label, color=color)
        ax[0].set_ylabel('累计节费 / 万元'); ax[0].legend(frameon=False, ncol=2)
        ax[1].set_ylabel('累计紧急量增量 / kWh'); ax[1].tick_params(axis='x', rotation=25)
        for a in ax: grid(a)
        save(fig, f'q2_value_gain', [f'artifacts/v5b/q2_value_zero_daily.csv', 'corresponding greedy daily trace'], '问题二价值控制节费并不要求紧急电量下降')
    base=pd.read_csv('artifacts/v3/q3_m0_daily.csv'); roll=pd.read_csv('artifacts/v3/q3_m7_daily.csv'); online=pd.read_csv('artifacts/v5/q3_online_daily.csv'); last=pd.read_csv('artifacts/v5b/q3_value_release_daily.csv')
    fig, ax=plt.subplots(1, 2, figsize=(6.3, 2.8), layout='constrained')
    ax[0].plot(pd.to_datetime(roll.date), (base.total_cost_yuan-
```

```

roll.total_cost_yuan).cumsum()/10000,color=C['grid']);ax[0].set(title='(a) 日内更新',ylabel='累计节费 / 万元')
    for f0,f1,label,color in [(roll,online,'负载修正',C['load']), (online,last,'库存价值',C['soc'])]:ax[1].plot(pd.to_datetime(last.date),(f0.total_cost_yuan-f1.total_cost_yuan).cumsum()/10000,label=label,color=color)
    ax[1].set(title='(b) 顺序加入的改进',ylabel='累计节费 / 万元');ax[1].legend(frameon=False)
    for a in ax:grid(a);a.tick_params(axis='x',rotation=25)

save(fig,'q3_information_value',['artifacts/v3/q3_m0_daily.csv','artifacts/v3/q3_m7_daily.csv','artifacts/v5/q3_online_daily.csv','artifacts/v5b/q3_value_release_daily.csv'],'固定预测下先比较信息更新,再顺序分解增益')

metrics=pd.read_csv('artifacts/v2/price_errors_by_issue.csv');fig,ax=plt.subplots(figsize=(6.3,2.7),layout='constrained')
    for model,label,color,marker in [(('seasonal','同期',C['base'],'o'),('ridge','岭回归',C['grid'],'s'),('gru_mean','GRU',C['soc'],'^'))]:
        z=metrics[metrics.model==model];ax.plot([0,6,12,18],z.mae,label=label,color=color,marker=marker)
    ax.set(xticks=[0,6,12,18],xlabel='发布时刻 / h',ylabel='价格 MAE / 元每kWh',ylim=(0,.075));ax.legend(ncol=3,frameon=False);grid(ax)
    save(fig,'q4_accuracy',['artifacts/v2/price_errors_by_issue.csv'],'同一发布与目标的价格预测误差')
    dump_json('artifacts/v7/figure_audit.json',audits);print('Six numerical figures exported')
if __name__=='__main__':main()

```